

Anomaly Detection in Networks with Application to Financial Transaction Networks

Andrew Elliott*

The Alan Turing Institute; Dept. of Statistics, University of Oxford

Mihai Cucuringu

The Alan Turing Institute; Dept. of Statistics, University of Oxford

Milton Martinez Luaces
Accenture Plc

Paul Reidy
Accenture Plc

Gesine Reinert

The Alan Turing Institute; Dept. of Statistics, University of Oxford

May 28, 2019

Abstract

Detecting financial fraud is a global challenge, with financial fraud losses estimated to at least GBP 768 million in the UK in 2015. Financial transactions can be represented as networks, with accounts as nodes and a directed weighted edge between a pair of nodes when one transfers money to the other. The weight of the edge denotes the transaction amount between the two parties. The task considered in this paper can be cast as a special instance of detecting anomalies in networks.

Examples of anomalies in networks depend on the type of network, but may include structures such as long paths of large transaction amounts, rings of large payments, and cliques of accounts which send money to each other. There are a number of methods available which detect such specific structures in networks. In this paper, we introduce a method which is able to detect previously unspecified anomalies in networks. The method is mainly based on a combination of features from network comparison and spectral analysis. Combining network comparison and spectral analysis tools with various local statistics, altogether give rise to a total of 140 main features. We then use a simple sum (FEATURE SUM), as well as a random forest method (RANDOM FOREST), in order to classify nodes as normal or anomalous.

We test the method first on synthetic networks which we generated, and second, on a set of synthetic networks generated without the methods team knowing the generating mechanism during the method development stage. The first set of synthetic networks was split into a training, respectively test, set of size 70%, respectively 30%, of the networks. The resulting classifier was then applied to the second set of synthetic networks as well.

We compare our method with Oddball, a widely used method for anomaly detection in networks, as well as to random classification. While Oddball outperforms random classification, both the FEATURE SUM and RANDOM FOREST methods outperform Oddball. On the test set, the RANDOM FOREST outperforms FEATURE SUM, whereas on the second synthetic data set, initially FEATURE SUM tends to pick up more anomalies compared to RANDOM FOREST, with this behaviour reversing for lower-scoring anomalies. In all cases, the top 2 percent of flagged anomalies contained, on average, over 90 percent of the planted anomalies.

Keywords: *Anomaly Detection, Financial Transaction Networks, Network Comparison, Spectral Methods*

*Corresponding author: ande.elliott@gmail.com

1 Introduction

Financial fraud is a global challenge. According to [66], financial fraud losses in the UK payment industry across payment cards, remote banking and cheques amounted to £768.8 million in 2016, an increase of 2% compared to 2015.

Some of this fraudulent behaviour, such as money laundering, may manifest itself through unusual patterns in financial transaction networks. In such networks, customers are nodes, and two nodes u and v are linked by a directed edge if there is a money transfer from u to v ; the edge is annotated with the transferred amount as its weight. Unusual patterns could include long paths of transactions with large weights (denoted hereafter by *long heavy paths*) and large cliques. The patterns of financial fraud are highly variable, and many patterns are likely to remain undetected to date. Moreover, the number of cases is small compared to the number of legitimate transactions, and fraudsters aim to change their behaviour to avoid detection. Similar challenges arise in networks for cyber security [44]. In this work, we develop a method to uncover such unknown patterns.

Fraud detection and the related problem of anomaly detection are well studied problems; some reviews can be found in [3, 10, 20, 45, 57, 69]. Popular techniques which have been applied to the task of uncovering money laundering and crime span a wide range of topics, including autoencoders [50, 59], SVMs [64], social network analysis [21] and rule-based approaches [60]. Furthermore, anomaly detection is performed on many different types of data including (but not limited to), unstructured data [71], financial records [50], social network data [21], time series data [36] and investment bank data [37]. Some anomaly detection methods look for attempts to conceal behaviour as normal, such as [15] and [22].

Several approaches for anomaly detection exploit the network information. For example, [16] uses network statistics and a logit model to detect fraud in a factor house. In a different line of work, [58] starts with a weighted network based on multiple evidence sources, constructs a 3-hop snowball sampled network, and uses features of these networks in a machine learning framework to uncover suspicious transactions.

Many general network anomaly detection methods rely on community detection in networks. In [15], the authors declare an anomaly if the temporal evolution of network communities change sufficiently, whereas [27] compares the spectral embedding (a form of community detection) of individuals in different data sources in a cross data source embedding, declaring an anomaly if the embeddings deviate substantially. In a related fashion, [36] clusters accounts and then constructs a score based on the deviations of an account from the mean performance in the cluster. Furthermore, network comparison methods have been employed for anomaly detection. For example, the approach in [52] uses network comparison measures, and compares each network in a time series to their immediate neighbours, fitting an ARMA process to the resultant time series. The authors identify anomalies by looking for deviations from the model.

There is also a large literature on spectral localisation and general spectral approaches for anomaly detection in networks. Spectral localisation is defined as the phenomenon in which a large amount of the mass of an eigenvector is placed on a small number of its entries [17, 49]. When considering eigenvectors of various matrix operators derived from the graph adjacency matrix, the corresponding nodes on which the eigenvector localised can be regarded as different when compared to the rest of the nodes in the network, and constitute good candidates for the anomaly detection task. For example, Miller *et al.* [38, 42, 40, 41, 43, 39] develop a series of methods to uncover anomalies using spectral features of the modularity matrix. The approach that is most relevant to this work arises in [38], in which eigenvectors are assessed by their deviation in the z-score of the ℓ_1 norm. In [43], the authors extend these methods to use the ℓ_1 norm for eigenvectors of sparse PCA, which performs well at the cost of being more computationally intensive. A different spectral approach, proposed by Tong and Lin [65], is based on matrix factorisation in bipartite networks and leverages the intuition that the nodes and edges which are badly represented by the factorisation should be considered as anomalies.

In this paper, the standard network anomaly detection method which we shall employ for a comparison with our results is Oddball, by Akoglu *et al.* [2], which exploits power-law like relations between different network node level statistics. This is a widely used method which provides publicly available code; it detects several types of known local anomalies, see Subsection 3.7 for full details.

Our method is based on a combination of several approaches, namely an extension of the NetEMD

network comparison method from [67], an application of new and existing spectral localisation statistics, as well as several other modules as detailed in our pipeline in Figure 1. We combine these two main approaches with community detection, which is used to decompose the network into smaller but denser subnetworks, thus allowing for parallel computation of the code and rendering the approach computationally scalable. Community detection has additional advantages, as it increases the detectability of the embedded structures in the resultant subnetworks (see Sec. 3.2 for details), and it allows for the assignment of an anomaly score directly to each of the communities. However, the split of the network into communities has as drawback the fact that long paths may be cut (i.e., split over two or more clusters), and thus no longer detectable. To alleviate this risk, we modify our community detection approach in an attempt to avoid cutting through long heavy paths (see Sec. 3.2 for details), and we enrich our method by a component which is specifically designed to find paths in networks.

The two sets of spectral features developed in this paper are, firstly, an adaptation of the network comparison framework NetEMD to use eigenvectors. The second set of features is based on powers of the entries of eigenvectors, as well as their signs and magnitudes, see Sec. 3.3. These two sets of features may be of interest beyond the application considered in this paper.

Altogether, our combined method calculates 140 main features for each network, and uses these features to pinpoint anomalies. These features are processed in two different ways. Our FEATURE SUM method aggregates the features as an unweighted sum. Our RANDOM FOREST method is constructed by performing training and feature selection on the 140 main features using a generating model for networks with randomly added anomalies, which is developed in Sec. 2.1. As these planted anomalies are inspired by anti-money laundering, they focus on structures with heavy edges. For the synthetic models, we consider parameter regimes which ensure that in our simulated networks without anomalies, the expected count for each particular anomaly would be less than 1.

To understand what is driving our performance, we also explore the features which are selected by the random forest. We find that different parameter regions favour different features, with features based on geometric average of edge weights and short paths dominating the space of selected features.

We apply our method to a test data set derived from our model, and to a data set which is provided by the Accenture team. The first data set is generated in the same way as the data set which is used to train the RANDOM FOREST method, while the second data set is generated using a different model. On both data sets, our methods outperform random classification, and they perform favourably against Oddball from [2]. While the RANDOM FOREST method outperforms the FEATURE SUM method, in the second data set the RANDOM FOREST is outperformed by the FEATURE SUM method. Furthermore, on the second data set, both methods uncover on average 92.3% of the anomalous nodes within the top 1024 nodes, in a network of size $n = 55000$.

Finally, we investigate the nodes that are identified as anomalies but were not inserted in the graph in the Accenture model, using the RANDOM FOREST method. We empirically observe that the structure around many of the nodes have a heavy structure, including a reasonable number of heavy connections to the node of interest, and in some cases, a flow structure. Therefore, anomalies are detected without having prior explicit knowledge of their structure.

This paper is structured as follows. In Section 2, the two synthetic network data generation methods are described. Section 3 develops our new anomaly detection framework. We first perform a set of basic tests which look for deviations in the degree or the local edge weights of the nodes in the network, as described in Section 3.1. In Section 3.2, the network is divided into communities, within which we look for anomalies using modules relying on (i) basic features of the communities (Section 3.2), (ii) localisation (Section 3.3), and (iii) NetEMD (Section 3.4). We complement these methods with an approach that searches for paths and path-like structures directly in the whole network (Section 3.5), before finally combining the individual contributions either via a random forest, as described in Section 3.6, or by summing up all the features, without any prior feature selection. The results on the two synthetic data sets can be found in Section 4. The paper ends with a discussion of the results in Section 5. The appendix contains implementation details, calculations regarding regions of detectability, and a proof regarding the validity of an eigenvector score. It also contains a list of all the features considered, additional details on the feature selection step, in-sample

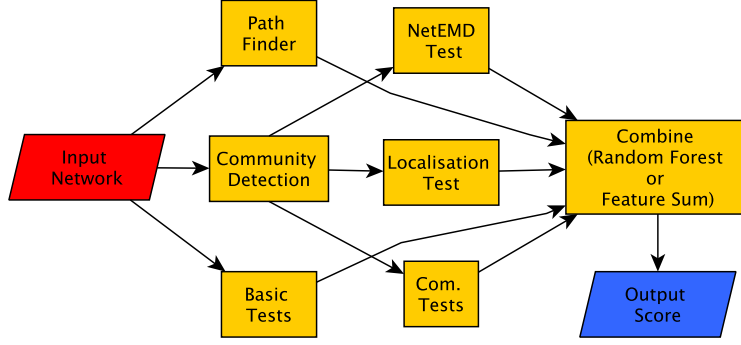


Figure 1: The complete pipeline; red parallelograms represent the input, blue parallelograms represent the output scores, and the yellow rectangles represent processes which are defined in the following sections: **Basic Tests** Sec. 3.1, **Community Detection** Sec. 3.2, **Path Finder** Sec. 3.5, **Com. Tests** Sec. 3.2, **Localisation Test** Sec. 3.3, **NetEMD Test** Sec. 3.4, **Combine** (RANDOM FOREST/FEATURE SUM) Sec. 3.6.

performance for the RANDOM FOREST, and more detailed Oddball results.

The code relies on NetworkX [29], Scikit-learn [51], Cython [8], Matplotlib [31], Scipy [33], NumPy [47], Rpy2 [28] and the NetEMD network comparison package [26] from [25].

Availability: The software for this paper can be found at <https://sites.google.com/site/elliotttande/anomalydetection>.

2 Network Models and Network Data

In the following, *at random* means *uniformly at random*, unless otherwise stated, and p_{val} represent a p -value.

2.1 Synthetic Data Set: Weighted ER Network

To develop and measure the success of our methodology, two network models with known embedded anomalies are employed. The first model guided the method development stage which lead to our pipeline (see Fig. 1). This synthetic model is a weighted directed Erdős-Rényi (ER) random graph model, in which small structures with unusually large weights are planted at random. The edge weights of the underlying ER network are drawn independently and uniformly from the interval $(0, 1)$, and the distribution of a non-anomalous weighted edge W_{ij} is given by

$$W_{ij} \sim \text{Bern}(p)\text{Uni}(0, 1),$$

where $\text{Bern}(p)$ is a Bernoulli trial with success probability p , and $\text{Uni}(0, 1)$ is a uniform random draw from the interval $(0, 1)$ which is independent of the network and of all other draws.

A random number of randomly sized anomalous structures of five different types are then embedded at random locations throughout the network. The considered five types are paths, rings, stars, cliques and a particular directed multipartite network (which we refer to as *trees*, for simplicity), as shown in Fig. 2.

The embedding procedure of such structures is as follows. For a path of size k , a directed series of edges $(\gamma_1, \gamma_2), \dots, (\gamma_{k-1}, \gamma_k)$ is embedded, with an ordered set of distinct nodes $(\gamma_1, \dots, \gamma_k)$ selected uniformly at random. For a ring, we additionally place a directed edge between γ_k and γ_1 . For the star, (Fig. 2C), k distinct nodes are chosen uniformly at random, and one of them is randomly selected to be the centre node. In order to make the network directed, the direction of each edge is selected uniformly at random. For the clique, (Fig. 2D), we select k distinct nodes uniformly at random, and choose a random direction between each pair of nodes.

The final embedded structure, slightly more complex, is a special case of a directed multipartite network (tree), as shown in Fig. 2E, and is inspired by the dynamics and flow of information or money in a real-world system. In this setting, we consider a one-dimensional ordered series of independent sets (to which we refer to as shells), with nodes in a given shell passing input to all (or a subset of) nodes in the neighbouring shell upstream (to the right in Fig. 2E). In our experiments, we place 5 nodes in the leftmost shell, which are connected by directed edges to all of the 3 nodes in middle shell, which in turn are full connected to the 1 node in the rightmost shell. For brevity in this document and a slight abuse of notation, we shall refer to this multipartite network as a ‘tree’. For simplicity, the directional flow of all (existing) edges across shells is from left to right, without exception. Such a structure could be representative, for example, of a hierarchical structure in a criminal organisation, with resources flowing from the lower-ranked members towards the head of the organisation.

To construct a heavy structure on the planted anomalies, rather than drawing the weights on the edges from $\text{Uni}(0, 1)$ as for the other edges in the network, we draw them independently from $\text{Uni}(w, 1)$, where w is a parameter to the model. The construction of potential edges which are not part of the embedded structure (i.e., edges in the clique which are in the direction opposite to that selected for an anomalous edge) are subject to the standard edge construction.

The detailed steps of our network construction is as follows.

1. Generate a directed ER network with n nodes and probability of connection p .
2. Add weights from $\text{Uni}(0, 1)$ to each of the existing edges.
3. Draw the number of anomalies to embed uniformly at random from $[5, \dots, 20]$.
4. Select the type of anomaly uniformly at random from {rings, paths, cliques, stars, trees}.
5. For each anomaly, select its size uniformly at random from $[5, \dots, 20]$, except for the tree which has fixed size.
6. Select the appropriate number of nodes from the network, uniformly at random.
7. Insert the chosen anomaly in the network.
8. Add/replace the edges in the chosen induced copy of the anomaly with weights drawn from $\text{Uni}(w, 1)$.

The choice of the size and number of the embedded structures is motivated by the consideration that we would like the structure to be large enough such that it can be found, but not too large or frequent to overwhelm the network. Our choice of constants in the above procedure produces on average around 147 anomalies (assuming no overlaps). Therefore, for $n = 10,000$, on average around 1.47 percent of nodes in the network are anomalies. This level of imbalance was chosen so that the anomalies do not overwhelm the underlying network.

2.2 Synthetic Data Set: Accenture model

The second network generating model was developed by the Accenture team (MML, PR), following discussions with the Turing team (AE, MC, GR); a network which was generated from this model was shared with the Turing team for performing anomaly detection. They were given access to several samples from the model, and they were informed that the anomalies would include heavy edges. The Turing team was oblivious to the model details; the details were only revealed to the Turing team when generating the final set of 100 test networks for inclusion in this paper. Therefore, we regard this model as an independent test of our proposed scoring methodology.

The synthetic Accenture network is constructed as follows. The number of nodes is fixed to 55000, and each node is assigned an in-degree and an out-degree. The assignment is made by rounding down random draws from a normal distribution; for the in-degree, the mean is set to 21 with a standard deviation of 3, while for the out-degree, the mean is set to 19 with a standard deviation of 2. If the sum of the in-degrees

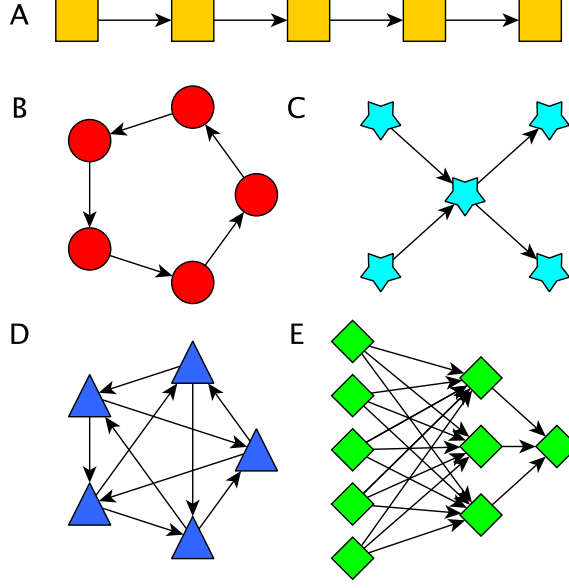


Figure 2: Diagram of the embedded anomalies. (A) **Top yellow squares** - Directed path. (B) **Middle left red circles** - Ring structure. (C) **Middle right light blue star** - Star structure with a directed flow from left to right. (D) **Bottom left blue triangles** - Clique structure. (E) **Bottom right green diamonds** - Directed multipartite structure showing flow of money/information from left to right.

does not agree with the sum of the out-degrees, then the larger sum is reduced by decreasing by 1 the appropriate degree of a random node until the totals match.

As in a standard configuration model, each node is assigned an in-degree and an out-degree, and a corresponding number of stubs are created. The stubs are then randomised and matched. In the case of self loops, the algorithm randomly selects a set of edges of same length as the number of random edges, and swaps the ‘sender’ node of a self loop with the sender node of the randomly selected edge. This procedure can generate multi-edges, which are permitted. This process repeats until all stubs are matched. Edge weights are then drawn from a normal distribution with mean 1000 and standard deviation of 200, with all edge weights being positive.

The following anomalies were added to the resulting network: a clique of size 8 and one of size 12, a ring of size 4 and one of size 10, and a heavy path of size 5 and one of size 10. In contrast to the weighted ER case (Sec. 2.1), all of the weights on the ring are constant, and the heavy path is sampled from paths that already exist in the network. Moreover, the edges in the clique of size k are not randomly arranged; instead, there is one edge between each pair of nodes and the out- (in-) degrees of the nodes in the clique are set to $[k - i : i \in [1, \dots, k]]$. Finally, the weights on the heavy edges are drawn as follows: draw a number uniformly at random from the interval $(0.99, 0.999990)$, and use as weight the value that would be required to obtain this number as percentile from the normal distribution with mean 1,000 and variance 200. Due to lack of other information about the network structure, the Turing team used a configuration model as the null model for the Monte Carlo tests in the anomaly detection pipeline.

3 Anomaly Detection Method

In this section, we describe the **node-based network summary statistics which are used to construct features**. Unless otherwise stated, a feature is constructed from a node-based statistic as follows. **We construct $N = 500$**

replicas (realisations) from the configuration model which fixes the in- and out-degrees, and randomly shuffles the weights. If the p -value p_{val} of this test is at least 0.05, the node score is set to 0. If p_{val} is less than 0.05, we use the percentiles of the standard normal distribution: the nodes score is $\Phi^{-1}(1 - p_{val})$, where Φ is the c.d.f. of the standard normal distribution.

3.1 Basic Detection Module

First, we include node-level tests of basic properties of the network in order to check for anomalies, which mainly investigate if the weights of edges which are adjacent to a node appear to deviate from random. As a null model for the test, we randomly shuffle the weights while keeping the network fixed, and thus, when testing if the local edge weights of a node deviate from random, we simply test if it deviates from a random draw with replacement of the weights.

The first set of test statistics are based on the geometric average of weights (GAW). To give the statistic more power in the case of an anomaly with extremely high weights but small number of nodes, we also construct a statistic over the largest 10% and 20% of weights of the edges attached to a given node. Letting $s_1^{(i)}, s_2^{(i)}, \dots, s_{d_i}^{(i)}$ denote the ordered edge weights (both incoming and outgoing) of a given node i with d_i neighbours, we define the following scores

$$\begin{aligned} \text{GAW}(i) &= \left(\prod_{j=1}^{d_i} s_j^{(i)} \right)^{d_i^{-1}} \\ \text{GAW10}(i) &= \left(\prod_{j=d_i - \lceil 0.1 d_i \rceil + 1}^{d_i} s_j^{(i)} \right)^{(\lceil 0.1 d_i \rceil)^{-1}} \\ \text{GAW20}(i) &= \left(\prod_{j=d_i - \lceil 0.2 d_i \rceil + 1}^{d_i} s_j^{(i)} \right)^{(\lceil 0.2 d_i \rceil)^{-1}}. \end{aligned}$$

To compare against random, we use the configuration model; however, as our implementation of the configuration model shuffles the edge weights (see Section A.1), rather than constructing the configuration model networks directly, we save computational time by comparing with random draws from the observed weights. To further save computational time, we draw with replacement, and for the Monte Carlo test we share an empirical null distribution between nodes with the same number of edges. The increased efficiency allows us to consider a larger number of samples, and thus, for the results in this paper we use 10,000 potentially shared samples in each Monte Carlo test.

As an additional feature, we include the standardised version (z-score based on empirical mean and empirical standard deviation) of the node degree (total number of connections), in order to capture when the degree is larger or smaller than expected at random.

3.2 Community Detection

The more intricate parts of our anomaly detection pipeline (NetEMD and Localisation) look for more subtle differences than the basic tests. However, looking for such more subtle changes is difficult, as it is unlikely that a small anomaly will cause a deviation over the whole network, that is large enough to trigger an alert. To address this, we perform a community detection step, in order to divide a network into a number of components, such that the density is higher within communities compared to the density across communities.

As we are looking for structures with heavy flow through the network, we augment paths of heavy edges in order to increase the likelihood of them being placed within the same community, as follows. For each path of size 3 (with 3 nodes and 2 edges) such that the weight on both edges (w_1 and w_2) is greater than the 99th percentile of the observed weight distribution:

1. If there is not an edge between the first and last node, then place one with weight $\min(w_1, w_2)$;
2. If there is an edge of weight w_3 , then update the weight to $\max(w_3, \min(w_1, w_2))$.

This procedure is repeated until no new edges can be added. The intuition behind this augmentation step is that structures of heavy average weight are transformed into structures that resemble cliques. **This augmentation step can also be seen as a regularisation of the graph, similar to what is typically performed in spectral clustering in the sparse regime;** for examples, see [5], [32], [14] and [53]. As a next step, we perform community detection on the augmented networks using the Louvain algorithm [9] on the symmetrised graph $(A + A^T)$, with the standard setting and resolution parameter 1. For this task, we use the *python-louvain* package from [6].

To test if a given community density or edge weights deviates from random, we employ five statistics on each community; three statistics are based on the density of the community and two are based on the strengths within the community. The density-based measurements are defined as follows.

1. The edge density of the community is divided by the density of the full network, in order to capture communities that are denser than expected.
2. The ratio above, divided by the size of the community, is also used in an attempt to further distinguish small communities (for which the penalty is smaller) from large communities.
3. We use replicas from a configuration null model and perform community detection on each of those to obtain a distribution for the expected density of a community. We consider one community from each network randomly, and measure the density in the non-augmented network to obtain a null distribution. As these community detection procedures are computationally expensive, we perform only 20 of them and then use the resultant null distribution to measure a p -value for the density of each of the communities in the resultant network, focusing on the upper tail, i.e. when they are denser than we would expect at random. If the resultant p -value is more than 0.5, we attribute all nodes in the community a score of 0, and otherwise use the normal percentiles as before. To account for very small communities (with less than 4 nodes) we return a separate feature which has value 1 for nodes in such a small community, and 0 otherwise.

The edge weight measures are similar to the first two density statistics, however, rather than the density, we use the geometric average of the weights (GAW) within each community and across the network. After this community detection step, we construct two sets of networks, the first induced by the communities in the original network, and the second induced by the communities in the augmented network. Crucially, each set of networks is useful for a different part of our pipeline, as we will describe in the following sections.

3.3 Spectral Localisation

Spectral *localisation* in networks is the phenomenon that a large amount of the mass of an eigenvector is placed on a small number of nodes. In many cases, such nodes are different in some way compared to the remaining ones, and hence they may be good candidates for the anomaly detection task. [54] calls an unit eigenvector *delocalised* if its entries are distributed roughly uniformly over the real or complex sphere, and quantifies this in various ways using tools from non-asymptotic random matrix theory. While most of the work on eigenvector localisation, including [7, 13, 34], has focused on the high frequency eigenvectors (i.e., those corresponding to the larger eigenvalues), recent work [30] revealed localised eigenvectors associated to small eigenvalues. Sapoval [55] studied localised eigenvectors in different domains and pointed out their importance for physical applications, including the design of efficient noise-protective walls.

In this section, we also consider other forms of eigenvector behaviour which are not strictly localisation, such as a small number of nodes that have positive entries, in contrast to the majority of the nodes having negative values. However, for compactness, in this paper we shall broadly refer to each of these behaviours as localisation.

As shown in Figure 1, we look for localisation in each of the communities separately, attempting to use the fact that small anomalies are more likely to be identifiable within the communities rather than the whole graph. We consider eigenvectors of three matrices, namely the symmetrised adjacency matrix, the combinatorial Laplacian of the symmetrised adjacency matrix, and the random walk Laplacian of the

symmetrised adjacency matrix, which are defined as follows. Let $\mathbf{W} = (\mathbf{W}_{ij}) \in \mathbb{R}^{n \times n}$ be a weighted (possible directed) adjacency matrix with non-negative entries, and let $\mathbf{W}^{(s)} = \mathbf{W} + \mathbf{W}^T$ be the symmetrised version. We consider the following Laplacians

$$\text{combinatorial Laplacian:} \quad \mathbf{L}_{Comb} = \mathbf{D} - \mathbf{W}^{(s)}, \quad (1)$$

$$\text{random-walk Laplacian:} \quad \mathbf{L}_{RW} = \mathbf{D}^{-1} \mathbf{W}^{(s)}, \quad (2)$$

where $\mathbf{D}$ is a diagonal matrix with the weighted degrees (row-sums) of the symmetrised adjacency matrix $\mathbf{W}^{(s)}$ on the diagonal, i.e. $\mathbf{D}_{ii} = \sum_{j=1}^n \mathbf{W}_{ij}^{(s)}$. Eigenvectors are always normalised so that the sum of the squares of the entries is 1.

Rather than looking at the complete network, localisation is assessed in each of the communities separately, in order to increase the chance of finding small-sized anomalies. Furthermore, rather than using the subnetworks corresponding to the communities in the original network, subnetworks of the augmented network are used as the additional augmented edges around the anomalies (as well as other heavy structures that appear at random) are likely to help the localisation methods.

Localisation is assessed in the top 20 and bottom 20 eigenvectors of the symmetrised adjacency matrix $\mathbf{W}^{(s)}$, and the top 20 non-trivial eigenvectors of the combinatorial Laplacian $\mathbf{L}_{Comb}$ and random-walk Laplacian $\mathbf{L}_{RW}$ of the augmented subnetwork corresponding to each community. Note that, in the case of the Laplacian matrices, the trivial eigenvectors are ignored; for $\mathbf{L}_{Comb}$ the bottom eigenvalue (smallest) is always 0, and in the case of $\mathbf{L}_{RW}$, the top eigenvalue (largest) is always 1. Whenever the network has fewer than 20 nodes, we consider as many eigenvalues as there are available. Ties (i.e., eigenvalues with multiplicity) are broken at random.

To test if a vector is localised, we compare the vectors to random, following the approach used in Sec. 3.1, we compare to the eigenvectors of configuration model replicas. If a replica network, which is generated under the null model, has fewer than the required number of eigenvectors, the replica is resampled to obtain a network with sufficiently many eigenvectors for the adjacency comparison (note that the Laplacian comparisons need one more eigenvector than the symmetrised adjacency matrix due to the trivial eigenvector). To avoid running the procedure indefinitely, if the 10^{th} network has more than 2 nodes, this network is used, else we continue to resample until the network has more than the required number of eigenvectors. Then, the i th eigenvector is compared with the networks in the ensemble of networks which are generated under the null model and which contain at least i eigenvectors. For a more in-depth explanation see Appendix A.2.4.

To capture the different types of localisation, we use several statistics from three broad classes, namely: norm-based, sign-based, and direct localisation.

Norm-Based Statistics As in [19], we look for eigenvectors such that the sum of the 4^{th} powers of the eigenvector entries (also known as the inverse participation ratio, IPR) is larger than expected at random. As the sum of the squares of an eigenvector is 1, a large sum of fourth powers implies there are large elements in the vector. Extending this idea, for an eigenvector v with elements v_i , we also consider the statistic $\sum_i (e^{|v_i|} - |v_i| - 1)$. An eigenvector with all the mass on one entry would have a value of $e - 2 \approx 0.71828$, whereas a perfectly delocalised eigenvector ($v_i = n^{-0.5}$, $\forall i$) would have a value of $n(e^{n^{-0.5}} - n^{-0.5} - 1)$, which for $n = 10,000$ amounts to ≈ 0.50167 . Note that [38] also uses localisation, but focuses on the ℓ_1 norm rather than the inverse participation ratio.

Sign-Based Statistics Some eigenvectors can be used to identify anomalous structures by the sign of the observation on each node, even when many entries in the eigenvector are zero. To capture this effect, we compute the number of nodes that have a strictly positive or strictly negative value, and we use the minimum of these quantities divided by $\text{Min}(20, \text{Number of Non Trivial Eigenvectors})$ as a test statistic. There are a number of boundary cases for this statistic (for example a vector that is all positive), the treatment of which we detail in Appendix A.2.3.

Direct Localisation Statistics To directly capture the localisation phenomenon, we take the minimum number of entries required such that their sum is at least 90% of the sum of a chosen statistic over the eigenvector. The chosen statistics for this procedure are, the sum of the fourth power of each of the entries,

$$0 = d_{NetEMD}(\text{red normal}, \text{blue normal}), \quad 0 < d_{NetEMD}(\text{orange exponential}, \text{blue normal})$$

Figure 3: An example to illustrate that NetEMD only considers the shape of the distribution rather than the position or the scale: In the left panel, the NetEMD score for comparing two normal distributions with different mean and variances is 0, whereas in the right panel the NetEMD score for comparing a normal distribution and an exponential distribution is greater than 0.

and the sum of the absolute values of these statistics. For consistency, whenever the entries are exactly equal, all nodes with the given value are included in the count.

The next step is to evaluate whether the above test statistics deviate in the appropriate direction, based on a p -value test using a set of replicas generated under the null model. For computational reasons, we share the set of configuration model realisations across different eigenvectors. For these experiments the number of null model repeats is $N = 500$. For computational reasons, we fix the number of null model replicas at 500, and we share the replicas between eigenvectors of the same matrix. Thus, the tests for different eigenvectors of the same matrix are not independent; this procedure is not a rigorous significance test, but rather a procedure to generate a score.

Once vectors that deviate significantly from what is expected at random are identified, we then select the nodes which caused the deviation. For the norm-based approaches, the divide between significantly different nodes turns out to be difficult to distinguish, and thus we consider four different statistics to capture this divide, which are related to the raw value, the significance of the test statistic that highlighted the eigenvector, and the element-wise behaviour observed at random; see Appendix A.2.2 for details. For the sign-based approaches, the nodes which possess the rare sign are selected, and following Sec. 3.1, a score based on the standard normal percentiles is assigned. To further highlight small communities, we also construct a score which is normalised by the number of nodes that have the rare sign. A small number of special cases in selecting the nodes, e.g. if the number of positive entries equals the number of negative entries, is treated in Appendix A.2.3. A small number of additional implementation details are described in Appendix A.2, and a small number of additional features which highlight special cases can be found in Appendix D.5.

For the random forest or other machine learning approaches, it may also be interesting to consider the absolute value of the raw eigenvector, as this would allow our random forest to decide if there is sufficient localisation based on the raw eigenvector rather than relying on handcrafted statistics. We include this feature for completeness, but for simplicity, in all of the experiments in this paper, we set the value to zero.

To reduce the total number of features, the contribution from a given localisation test and scoring method in each eigenvector of a given matrix is summed to give one feature. Thus, rather than obtaining 1200 features which are composed of all combinations of each of the statistics and eigenvectors, the 20 eigenvectors in each set ($\mathbf{L}_{RW}$, $\mathbf{L}_{Comb}$, large eigenvalues of $\mathbf{W}^{(s)}$, and small eigenvalues of $\mathbf{W}^{(s)}$) are combined to yield 60 features; for a complete list see Appendix D.4.

3.4 The NetEMD Module

The NetEMD module extends the NetEMD network comparison tool from [67] into an anomaly detection method by measuring the difference between the observed network and an expected network. Rather than comparing raw values of a given network statistic, NetEMD compares the shapes of the distributions of a statistic over the nodes of a network. As an illustration of NetEMD, Fig. 3 shows that the NetEMD score for comparing two normal distributions with different mean and variances is 0. In contrast, the NetEMD score for comparing a normal distribution and an exponential distribution is greater than 0.

To describe NetEMD, following the notation in the localisation section, we represent networks as a (possibly) weighted (possibly) directed adjacency matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$. Let $T(j, \mathbf{W})$ be a node level statistic

for node j on a network $\mathbf{W}$, which takes on real values, and let

$$U_T(x, \mathbf{W}) = \frac{1}{n} \sum_j \mathbb{1}(x \leq T(j, \mathbf{W})) \quad (3)$$

denote the empirical distribution function of the statistic T evaluated on node j of network $\mathbf{W}$. We define the standardised version of T in order to remove the effect of scale

$$\tilde{T}(j, \mathbf{W}) = \frac{T(j, \mathbf{W})}{\sigma(T, \mathbf{W})}, \quad (4)$$

where $\sigma^2(T, \mathbf{W}) = \frac{1}{n} \sum_{i=1}^n (T(i, \mathbf{W}) - \mu(T, \mathbf{W}))^2$ is the empirical variance, and the empirical mean is given by $\mu(T, \mathbf{W}) = \frac{1}{n} \sum_{i=1}^n T(i, \mathbf{W})$. For two networks $\mathbf{W}$ and $\mathbf{W}'$, and a given statistic T , the NetEMD score is given by

$$\text{NetEMD}_T(\mathbf{W}, \mathbf{W}') = \min_s \int_{-\infty}^{\infty} |U_{\tilde{T}}(x + s, \mathbf{W}) - U_{\tilde{T}}(x, \mathbf{W}')| dx,$$

where $\tilde{T}$ rather than T is used in $U_{\tilde{T}}$, in order to remove the effect of scale. The minimum s of $\int_{-\infty}^{\infty} |U_{\tilde{T}}(x + s, \mathbf{W}) - U_{\tilde{T}}(x, \mathbf{W}')| dx$ will be achieved as the empirical distribution functions only have finitely many jumps, see for example [70].

Let $C_T = \{T^{(m)}, m = 1, \dots, M\}$ denote a set of node statistics such that for each node j and each network $\mathbf{W}$, $T^{(m)}(j, \mathbf{W}) \in \mathbb{R}$. The NetEMD statistic for such a set of node level statistics is the average over the single NetEMD scores

$$\text{NetEMD}(\mathbf{W}, \mathbf{W}') = \frac{1}{|C_T|} \sum_{T \in C_T} \text{NetEMD}_T(\mathbf{W}, \mathbf{W}'). \quad (5)$$

In [67], the statistics often used as $T^{(m)}$ are the so-called orbit counts, i.e., the number of times that a given node is part of a given subnetwork in a given position.

The Set of Statistics As a set of NetEMD statistics, we use the in-strength (i.e., the sum of all weights coming into a node), out-strength (i.e., the sum of all weights going out of a node), the sum of out-strength and in-strength, and finally each of the directed motifs of size 3; these are shown in Fig. 4,. Here, a motif is just a small network; no over- or under-representation is implied. Following the approach used by [68], we let the weight of each motif be the product on the weights on the edges. Formally, for a given motif $\mathbf{M}$ on 3 nodes, letting $\mathbf{W}([i, j, k])$ be the induced subnetwork of $\mathbf{W}$ with nodes i, j and k , the test statistic is

$$T^{\text{Motif}(i)}(j, W) = \sum_{a < b} \mathbb{1}(\mathbf{W}([j, a, b]) \text{ is an induced copy of } M) \prod_{c, d \in [j, a, b]} ((W_{cd} - 1) \mathbb{1}(W_{cd} > 0) + 1),$$

where the indicator function $\mathbb{1}$ is 1 if the statement is correct, and 0 otherwise. The indicator in the product ensures that the factor in the product is 1 when the edge W_{cd} has zero weight, and it is the weight W_{cd} when this weight is not zero. These motifs are counted in the non-augmented graph.

To the statistics shown in Figure 4 we add the following spectral measures based on Subsection 3.3, which are obtained from the augmented graph. For the symmetrised adjacency matrix, we consider the eigenvectors corresponding to the 5 largest and 5 smallest eigenvalues, which we name $T^{\text{Adj}U(i)}(j, \mathbf{W})$ and $T^{\text{Adj}L(i)}(j, \mathbf{W})$ respectively. For the Laplacians, the top 5 eigenvalues are used, which corresponds to the smallest eigenvalues in the combinatorial Laplacian and the largest eigenvalues in the random-walk Laplacian. As a remark, we again ignore the trivial first eigenvalue in the Laplacians. We refer to these statistics as $T^{\text{Comb}(i)}(j, \mathbf{W})$ and $T^{\text{RW}(i)}(j, \mathbf{W})$. Ties are broken at random.

An issue arises from the sign of the eigenvectors. Let $\mathbf{B}$ denote one of the matrices of interest (one of $\mathbf{W}^{(s)}$, $\mathbf{L}_{\text{Comb}}$, $\mathbf{L}_{\text{RW}}$). Let $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ and $v^{(1)}, \dots, v^{(n)}$ such that $\mathbf{B}v^{(i)} = \lambda_i v^{(i)}$. As the direction of an eigenvector $\mathbf{v} = (v_1, \dots, v_n)$ is arbitrary, comparing the shapes of the distributions as in Fig. 3 has to be

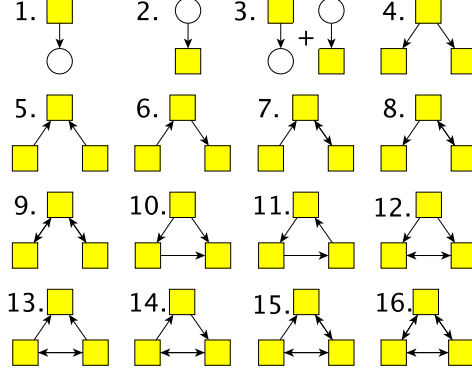


Figure 4: The subnetwork statistics that are used in the NetEMD module. Statistic **1** is the total the out-strength, statistic **2** is the total the in-strength and statistic **3** is the total of the in-strength and out-strength. The remaining statistics are the directed motifs of size 3, where the weight is given by the product of the weights on each of the edges.

adjusted to account for the sign of the eigenvector. To this end, first we check if the distribution of elements in the vector is symmetric. If so, then we arbitrarily select the direction as this does not effect the result of the computation. If the distribution of elements in the vector is not symmetric, then we choose the sign which yields the largest number of positive elements. If this procedure results in a tie, we then break ties using $g(a) = \sum_j ((v_i)_j)^{2a+1}$; letting a^* be the smallest $a \in \mathbb{N} \cup \{0\}$ such that $g(a^*) \neq 0$, and we use the sign of $g(a^*)$. Appendix C gives a proof that, for a finite network, the value a^* is finite.

Converting NetEMD into an anomaly detection method. To convert NetEMD values into node scores, following [48], we use a Monte Carlo approach with NetEMD as a statistic. First, we generate a reference set of 15 networks, which are generated under the null model, and calculate the NetEMD distance of our target network to the networks in the reference set. To produce a robust measure of deviation, we use a trimmed mean in which the largest and smallest distances out of the 15 reference network distances are excluded.

We then test if the deviation is large enough using a Monte Carlo p -value test, with a second set (which we call the null set) of networks which are generated under the null hypothesis and which are independent of the reference set. Instead of considering the average NetEMD over a set of NetEMD values for different statistics as in Eq. (5), we instead compute the trimmed NetEMD mean, and run the Monte Carlo test on each of the statistics in C_T^* individually in each community. We combine the results for a given statistic into a single feature, resulting in one feature per statistic, that we will later combine into an overall ranking.

To address the case where the number of eigenvectors in the null replica is lower than the required number (6 in this case), if a null replica has too few entries, we resample up to 10 times to construct a network with at least 6 nodes, and if by the 10th resample we do not obtain one, then we consider the next network with more than 3 nodes. Again, following the localisation approach detailed in Appendix A.2.4, we only make comparisons between pairs of networks that have a sufficient number of eigenvectors. A few special cases are deferred to Appendix A.3.

For statistics for which NetEMD appears to deviate from the expected behaviour, i.e., with a low Monte Carlo p -value, we assign scores to nodes in two ways. As we know we are interested in structures that are likely to be over- or under-represented, we focus on the tails of the empirical distribution of a given statistic over the network.

First, we select the nodes that deviate more than two empirical standard deviations from the average of the node contributions, and assign them a score which is the absolute value of their deviation. Mathematically,

for a network $\mathbf{W}$ and a node statistic $T \in C_T^*$, we let

$$score_1^{NetEMD}(j, T, \mathbf{W}) = \begin{cases} 0 & \text{if } |\frac{T(j, \mathbf{W}) - \mu(T, \mathbf{W})}{\sigma(T, \mathbf{W})}| < 2, \\ |\frac{T(j, \mathbf{W}) - \mu(T, \mathbf{W})}{\sigma(T, \mathbf{W})}| & \text{otherwise.} \end{cases} \quad (6)$$

To account for statistics with heavy-tailed distributions, our second method of identifying nodes that contribute most to the deviation, considers the top 5% of node contributions based on the absolute standardised score above, and set

$$score_2^{NetEMD}(j, T, \mathbf{W}) = \begin{cases} 0 & \text{If in bottom 95\% of } |\frac{T(j, \mathbf{W}) - \mu(T, \mathbf{W})}{\sigma(T, \mathbf{W})}| \\ \Phi^{-1}(1 - p_{val}) & \text{else,} \end{cases} \quad (7)$$

where Φ^{-1} is the inverse c.d.f. of a mean 0 and variance 1 normal distribution.

Finally, in order to reduce the total number of features, we sum the contribution in a given scoring method in each eigenvector of a given matrix into one feature. For example, for a given node j , this amounts to

$$score_X^{NetEMD}(j, T^{AdjL}, \mathbf{W}) = \sum_{i=1}^5 score_X^{NetEMD}(j, T^{AdjL(i)}, \mathbf{W}),$$

with analogous forms for the different matrix operators we consider.

3.5 Finding Heavy Paths

The separation of the network into communities is computationally advantageous as it facilitates parallelisation, the disadvantage being that it can split long paths. To alleviate this risk, we include a separate feature for finding long paths. As enumerating all paths of a given (long) size is computationally prohibitive, we take inspiration from [23] and [22], and use a BeamSearch method in which we retain a small set of promising examples to extend, with a BeamWidth of 5000. This allows us to tradeoff the optimality of the solution against the running time, and leads to the following algorithm.

We start by constructing a set of paths of size 3 (i.e., with 3 nodes and 2 edges). Rather than considering all possible paths of size 3, we consider a more limited set as described in Appendix A.4. We remove all but the top BeamWidth paths (for the case of ties, see Appendix A.4), store the result, and form a new list considering all extensions of the paths which can be formed by adding an additional edge. We repeat this procedure until the paths in the list are of the required size. The set of paths of different sizes, generated by this procedure, are then used to construct features as usual, based on a Monte Carlo test and standard normal percentiles.

Next, we obtain a score for each node by summing the score for each significant path that contains the node in question. For computational reasons, the null replicates are shared between path sizes rather than running the algorithm for each path size. Finally, we test for path sizes. In general, in our method we consider 30 different path sizes, varying the size from 3 up to 32. However, for computational efficiency, in the experiments detailed in this paper, we only compute paths of size less than or equal to 21 (i.e. path length less than or equal to 20).

3.6 Combining the scores

Each of the network statistics described thus far produces a score (a full list of all procedures can be found Appendix D). One way to combine the scores is simply to sum them, which yields a raw measure of performance of the pipeline; this is the FEATURE SUM method. As the FEATURE SUM may not be the most effective combination of the underlying features, following [58], we also combine the features using a random forest approach which relies on a training set of labelled data. Such training data are generated from the model in Sec. 2.1, with the parameter sets chosen such that under the model without anomalies, each anomaly has expected value less than 1. We then perform feature selection, prioritising features that

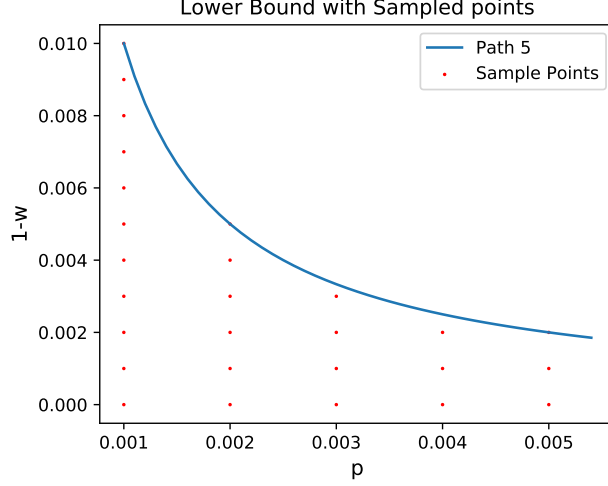


Figure 5: Bounds on (w, p) and chosen parameter combinations. The set of parameter combination points (red dots) is contrasted with the bounds on $1 - w$ for different values of p below which the expected number of paths of size 5 in a network with 10,000 nodes is less than 1.

perform well across the ensemble of parameter regimes, as will be described in Sec. 3.6. Finally, we combine the data from each of the parameter regimes, and fit a random forest on the reduced feature set to produce our final score which we denote by RANDOM FOREST.

Selecting the parameters of the training set. To ensure that each embedded anomaly is detectable, the parameter regimes for (p, w) are chosen such that the expected number of anomalous instances of each of the planted substructures in the background network is less than 1. In Appendix B.1 we derive the bounds for each structure, and it is shown that the path of size 5 is the most restrictive anomaly. It requires the expected counts of this anomaly under the null model to be less than 1, leading to the inequality

$$(1 - w)p < \left(\binom{n}{5} 5! \right)^{-\frac{1}{4}}. \quad (8)$$

This region is plotted in Fig. 5. We vary p in fixed increments of size $\frac{1}{1000}$, between $\frac{1}{1000}$ up to and including $\frac{1}{200}$, and we vary w , starting with 0.99 and increasing in increments of $\frac{1}{1000}$ up to and including 1.0. This results in 55 possible sample points, of which 27 are within the bounds of detectable points. The points can be found in Fig. 5. For each of the 27 parameter sets, we construct 100 networks and split them, with the first 70% as the training set and the remaining 30% as the test set. We run the entire pipeline on each of the 2700 networks.

Feature selection across the ensemble. To select a set of features which work well across each of our parameter regimes, we fit a random forest to the training data for each given parameter set using the scikit-learn *RandomForestRegressor* with default parameters in version 0.19.2, and use the feature importance vector [11, 51, 62] to rank the features. Next, we compute the average rank of each feature

$$\text{FeatureRank}_i^{(\text{All})} = \frac{1}{27} \sum_{p,w} \text{FeatureRank}_i^{(p,w)}, \quad (9)$$

where $\text{FeatureRank}_i^{(p,w)}$ is the rank of feature i in the parameter set (p, w) . We select an appropriate cut-off by finding a large jump in a plot of the resulting average ranks.

3.7 Oddball

The original formulation of Oddball from [2] is built to detect 4 different types of anomalies in weighted networks which can be described as “almost” star, “almost” clique, the so-called “heavy vicinities” based on weighted degree, and heavy edges. [2] observes an empirical relationships between measured statistics related to each type of anomaly. The relationships are power-laws which are identified between statistics of the 1-hop snowball sample of each node, namely the number of edges against the number of nodes, the total weight of edges against the number of edges, and the largest eigenvalues (of the adjacency matrix on the sample) against the total weight. For each relationship, Oddball fits the resultant power-law and then computes the following outlier score for each node

$$\frac{\max(\text{observed}, \text{predicted})}{\min(\text{observed}, \text{predicted})} (\log(|\text{observed} - \text{predicted}| + 1)). \quad (10)$$

The authors augment this score by summing the scores (10) with the local outlier factor (see [12]) to capture points which may be close to prediction, but far from other observed points. The authors’ website, available at <http://www.andrew.cmu.edu/user/lakoglu/> provides a version of Oddball, called *Oddball-lite*, which does not use Python and which we use in this paper; *Oddball-lite* appears not to carry out the augmentation by the local outlier factor. Furthermore, while Oddball was originally formulated as an anomaly detection scheme for weighted non-directed networks, the *Oddball-lite* implementation provided by the authors of [1] and [2] also provides a directed version, which we use. For completeness, we include in Appendix G.1 the relationships used in the directed case, as well as small modifications to the code we performed in order to run the experiments in this paper.

Finally, in the original Oddball paper [2], deviations from each observed relationships are considered separately with a ranking constructed for each relationship (e.g., number of nodes versus number of edges), and each relationship then related to a different type of anomaly. However, we construct one overall ranking from our main features. For a meaningful overall comparison with Oddball, we consider the sum over each of the relationships to construct an overall ranking. In Appendix G, we consider the individual performance of each of the features and discuss any well performing underlying relationships in the main paper.

3.8 Performance Measures

To measure the performance of our methods we consider three different statistics. Our primary use case is in identifying financial crime, and as we know that each of our hits will have to be validated by a human, only a relatively small number of candidates could be validated. To that end, we consider the performance of the top $\{1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024\}$ nodes in a ranking. We measure performance and compare graphically, using precision and recall which are defined as follows [51]

$$\begin{aligned} \text{Recall} &= \frac{\text{Number of Anomalies in sample}}{\text{Total Number of Anomalies}}, \\ \text{Precision} &= \frac{\text{Number of Anomalies in sample}}{\text{Size of Sample}}. \end{aligned}$$

Furthermore, we construct our ranking and thus our fixed-size sample using a score. In the case of ties, we use the average number of anomalies if we selected randomly between the tied nodes. Further, if a node is involved in two anomalous structures we only count the node once, and the total number of anomalous nodes is reduced accordingly.

Comparing the precision and recall of two different networks with different numbers of anomalies can be misleading. As the number of anomalies in the weighted ER graph is unknown, ranging from 25 to 400, we interpret the precision and recall results as advisory. In the Accenture model case, the number of anomalous nodes is fixed (with the exception of a small number of overlapping anomalous structures), and hence the results are easier to interpret.

The third performance measure is the average precision, which is defined as follows. Let $R_{ec}(i)$ denote the recall if we threshold at the top i -th nodes in a ranking, and $P_{re}(i)$ denote the precision for the same

threshold. Following [51], the average precision is defined as

$$\sum_{i=1}^{n-1} (R_{ec}(i+1) - R_{ec}(i)) P_{re}(i),$$

and has been shown to give additional weight to the highest ranked items [63], thus making it a good measurement for this work. Other assessment methods for classification performance are discussed, for example, in [56].

Finally, we also compare the performance of our measures to that of a random classification. As shown in [63], under a continuous approximation, a random ordering will have an average precision equal to the percentage of anomalies in the network. Therefore, in order to compare our performance to random, we compare against the percentage of anomalies in the network.

4 Results

The performance of the FEATURE SUM and the RANDOM FOREST methods is first tested on the weighted ER networks, and then on the Accenture model.

4.1 Results on the Weighted ER Network

4.1.1 FEATURE SUM

The FEATURE SUM method simply sums the scores of all features. To avoid biasing the testing data, we compute the performance using only the training data.

The results can be seen in Fig. 6. Regarding the precision, the FEATURE SUM method performs well in all networks, with the top scoring 8 nodes often all being true anomalies. Regarding the recall, for low values of p , the FEATURE SUM method recovers most anomalies within the 1024 top scoring nodes, with a reasonable fraction being recovered within the first 512 top scoring nodes (approximately 5% of nodes). As the number of anomalies varies between 50 and 400, obtaining a good proportion of the nodes within the first 5% is a reasonably good performance. For higher densities, the performance is similarly high for the top nodes, but only capturing around 75% of the anomalous nodes. The standard deviation tends to be larger for the medium node score range, indicating that the random network generation appears to affect the ordering more in the medium node score range than it does in top and bottom ranges.

Finally, we compare the performance of the FEATURE SUM in different parameter regions using the average precision; the results can be seen in Fig. 7. The FEATURE SUM scoring system outperforms random classification (red dashed line) in all parameter regimes considered. The median average precision for the low density networks ($p \leq 0.003$) is reasonably consistent, with values around 0.5 – 0.6. For larger densities, we observe lower values, albeit still considerably above the random regime.

4.1.2 RANDOM FOREST

RANDOM FOREST feature selection. As described in Sec. 3.6, a separate random forest is fitted to each of the training sets and the ranking in each feature importance vector is obtained. We then plot the average importance rank of each feature across the parameter range; features with the same importance score are each assigned the lowest possible rank. The resultant curve can be seen in Fig. 8A. Although there is not a strong cut off candidate, there is a reasonable jump in average rank at 44, and hence 44 is used as the cut off. We also tested a similar procedure using the average score rather than the average rank (results in Appendix E.2), which produces a much clearer jump with only 6 features. However, concerns about correlated features reducing the importance score and thus removing useful signal, led to favouring the rank-based threshold. The rankings are relatively stable with 42 of the most informative 44 features in the ranking ordering also in the average score based ordering. A list of the selected features can be found in Appendix E.1.

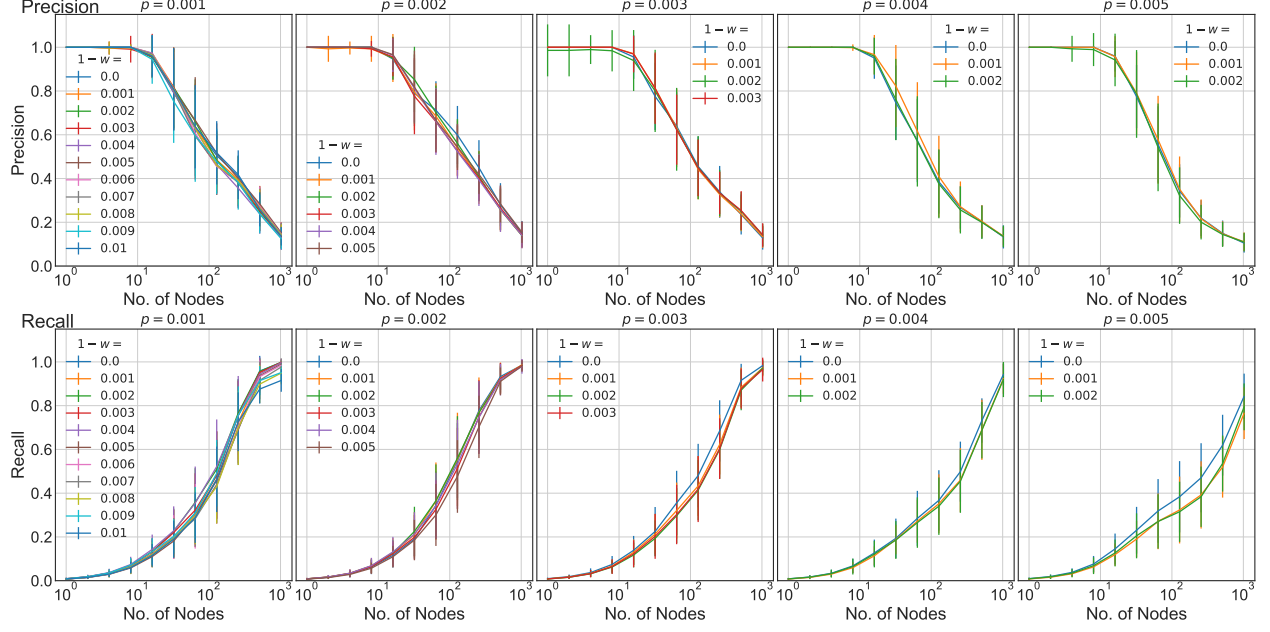


Figure 6: The mean precision (upper plots) and recall (lower plots) for the FEATURE SUM method across the parameter regimes over the training networks. In the set of panels on the left, the mean precision and recall over the training networks is plotted over the parameter regimes (p, w) which have $p = 0.001$, considering the 2^i top scoring nodes, where $i = 0, \dots, 10$. The error bars are given as one estimated standard error over the relevant networks. The remaining panels show the same plots for different densities.

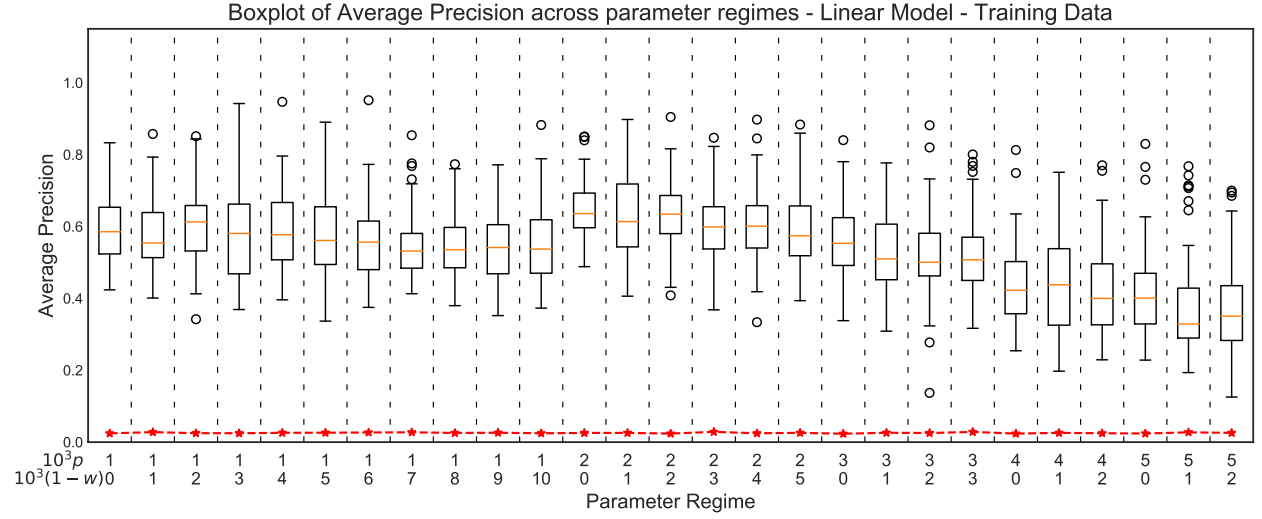


Figure 7: The average precision (computed using scikit-learn [51]), for each of the parameter regimes using the FEATURE SUM method. With a random ordering, the average precision should be equal to the percentage of anomalous nodes [51] (see Sec. 3.8 for details). The red star line shows the largest percentage of anomalies over the networks which are observed in each parameter regime.

With 44 as the cut off, Fig. 8B shows the underlying modules which dominate the selected features. Localisation measures appear in $\approx 52\%$ of the selected features. Furthermore, all of the basic methods

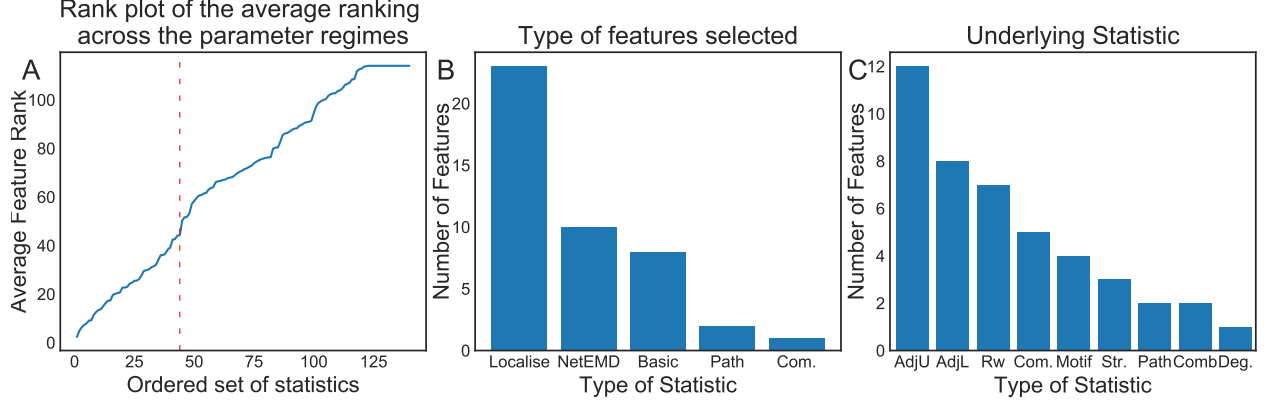


Figure 8: **(A)** Rank plot of the average rank of each of the statistics over the parameter regimes. The red dashed line details a small jump at the 44th feature which is used as a threshold for feature selection. **(B)** Histogram of the underlying type of method behind each feature. **(C)** Histogram of the underlying matrix or statistic used in each feature.

are present in the 44 selected features. Fig. 8C shows the underlying statistic which drives each of the selected features. Around half of the statistics are driven by a version (non-augmented or augmented) of the adjacency matrix.

To explore the variation of the most important features between the parameters regimes, Fig. 9 displays the top 3 features of each of the parameter regimes. Local statistics dominate the figure, with test statistics based on the top 10% of neighbour edges by weight, and paths of size 5, appearing to be important. The high performance of these paths could be related to the fact that the size of the smallest embedded structure is 5 and therefore paths of this size are present in all embedded paths and all embedded rings. Slightly longer path-based methods also are well represented, with paths of size 6 taking up a reasonably large amount of the region where p is large but $1 - w$ is small. While paths of size 7 appears to be important for $(p, 1 - w) = (0.001, 0.01)$, this statistic does not appear in the list of 44 statistics that were selected earlier in this section, as it is not important enough in other parameter regimes.

As the second most important feature, a larger fraction of the more complex measures appear, with features relying on both localisation and NetEMD. The underlying statistic appears to always be the adjacency matrix, perhaps indicating that the adjacency matrix has a stronger signal than the other matrices for the anomalies we are aiming to detect. The third most important feature shows a similar pattern, with many of the NetEMD/Localisation based statistics directly using the adjacency matrix.

Finally, the most important and second most important features appear to contain several regions in which the selected statistic is stable. We take this level of stability as an indication that the number of training examples is reasonably large. In the case of the third most important statistic, we observe a little more variation, which could either indicate that different statistics specialise in different parameter regimes, or it could simply be a consequence of the number of repeats being relatively small.

Using the selected 44 features and the standard parameters in the RandomForestRegressor from Sklearn, we fit a random forest over the full training data set, resulting in 1,890,000 observations with 278,056 anomalies (1.47%). We tested in-sample performance using the same procedure we used in the FEATURE SUM model, achieving near perfect scores. The relevant plots can be seen in Appendix F.

Performance of RANDOM FOREST on the test set. To measure out-of-sample performance, we applied the fitted RANDOM FOREST to the test set. When interpreting these results, it is crucial to understand that the default RandomForestRegressor has 10 trees, which means that each node will be given a score from $\{0, 0.1, \dots, 1.0\}$.

The average precision, as seen in Fig. 10, shows a substantially better performance than was achieved

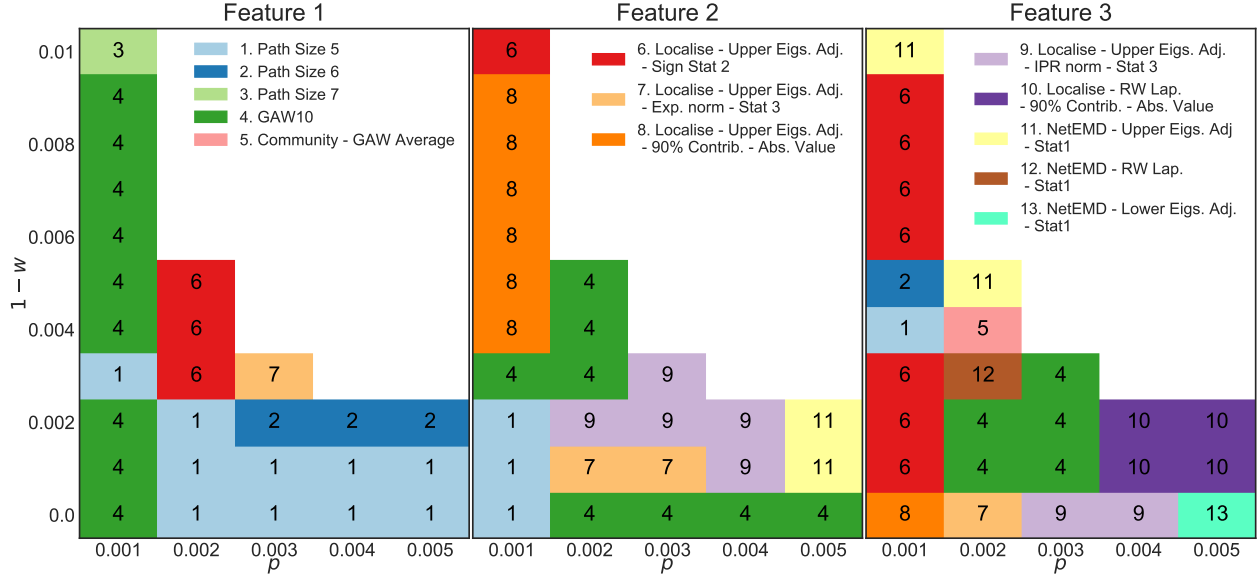


Figure 9: The top three most important statistics in each parameter regime, according to the feature importance calculation performed by Sklearn and detailed in Sec. 3.6. Feature 1 is the most important feature, followed by Feature 2, then Feature 3. To aid comprehension when printed in black and white, we include a numerical index of each feature.

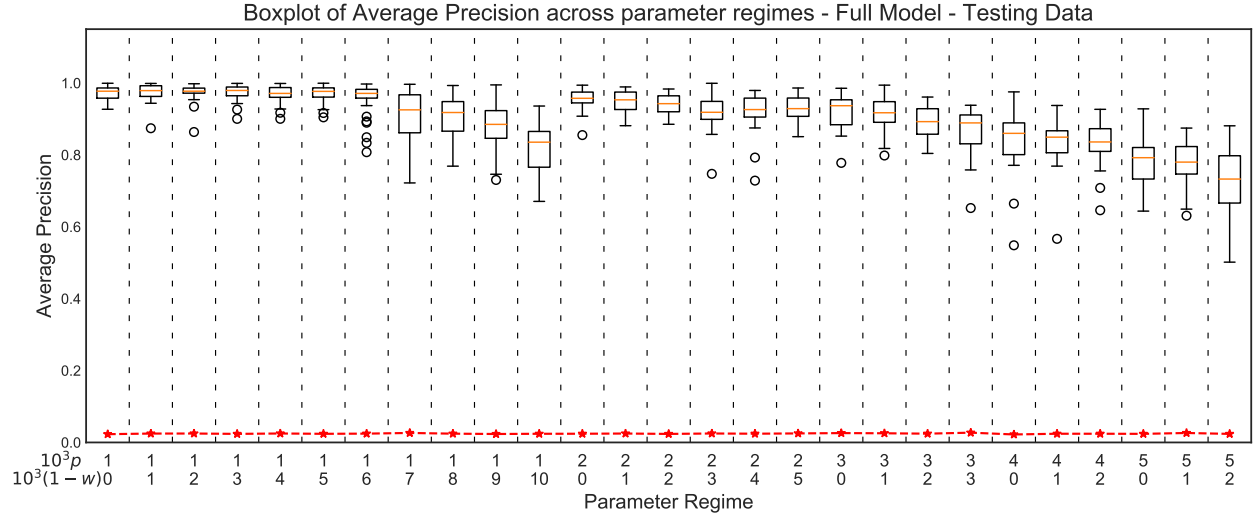


Figure 10: The average precision for each of the parameter regimes using the RANDOM FOREST Model after feature selection and training on the test data. With a random ordering, the average precision should be equal to the percentage of anomalous nodes [51] (see Sec. 3.8 for details). The red star line shows the largest percentage of anomalies over the networks which are observed in each parameter regime.

using the FEATURE SUM model on the training data. For low p and low $1 - w$ (left-hand side of the plot), almost perfect performance is achieved, and while larger values of $1 - w$ appear to reduce the level of performance, the RANDOM FOREST still outperforms the FEATURE SUM method. For low p , the region which shows the different behaviour is also the region in which the feature selection step proposed different features

which were not included in the model (namely Paths of size 7), possibly suggesting that including these features could improve performance in this domain. The mid-range densities $p = \{0.002, 0.003\}$ also show reasonable performance, with performance in the higher densities namely $p = \{0.004, 0.005\}$ performing a little worse but still above the FEATURE SUM method.

In the recall plots of Fig. 11, the recall performance appears to plateau, indicating that there is a subset of nodes which is difficult to classify correctly by the RANDOM FOREST; a similar pattern appears for the FEATURE SUM method, see Fig. 6. The precision plots in Fig. 11 indicate that, in most networks across all parameter regimes, the first 32 nodes are all anomalous nodes in all networks in the ensemble, with the first 64 being almost all anomalies for densities 0.001 – 0.003.

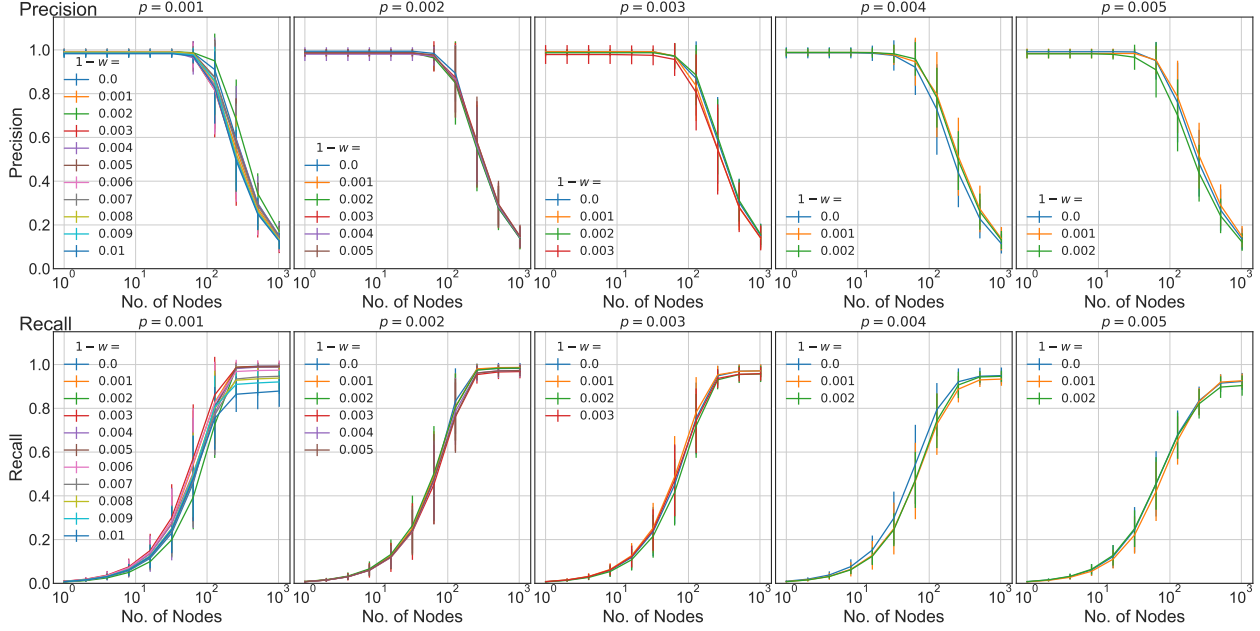


Figure 11: The mean precision (upper plot) and recall (lower plot) across each of the parameter regimes. The set of panels on the left shows the mean precision and recall over the training networks of the parameter regimes which have $p = 0.001$, using the RANDOM FOREST model to compute the scores considering the 2^i top scoring nodes where $i = 0, \dots, 10$. The error bars are given as one estimated standard error over the relevant networks. The remaining panels show the same plots for different densities.

Performance of Oddball on the test set. To further test the performance of our method, we also compared the performance of FEATURE SUM and RANDOM FOREST to that of Oddball [2], see Sec. 3.7. The performance of the combined Oddball statistics can be found in Fig. 12, and the performance on each of the underlying statistics can be seen in Appendix G.2. The performance of Oddball appears to be driven by summaries that are related to the local density (nodes vs edges) and to the deviations in average local weight, namely (and loosely quoting from the *Oddball-lite* code) “egonet nodes vs egonet edges”, “egonet edges vs egonet weight”, “ego out-degree vs ego out-weight” and “ego in-degree vs ego in-weight”.

Fig. 13 shows a plot of the median of the average precision; in all parameter regimes the RANDOM FOREST outperforms Oddball. The FEATURE SUM method, ran on the training dataset, also performs favourably to Oddball, indicating that the underlying features already have sufficient information to capture to anomalies.

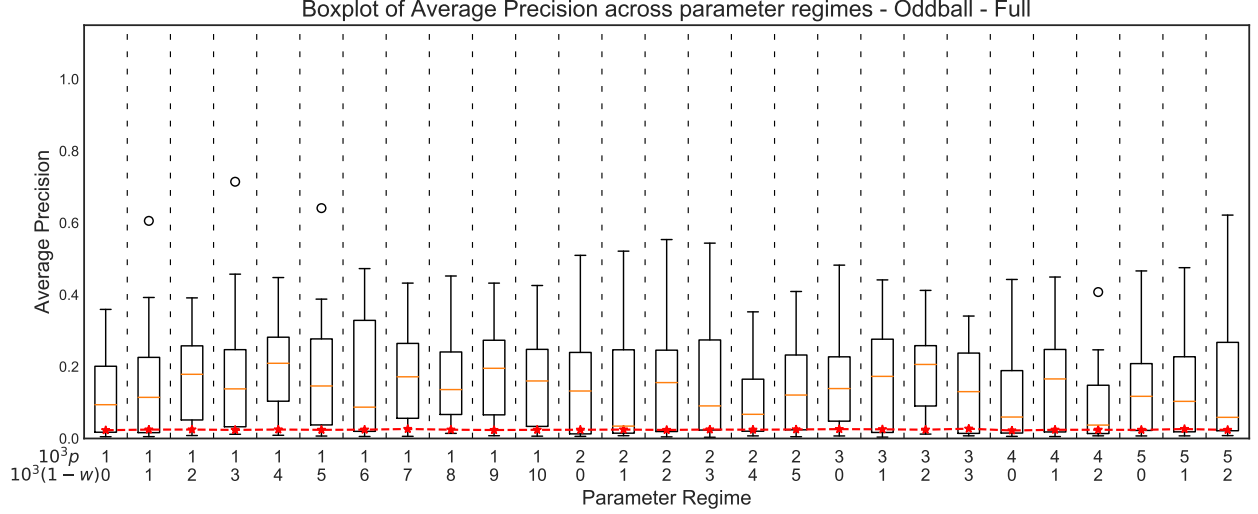


Figure 12: The average precision for each of the parameter regimes, using the sum of Oddball over each of the features. With a random ordering, the average precision should be equal to the percentage of anomalous nodes [51] (see Sec. 3.8 for details). As our networks have different numbers of anomalies to compare our performance against random, we plot (red star line) the largest percentage of anomalies over the networks we observe in each parameter regime.

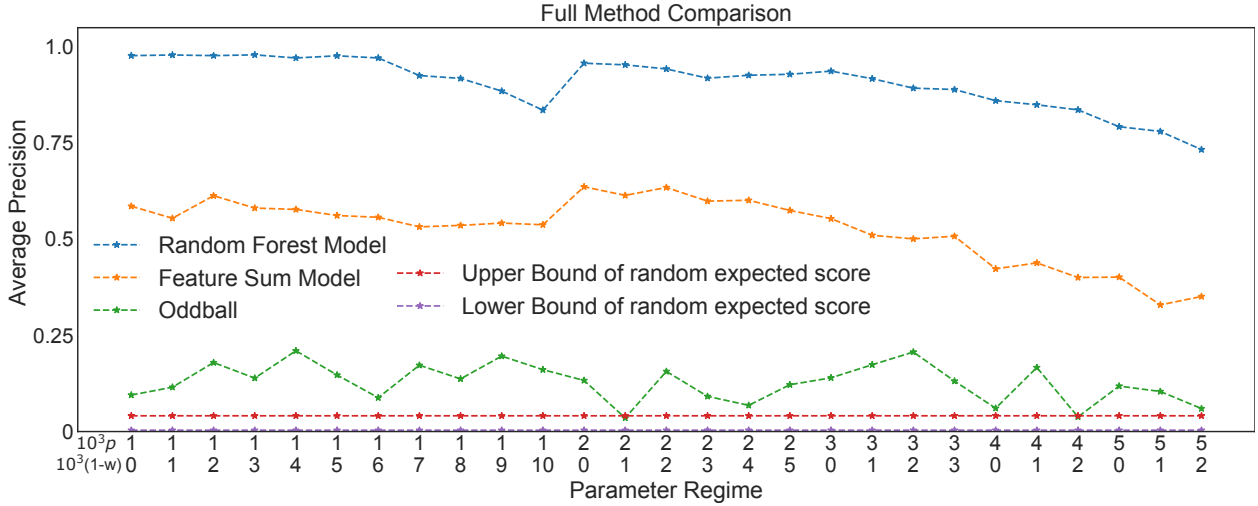


Figure 13: The median of the average precision from scikit-learn [51], for each of the parameter regimes. Oddball and the RANDOM FOREST are applied to the test data, whereas the FEATURE SUM method is applied to the training data. As the average precision under a random ranking is equal to the percentage of anomalies (see Sec. 3.8 for details), and in this model there are between 25 and 400 anomalies embedded (under the assumption of no overlapping structures), we plot the lower bound ($\frac{25}{10000}$) and upper bound ($\frac{400}{10000}$) of the performance of a random ranking.

4.2 Performance on the Accenture Model

For the Accenture generating model described in Sec. 2.2, all of the structures from Fig.2, except the path of size 5, are detectable in the Accenture model under the approximation that the network is a directed

weighted ER network with connection probability $p = 19/55000$ and heavy weights exceeding $w = 0.99$ (see Appendix B.3 for details). We generated 100 networks from the Accenture model and measured the performance of the RANDOM FOREST which is obtained using the training data from the weighted ER network in Sec. 3.6, and of the FEATURE SUM, on this data set, see Fig. 14. Unlike in the weighted ER case, the number of anomalies is fixed in the Accenture model, so that the average of the recall and the precision is well defined.

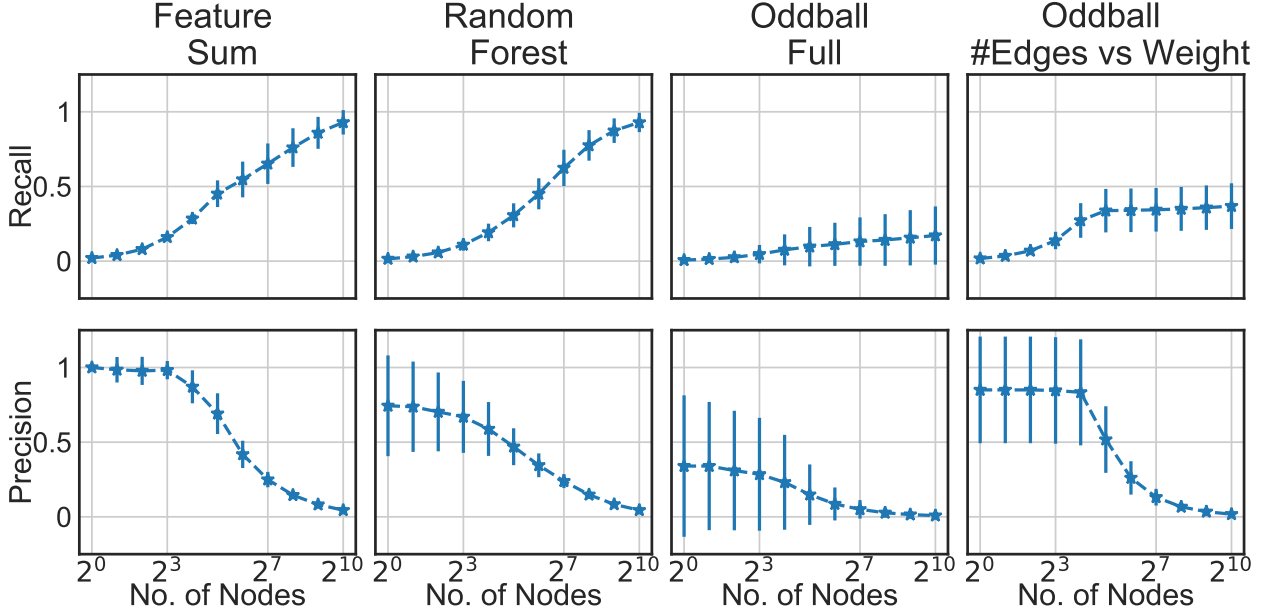


Figure 14: Precision and recall on 100 networks from the Accenture data generator, using the FEATURE SUM and the RANDOM FOREST; averages with error bars (one estimated standard error).

First focusing on the average precision, when analysing Oddball, one of the underlying statistics, namely comparing the number of edges and the weight, outperforms the RANDOM FOREST method, although it is still outperformed by the FEATURE SUM; the results can be seen in Fig. 15. An inspection of the recall and precision plots of Fig. 14 indicate that Oddball Edges vs Weight does very well on the first few nodes, finding the clique nodes, but then, in contrast to FEATURE SUM and RANDOM FOREST, it struggles to find any of the additional planted anomalies. One additional underlying Oddball statistic also highlighted cliques but similarly struggled to uncover other planted anomalies, see G.3) for full results of the underlying statistics.

Concentrating on the 1024 nodes with highest scores, as shown in Fig. 14, both the FEATURE SUM and the RANDOM FOREST methods have an average recall of 0.93. Both of these methods are, on average, able to recover the vast majority of the embedded anomalies within the top 1.8% of the nodes, thus indicating good performance. With regards to precision, on this data set the FEATURE SUM performs better at smaller thresholds, while the RANDOM FOREST does not always achieve a precision close to 1, even for the very top scoring nodes.

Finally, we consider the average precision of all of the methods as shown in Fig. 15. The average precision of the RANDOM FOREST is lower in absolute value than on the weighted ER model, but still considerably higher than the performance of the random classifier. The FEATURE SUM significantly outperforms the RANDOM FOREST on this data set, which could be an indication that the RANDOM FOREST is overfitted. Based on average precision, both the FEATURE SUM and the RANDOM FOREST outperform Oddball, echoing the results from the weighted ER model.

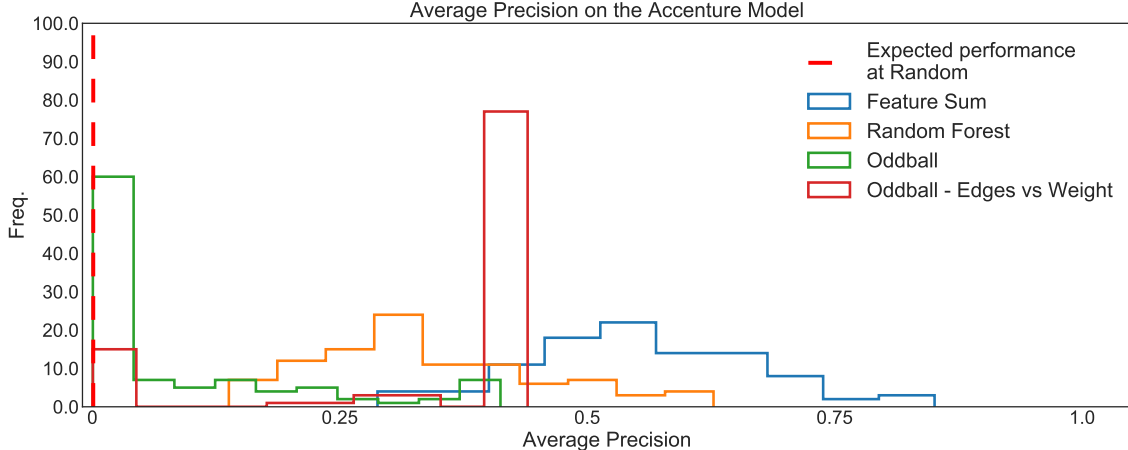


Figure 15: The average precision on 100 networks from the Accenture data generator, using FEATURE SUM, RANDOM FOREST, Oddball, and Oddball - Edges vs Weight. The dotted line shows the performance of the random classifier, which is indistinguishable from the x -axis.

4.3 Finding Non-planted Structures

While we measure the performance based on an embedded set of structures, we now illustrate that the method can uncover structures that we have not embedded. To this end, we inspect the pattern of connections around those nodes which are ranked highly, yet are not part of an embedded anomaly. Keeping in mind that the random forest scores, based on an average over 10 decision trees, take values in $\{0, 0.1, 0.2, \dots, 0.8, 0.9, 1\}$, we performed this analysis in the first of our networks from the Accenture benchmark. We considered the top 8 nodes in the ranking constructed from RANDOM FOREST, corresponding to those nodes with a score of 0.8 or higher. To visualise, we plot the two-hop snowball sampled network, around each of these anomalous nodes. For brevity, in the paper, we only display a subnetwork of the 2 hop snowball sample, namely all edges with weight greater than 1400 (two standard deviations of the null weight distribution), for which there is a path (ignoring direction) of heavy edges to the seed node. The subnetworks are displayed in Fig. 16, whereas the whole snowball samples can be seen in Appendix H.

We observe that each node has a heavy structure around it. In many cases, substantially higher than expected edge weights are present and several heavy flow-like structures, for example in (B), (C), (D), (F), (G) and (H), which were not embedded anomalies.

5 Discussion

Summary This paper presents a novel anomaly detection method which combines spectral approaches with recent advances in network comparison, to uncover unknown anomalies in networks. The method is developed with applications to money laundering in mind. Using such spectral and network comparison approaches, as well as exploring paths in the network, we construct 140 main features. We consider two methods to classify nodes based on the resulting features. The first method, denoted by FEATURE SUM, simply sums over the 140 main features.

To improve on the FEATURE SUM method, we fitted a random forest to the underlying set of 140 main features. The random forest was trained using a synthetic network generator which plants anomalies in weighted directed Erdős-Rényi networks. Using this network generator, we constructed 1,890 training networks with 10,000 nodes each, sampled evenly across the parameter range of interest, which is determined by requiring that some known anomalies have expected counts less than 1 under the weighted directed Erdős-Rényi model. We select 44 features, focusing on features which have high scores across the parameter regimes of interest, and we fit a random forest using these features; we denote the resulting methodology as

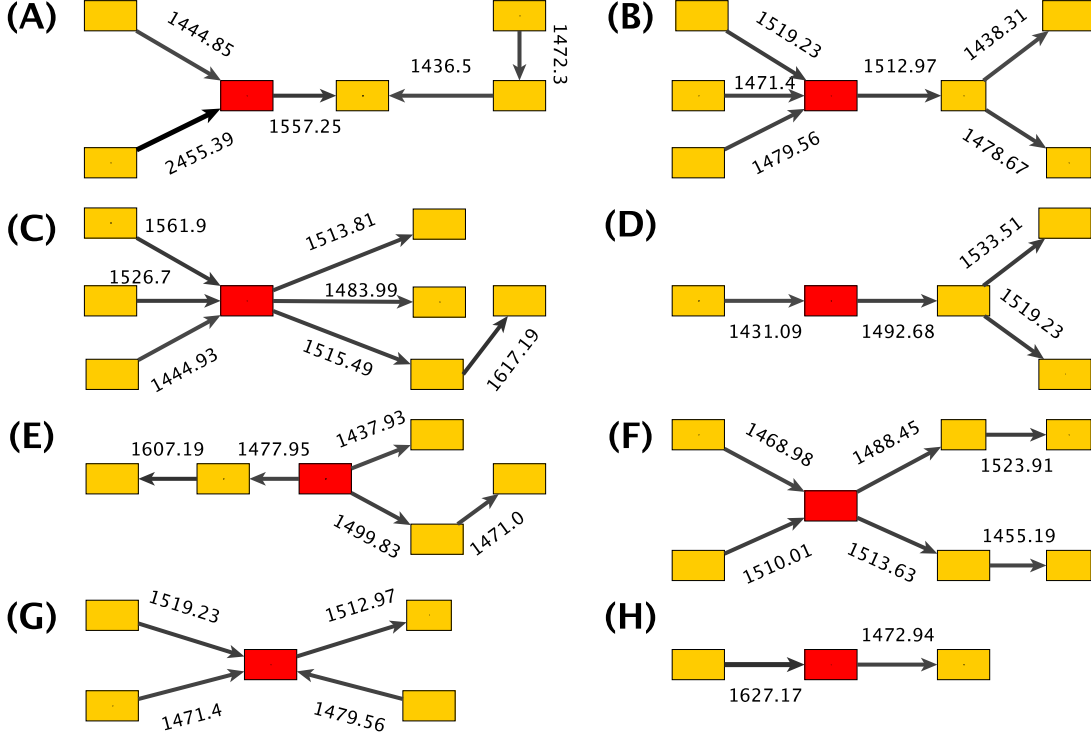


Figure 16: Figures shows the heavy structures around nodes are identified as potential anomalies using RANDOM FOREST on the Accenture data generator. The identified node is in red with the remaining nodes in yellow, the edges show their weight both by their colour and the annotated weight next to each edge.

RANDOM FOREST. It performs favourably on a test set in comparison to FEATURE SUM, as well as when compared to random selection and OddBall, a popular anomaly detection method introduced in [2].

We have also explored in more detail the features which appear to be driving our performance in the weighted ER network, and observed that many features relate to localisation and basic statistics. Around half of the selected features are driven by eigenvectors of the symmetrised augmented adjacency matrix. This fact could be related to the augmentation step, which repeatedly adds edges around heavy paths of size 3, as described in Sec. 3.2, and can be thought of as a form of regularisation of a sparse graph. Thus, in augmented matrices, heavy structures are turned into cliques. In contrast, the most important features, as computed by the feature importance score across the parameter ensemble, are dominated by non-augmented statistics related to small structures around nodes, such as the strength of the top 10% of edges, or short but heavy weighted paths.

The FEATURE SUM and the RANDOM FOREST were employed to detect anomalies in an unseen network model developed by co-authors PR and ML at Accenture. On this network, both the FEATURE SUM and the RANDOM FOREST outperform random classification, and more importantly, the Oddball algorithm. Both FEATURE SUM and RANDOM FOREST extract, on average, 92.3% of anomalous nodes within the top 1.86% ranking nodes. On the Accenture generating model, using the average precision measure, FEATURE SUM outperforms RANDOM FOREST.

We further validated our method by looking at the local neighbourhood of nodes that were not embedded in the graph, yet were highlighted by our method. A visual inspection of the heavy structures (edge weights more than 2 standard deviations above the mean) showed heavy star-like structures, with several examples having a flow-like structure, giving evidence that these structures are indeed anomalous.

The method used in this paper includes a number of parameter choices. Usually, p -values of tests are used when they are below 5%; other choices are possible. In the generating models, the weights of the anomalies were chosen such that the anomalies have a fairly strong signal, with other choices being also possible. As a guideline, the edge probability p should be chosen such that the resulting density of the network matches that of the network of interest, and the weight parameter w should be chosen such that anomalies have low expected value (taken as less than 1 in this paper) under the model without added anomalies.

Potential improvements There are a few drawbacks of the pipeline we proposed in this paper. First, the limited number of trees in a default *RandomForestRegressor* reduces the number of possible states, which further limits our ability to distinguish between nodes. A solution to this problem would be to simply increase the number of trees, which we will pursue in future work. It should also be noted that a future version of *RandomForestRegressor* will have 100 trees by default. Second, the network augmentation used to perform community detection, starts to break down at higher densities; at such densities, the expected number of heavy paths under the null model is fairly high. Community detection on these dense networks produces a small number of very large communities, along with some very small communities. While a high edge density is rarely encountered in large real networks of financial transactions, an exploration of replacing the fixed augmentation threshold with a threshold which is adjusted to the network density could be of general interest.

It remains to note that our implementation of the routines described in this paper is not fast. This is partly a consequence of our choice of programming language (Python), but also of the assumptions about an unknown null model, and the aim to detect general anomalies, which prevented us from specialising either on the anomalies or the null model. There are several future directions one could pursue, in order to make this method substantially faster and thus scalable to very large networks. Firstly, as the run-time of this algorithm is heavily driven by the number of null replicates used, with the complexity of the algorithm scaling multiplicatively with the number of replicates, reducing the number of replicates is a first option. Secondly, throughout this paper, the so-called resolution parameter in modularity-based community detection has been fixed at 1. To reduce the number of communities, and thus computational complexity, one could modify this parameter. Thirdly, there are clear computational advances from exploiting the embarrassingly parallel nature of the multiple communities, from moving the development language from Python to C, and finally, from taking advantage of new speed-ups in NetEMD [24].

Future work There are also some promising avenues for further research. Firstly, many of the statistics in our method are expensive to compute. It would be interesting to assess the impact on performance of removing some features, such as long path counts, which appear as important in only a few parameter settings and which take a long time to compute. Moreover, in future work we shall explore training sets which are less homogeneous than the set we used. We shall also investigate refined weightings of the summands in FEATURE SUM. A more systematic analysis of the non-planted anomalies which the methods detect is also of interest.

Thus far, the method developed in this paper assumes that the underlying network is static. It would be interesting to expand these approaches for temporal networks (e.g., as treated in [35]), and to incorporate node level features to uncover anomalies which otherwise cannot be extracted from static topological data alone. Yet another interesting direction is to no longer consider the symmetrised matrix $\mathbf{W}^{(s)} = \mathbf{W} + \mathbf{W}^T$ (where $\mathbf{W}$ is the adjacency matrix of the given directed graph), but to rely on recent developments in the literature for clustering directed networks. The algorithms proposed in [18] directly rely on a (potentially normalised) complex-valued Hermitian matrix representation of the asymmetric adjacency matrix $\mathbf{W}$, an approach which could also be adapted to the setting considered in our current anomaly detection pipeline.

Finally, in real networks, the nodes which have a high anomaly score should be investigated further with respect to their properties. This question falls under the area of behavioural analysis, and hence falls outside the scope of this paper.

Acknowledgements We thank Nicolas Guernion, Stephen Law, Luis Ospina Forero, Martin O’Reilly, Anastasia Shteyn, Kieran Towey, and Mahlet Zimeta for useful discussions and support. This work was funded by EPSRC grant EP/N510129/1 at The Alan Turing Institute and Accenture Plc.

References

- [1] Leman Akoglu, Mary McGlohon, and Christos Faloutsos. Anomaly detection in large graphs. In *In CMU-CS-09-173 Technical Report*, 2009.
- [2] Leman Akoglu, Mary McGlohon, and Christos Faloutsos. Oddball: Spotting anomalies in weighted graphs. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 410–421. Springer, 2010.
- [3] Leman Akoglu, Hanghang Tong, and Danai Koutra. Graph based anomaly detection and description: a survey. *Data mining and knowledge discovery*, 29(3):626–688, 2015.
- [4] André Altmann, Laura Toloşi, Oliver Sander, and Thomas Lengauer. Permutation importance: a corrected feature importance measure. *Bioinformatics*, 26(10):1340–1347, 2010.
- [5] Arash A Amini, Aiyu Chen, Peter J Bickel, Elizaveta Levina, et al. Pseudo-likelihood methods for community detection in large sparse networks. *The Annals of Statistics*, 41(4):2097–2122, 2013.
- [6] T Aynaoud. Python-louvain: Implementation of the louvain algorithm in python. <https://github.com/taynaud/python-louvain>, 2018.
- [7] V. M. Babič and V. F. Lazutkin. Eigenfunctions concentrated near a closed geodesic. *Problems of Mathematical Physics, Spectral Theory, Diffraction Problems (Russian)*, 2:15–25., 1967.
- [8] Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31–39, 2011. doi: 10.1109/MCSE.2010.118. URL <https://aip.scitation.org/doi/abs/10.1109/MCSE.2010.118>.
- [9] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10): P10008, 2008.
- [10] Richard J Bolton and David J Hand. Statistical fraud detection: A review. *Statistical science*, pages 235–249, 2002.
- [11] L. Breiman and A. Cutler. Random forests. online documentation. http://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm, 2004.
- [12] Markus M Breunig, Hans-Peter Kriegel, Raymond T Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *ACM sigmod record*, volume 29, pages 93–104. ACM, 2000.
- [13] Nicolas Burq and Maciej Zworski. Bouncing ball modes and quantum chaos. *Siam Review*, 47(1):43–49, 2005.
- [14] Kamalika Chaudhuri, Fan Chung, and Alexander Tsiatas. Spectral clustering of graphs with general degrees in the extended planted partition model. In *Conference on Learning Theory*, pages 35–1, 2012.
- [15] Zhengzhang Chen, William Hendrix, and Nagiza F Samatova. Community-based anomaly detection in evolutionary networks. *Journal of Intelligent Information Systems*, 39(1):59–85, 2012.
- [16] Andrea Fronzetti Colladon and Elisa Remondi. Using social network analysis to prevent money laundering. *Expert Systems with Applications*, 67:49–58, 2017.

- [17] M. Cucuringu, V. Blondel, and P. Van Dooren. Extracting spatial information from networks with low order eigenvectors. *Physical Review E*, 87, 2013.
- [18] M. Cucuringu, H. Li, H. Sun, and L. Zanetti. Hermitian matrices for clustering directed graphs: insights and applications. *under review*, 2019.
- [19] Mihai Cucuringu and Michael W Mahoney. Localization on low-order eigenvectors of data matrices. *arXiv preprint arXiv:1109.1355*, 2011.
- [20] Linda Delamaire, HAH Abdou, John Pointon, et al. Credit card fraud and detection techniques: a review. *Banks and Bank systems*, 4(2):57–68, 2009.
- [21] Rafał Dreżewski, Jan Sepielak, and Wojciech Filipkowski. System supporting money laundering detection. *Digital Investigation*, 9(1):8–21, 2012.
- [22] William Eberle and Lawrence Holder. Discovering structural anomalies in graph-based data. In *Data Mining Workshops, 2007. ICDM Workshops 2007. Seventh IEEE International Conference on*, pages 393–398. IEEE, 2007.
- [23] William Eberle, Jeffrey Graves, and Lawrence Holder. Insider threat detection using a graph-based approach. *Journal of Applied Security Research*, 6(1):32–81, 2010.
- [24] Andrew Elliott and Martin O’Reilly. Algorithmic improvements and optimality guarantees for netemd. In preparation, 2018.
- [25] Andrew Elliott, Martin O’Reilly, Luis Ospina-Forero, Gesine Reinert, and Anatol E Wegner. Netdist: Network comparison in R. *In preparation*, 2018.
- [26] Andrew Elliott, Martin O’Reilly, Luis Ospina-Forero, Gesine Reinert, and Anatol E Wegner. Github repository: Network comparison. <https://github.com/alan-turing-institute/network-comparison>, 2018.
- [27] Jing Gao, Nan Du, Wei Fan, Deepak Turaga, Srinivasan Parthasarathy, and Jiawei Han. A multi-graph spectral framework for mining multi-source anomalies. In *Graph Embedding for Pattern Analysis*, pages 205–227. Springer, 2013.
- [28] Laurent Gautier. rpy2 - r in python. Online <https://bitbucket.org/rpy2/rpy2/overview>, 2018.
- [29] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15, Pasadena, CA USA, August 2008.
- [30] Steven M Heilman and Robert S Strichartz. Localized eigenfunctions: Here you see them, there you don’t. *Notices of the AMS*, 57(5):624–629, 2010.
- [31] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55. URL <https://aip.scitation.org/doi/abs/10.1109/MCSE.2007.55>.
- [32] Jiashun Jin et al. Fast community detection by score. *The Annals of Statistics*, 43(1):57–89, 2015.
- [33] Eric Jones, Travis Oliphant, Pearu Peterson, et al. SciPy: open source scientific tools for Python. <http://www.scipy.org/>, 2001–. URL <http://www.scipy.org/>.
- [34] Peter W Jones, Mauro Maggioni, and Raanan Schul. Manifold parametrizations by eigenfunctions of the laplacian and heat kernels. *Proceedings of the National Academy of Sciences*, 105(6):1803–1808, 2008.

- [35] MI Knight, MA Nunes, and GP Nason. Modelling, detrending and decorrelation of network time series. *arXiv preprint arXiv:1603.03221*, 2016.
- [36] Asma S Larik and Sajjad Haider. Clustering based anomalous transaction reporting. *Procedia Computer Science*, 3:606–610, 2011.
- [37] Nhien An Le Khac, Sammer Markos, and M-Tahar Kechadi. A data mining-based solution for detecting suspicious money laundering cases in an investment bank. In *Advances in Databases Knowledge and Data Applications (DBKDA), 2010 Second International Conference on*, pages 235–240. IEEE, 2010.
- [38] Benjamin Miller, Nadya Bliss, and Patrick J Wolfe. Subgraph detection using eigenvector l1 norms. In *Advances in Neural Information Processing Systems*, pages 1633–1641, 2010.
- [39] Benjamin A Miller, Nadya T Bliss, and Patrick J Wolfe. Toward signal processing theory for graphs and non-euclidean data. In *Acoustics Speech and Signal Processing (ICASSP), 2010 IEEE International Conference on*, pages 5414–5417. IEEE, 2010.
- [40] Benjamin A Miller, Michelle S Beard, and Nadya T Bliss. Eigenspace analysis for threat detection in social networks. In *Information Fusion (FUSION), 2011 Proceedings of the 14th International Conference on*, pages 1–7. IEEE, 2011.
- [41] Benjamin A Miller, Michelle S Beard, and Nadya T Bliss. Matched filtering for subgraph detection in dynamic networks. In *Statistical Signal Processing Workshop (SSP), 2011 IEEE*, pages 509–512. IEEE, 2011.
- [42] Benjamin A Miller, Nicholas Arcolano, and Nadya T Bliss. Efficient anomaly detection in dynamic, attributed graphs: Emerging phenomena and big data. In *Intelligence and Security Informatics (ISI), 2013 IEEE International Conference on*, pages 179–184. IEEE, 2013.
- [43] Benjamin A Miller, Michelle S Beard, Patrick J Wolfe, and Nadya T Bliss. A spectral framework for anomalous subgraph detection. *IEEE Transactions on Signal Processing*, 63(16):4191–4206, 2015.
- [44] Joshua Neil, Curtis Hash, Alexander Brugh, Mike Fisk, and Curtis B Storlie. Scan statistics for the online detection of locally anomalous subgraphs. *Technometrics*, 55(4):403–414, 2013.
- [45] Eric WT Ngai, Yong Hu, YH Wong, Yijun Chen, and Xin Sun. The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3):559–569, 2011.
- [46] R. J. Nowling. Blogpost: Feature correlation and feature importance bias with random forests. <http://rnowling.github.io/machine/learning/2015/08/11/random-forest-correlation-bias.html>, 2015.
- [47] Travis E Oliphant. *A guide to NumPy*, volume 1. Trelgol Publishing USA, 2006.
- [48] Luis. Ospina-Forero, Charlotte M. Deane, , and Gesine. Reinert. Assessment of model fit via network comparison methods based on subgraph counts. *Journal of Complex Networks*, to appear., 2018.
- [49] Romualdo Pastor-Satorras and Claudio Castellano. Distinct types of eigenvector localization in networks. *Scientific reports*, 6:18847, 2016.
- [50] Ebberth L Paula, Marcelo Ladeira, Rommel N Carvalho, and Thiago Marzagão. Deep learning anomaly detection as support fraud investigation in brazilian exports and anti-money laundering. In *Machine Learning and Applications (ICMLA), 2016 15th IEEE International Conference on*, pages 954–960. IEEE, 2016.

- [51] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011.
- [52] Brandon Pincombe. Anomaly detection in time series of graphs using arma processes. *Asor Bulletin*, 24(4):2, 2005.
- [53] Tai Qin and Karl Rohe. Regularized spectral clustering under the degree-corrected stochastic block-model. In *Advances in Neural Information Processing Systems*, pages 3120–3128, 2013.
- [54] Mark Rudelson. Delocalization of eigenvectors of random matrices. lecture notes. *arXiv preprint arXiv:1707.08461*, 2017.
- [55] S Russ, B Sapoval, and O Haeblerlé. Irregular and fractal resonators with neumann boundary conditions: Density of states and localization. *Physical Review E*, 55(2):1413, 1997.
- [56] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets. *PLoS one*, 10(3):e0118432, 2015.
- [57] David Savage, Xiuzhen Zhang, Xinghuo Yu, Pauline Chou, and Qingmai Wang. Anomaly detection in online social networks. *Social Networks*, 39:62–70, 2014.
- [58] David Savage, Qingmai Wang, Pauline Lienhua Chou, Xiuzhen Zhang, and Xinghuo Yu. Detection of money laundering groups using supervised learning in networks. *CoRR*, abs/1608.00708, 2016.
- [59] Marco Schreyer, Timur Sattarov, Damian Borth, Andreas Dengel, and Bernd Reimer. Detection of anomalies in large scale accounting data using deep autoencoder networks. *arXiv preprint arXiv:1709.05254*, 2017.
- [60] Ted E Senator, Henry G Goldberg, Jerry Wooton, Matthew A Cottini, AF Umar Khan, Christina D Klinger, Winston M Llamas, Michael P Marrone, and Raphael WH Wong. Financial crimes enforcement network ai system (fais) identifying potential money laundering from reports of large cash transactions. *AI magazine*, 16(4):21, 1995.
- [61] Mike Spivey. Odd order moments of a symmetrical distribution. Mathematics Stack Exchange, 2011. URL <https://math.stackexchange.com/q/72454>. <https://math.stackexchange.com/q/72454> (version: 2011-10-14).
- [62] Srngn. The mathematics of decision trees, random forest and feature importance in scikit-learn and spark. Blog Post. <https://medium.com/@srngn/the-mathematics-of-decision-trees-random-forest-and-feature-importance-in-scikit-learn-and-spark-f2861df67e3>, May 2018.
- [63] Wanhua Su, Yan Yuan, and Mu Zhu. A relationship between the average precision and the area under the roc curve. In *Proceedings of the 2015 International Conference on The Theory of Information Retrieval*, pages 349–352. ACM, 2015.
- [64] Jun Tang and Jian Yin. Developing an intelligent data discriminating system of anti-money laundering based on svm. In *Machine Learning and Cybernetics, 2005. Proceedings of 2005 International Conference on*, volume 6, pages 3453–3457. IEEE, 2005.
- [65] Hanghang Tong and Ching-Yung Lin. Non-negative residual matrix factorization with application to graph anomaly detection. In *Proceedings of the 2011 SIAM International Conference on Data Mining*, pages 143–153. SIAM, 2011.
- [66] Financial Fraud Action UK. Fraud the facts 2017. Available at <https://www.financialfraudaction.org.uk/fraudfacts17/>, 2017.

- [67] Anatol E Wegner, Luis Ospina-Forero, Robert E Gaunt, Charlotte M Deane, and Gesine Reinert. Identifying networks with common organizational principles. *Journal of Complex Networks*, page cny003, 2018. doi: 10.1093/comnet/cny003. URL <http://dx.doi.org/10.1093/comnet/cny003>.
- [68] Wen Wen. Comparison of world trade networks. Masters Thesis, 2016.
- [69] Jarrod West, Maumita Bhattacharya, and Rafiqul Islam. Intelligent financial fraud detection practices: An investigation. In *International Conference on Security and Privacy in Communication Systems*, pages 186–203. Springer, 2014.
- [70] X. Xu and G. Reinert. Triad-based comparison and signatures of directed networks. In *Proceeding of Complex Networks 2018*. Springer, 2018.
- [71] Zhongfei Mark Zhang, John J Salerno, and Philip S Yu. Applying data mining in investigating money laundering crimes. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 747–752. ACM, 2003.

Appendix

For convenience here is Table of Contents which includes the main body of the paper.

Contents

1	Introduction	2
2	Network Models and Network Data	4
2.1	Synthetic Data Set: Weighted ER Network	4
2.2	Synthetic Data Set: Accenture model	5
3	Anomaly Detection Method	6
3.1	Basic Detection Module	7
3.2	Community Detection	7
3.3	Spectral Localisation	8
3.4	The NetEMD Module	10
3.5	Finding Heavy Paths	13
3.6	Combining the scores	13
3.7	Oddball	15
3.8	Performance Measures	15
4	Results	16
4.1	Results on the Weighted ER Network	16
4.1.1	FEATURE SUM	16
4.1.2	RANDOM FOREST	16
4.2	Performance on the Accenture Model	21
4.3	Finding Non-planted Structures	23
5	Discussion	23
A	Implementation Details and Special Cases	32
A.1	Configuration Model Implementation	32
A.2	Localisation	33
A.2.1	Eigenvector Computation	33
A.2.2	Node Selection Derivation for Norm-based Localisation Statistics	33
A.2.3	Sign-based Localisation Statistics	34
A.2.4	Configuration Null Model Replicates which have Insufficient Size	35
A.3	NetEMD Implementation Details	35
A.4	PathFinder	36
B	Regions of Detectability	36
B.1	Derivation of Regions of Detectability	37
B.2	Proof that Path 5 is the most restrictive case for $k \geq 5$	39
B.3	Approximate Detectability of the Anomalies in the Accenture Model	41
C	Proof that the Selection of the Direction in the Eigenvector Terminates	42

D	List of All Anomaly Detection Features	43
D.1	Basic Node Statistics	43
D.2	Path Statistics	44
D.3	NetEMD Measures	44
D.4	Localisation Statistics	45
D.5	Additional Features	47
E	Feature Selection	48
E.1	List of Selected Features	48
E.2	Alternative Feature Selection	49
E.3	Full Feature Selection Plots	49
F	Within-Sample Scores for the RANDOM FOREST Method	49
G	Oddball	50
G.1	Oddball for Directed networks	50
G.2	Oddball Individual Statistics - Weighted ER Network	52
G.3	Oddball Individual Statistics - Accenture Model	52
H	Exploring Anomalous Non-Embedded Nodes	67

A Implementation Details and Special Cases

We present additional details on the underlying network construction and the implementation of the methods, paying particular attention to special cases (boundary cases), such as graphs which do not have a sufficient number of eigenvectors.

A.1 Configuration Model Implementation

The null model for the underlying network is a slightly modified configuration model. The implementation is heavily inspired by the following procedure from the directed NetworkX implementation [29] of the configuration model on a fixed finite node set with fixed in-degrees and out-degrees. In this setting, each node is given as many outgoing stubs as its out-degree, and as many incoming stubs as its in-degree. Nodes with degree 0 are excluded. The next steps are as follows.

1. Construct a list of outgoing stubs and incoming stubs.
2. Construct a list of weights.
3. Shuffle the list of incoming stubs.
4. Shuffle the list of weights.
5. Construct a network from the resultant set of outgoing stubs, incoming stubs and weights, by connecting outgoing stubs with incoming stubs at random.
6. Remove all self-loops.
7. For multiple edges, randomly select an edge at random, and remove all others.
8. Remove all resulting degree 0 nodes.

Note that for multi-edges, an edge is selected uniformly at random between the multi-edges regardless of the edge weight.

A.2 Localisation

In the computation of localisation, we made a small number of implementation choices which are not obvious from the description of the method in the main body. In this section, we describe these implementation details.

A.2.1 Eigenvector Computation

For computational reasons, in cases where the number of eigenvectors required is less than the size of the matrix minus one, SciPy’s (Ref. [33]) sparse eigenvector routines are used, which only computes the top k eigenvectors.

However, during the experiments, the sparse routines were occasionally hanging, which may indicate that the underlying routines were unable to converge, possibly due to badly conditioned matrices. In contrast, the non-sparse eigenvector routines had comparatively little problem with these matrices. Keeping the same random seed, we used the following procedure.

1. Attempt to compute the eigenvectors using the fast sparse methods.
2. If the fast method has completed within 30 seconds, then return the resultant eigenvectors.
3. If the method has not completed within 30 seconds, then stop the fast sparse approach and use the slower eigenvector routine from NumPy which computes all of the eigenvectors.

A.2.2 Node Selection Derivation for Norm-based Localisation Statistics

In many of the approaches in localisation statistics, selecting the nodes that cause the localisation is comparatively simple; for the norm-based approaches (i.e., the exponential norm and the inverse participation coefficient), it is a little more arbitrary. A main issue is that the eigenvectors themselves do not include information about how likely they are. To address this, we construct 4 different statistics using the following procedure.

1. The first statistics are the absolute values of entries of the eigenvector, divided by the absolute value of the constant vector, scaled to have Euclidean norm 1.
2. To identify nodes for which the eigenvector has an entry larger than expected at random, we carry out a corresponding Monte Carlo test. For p -values that are below 0.5, we perform the normal c.d.f. transformation to obtain a feature vector, for those above 0.5 we assign it a value of 0.
3. For the third statistic, the inverse normal c.d.f. of the p -value of the norm-based test statistic which highlighted the localisation in this vector is assigned to all nodes whose element in the relevant eigenvector is greater than the average of the largest element in absolute value over the null distribution.
4. Finally, combining the ideas behind the first and third statistic, we follow the procedure in the third statistic, but rather than giving the relevant nodes the value of the transformed eigenvector level inverse normal c.d.f. p -value, we assign them their element-wise absolute value.

Mathematically, this is equivalent to the following. Let Ψ be the set of m eigenvectors generated from the null distribution (shared with the Monte Carlo p -value), and let v^* be the observed eigenvector. Let us represent the value of feature a for node u by $g_a(u)$, $a = 1, 2, 3, 4$. Denoting by n the number of nodes, the first feature is given by

$$g_1(u) = \sqrt{n}|v_u^*|. \quad (11)$$

For the remaining features, let $S = \{\max_j |x_j| : x \in \Psi\}$. Furthermore, let $t(a)$ be the Monte Carlo p -value of an observation a from over the empirical null distribution from S , i.e. $t(u) = \frac{1}{m+1} + \frac{1}{m+1} \sum_{x \in S} \mathbb{1}(v_u^* \leq x)$.

Let $\tilde{s}$ be the average over S , and let p_{val} be the p -value of the norm-based test statistic which highlighted localisation in this vector. The remaining three features are

$$g_2(u) = \begin{cases} \Phi^{-1}(1 - t(u)) & \text{if } t(u) < 0.5 \\ 0 & \text{else,} \end{cases} \quad (12)$$

$$g_3(u) = \begin{cases} \text{Max}(\Phi^{-1}(1 - p_{val}), 0) & \text{if } |v_u^*| \geq \tilde{s} \\ 0 & \text{else,} \end{cases} \quad (13)$$

$$g_4(u) = \begin{cases} |v_u^*| & \text{if } |v_u^*| \geq \tilde{s} \\ 0 & \text{else,} \end{cases} \quad (14)$$

where Φ^{-1} is the inverse c.d.f. of a mean 0 variance 1 normal distribution.

A.2.3 Sign-based Localisation Statistics

As discussed in the main text, there are a small number of special cases for the sign-based statistic, but for brevity we did not include them in the main body. Here, we adjust the test statistic to address these cases, and then adjust the procedure which constructs features from vectors that appear to deviate from their expectation.

Test Statistic First, we let $\theta(G) = \text{Min}(20, \text{Number of Non-Trivial Eigenvectors})$, and we recall that the sign-based test statistic for eigenvector v from G is $\frac{\min(N_{pos}(v), N_{neg}(v))}{\theta(G)}$, where $N_{pos}(v) = \sum_i \mathbb{1}(v_i > 0)$ and $N_{neg}(v) = \sum_i \mathbb{1}(v_i < 0)$.

Many of the special cases relate to the setting where all entries have the same sign, resulting in a test statistic of 0. A test statistic of 0 is problematic, as we are interested in cases where a small non-zero number of entries have the same sign (and thus those entries are anomalous). Thus, rather than assigning 0 to the test statistic, we assign it $\theta(G)$, alleviating the problem. The resulting test statistic is

$$\frac{1}{\theta(G)} \text{Min}\left(N_{pos}(v) + \mathbb{1}(N_{pos}(v) = 0)\theta(G), N_{neg}(v) + \mathbb{1}(N_{neg}(v) = 0)\theta(G)\right),$$

which is equivalent to the original test statistic when $N_{pos} \neq 0$ and $N_{neg} \neq 0$.

Feature Construction As with the other localisation features, in vectors where the test statistic appears to deviate from expectation, we attempt to highlight the nodes causing the deviation. In doing so, we consider three special cases, namely the presence of vectors where N_{neg} or N_{pos} equals 0, if $N_{neg} = N_{pos}$, and finally the extremely rare case where more than 50% of the nodes are considered anomalous.

The original features, as described in the paper, are given by

$$\begin{aligned} \text{SignStat}_i^{(1)} &= \mathbb{1}(N_{pos}(v) < N_{neg}(v))\mathbb{1}(v_i > 0)\Phi^{-1}(1 - p_{val}) \\ &\quad + \mathbb{1}(N_{pos}(v) > N_{neg}(v))\mathbb{1}(v_i < 0)\Phi^{-1}(1 - p_{val}), \\ \text{SignStat}_i^{(2)} &= \frac{\mathbb{1}(N_{pos}(v) < N_{neg}(v))\mathbb{1}(v_i > 0)\Phi^{-1}(1 - p_{val})}{N_{pos}(v)} \\ &\quad + \frac{\mathbb{1}(N_{pos}(v) > N_{neg}(v))\mathbb{1}(v_i < 0)\Phi^{-1}(1 - p_{val})}{N_{neg}(v)}, \end{aligned}$$

where Φ^{-1} is the inverse of the c.d.f. of a normal distribution with mean 0 and variance 1, and $\mathbb{1}$ is the indicator function.

When N_{pos} or N_{neg} equals zero, we replace N_{pos} with $\widehat{N}_{pos} = N_{pos} + \mathbb{1}(N_{pos} = 0)|v|$ and we replace N_{neg} with $(\widehat{N}_{neg} = N_{neg} + \mathbb{1}(N_{neg} = 0)|v|)$, leading to the following features

$$\begin{aligned} \text{SignStat}_i^{(1)} &= \mathbb{1}(\widehat{N}_{pos}(v) < \widehat{N}_{neg}(v))\mathbb{1}(v_i > 0)\Phi^{-1}(1 - p_{val}) \\ &\quad + \mathbb{1}(\widehat{N}_{pos}(v) > \widehat{N}_{neg}(v))\mathbb{1}(v_i < 0)\Phi^{-1}(1 - p_{val}), \end{aligned} \quad (15)$$

$$\begin{aligned} \text{SignStat}_i^{(2)} &= \frac{\mathbb{1}(\widehat{N}_{pos}(v) < \widehat{N}_{neg}(v))\mathbb{1}(v_i > 0)\Phi^{-1}(1 - p_{val})}{\widehat{N}_{pos}(v)} \\ &\quad + \frac{\mathbb{1}(\widehat{N}_{pos}(v) > \widehat{N}_{neg}(v))\mathbb{1}(v_i < 0)\Phi^{-1}(1 - p_{val})}{\widehat{N}_{neg}(v)}, \end{aligned} \quad (16)$$

which equal their original formulation in the cases where $N_{pos} \neq 0$ and $N_{neg} \neq 0$.

The second special case occurs if $N_{neg} = N_{pos}$; in this case, we note that the features as defined in (15) and (16) are always 0. This case can happen if the vector has a large number of 0's and a small number of non-zero entries, thus allowing the test statistic to be significant, while $N_{pos} = N_{neg}$. To address this, we introduce

$$\text{SignStatEqual}_i^{(1)} = \mathbb{1}(\widehat{N}_{pos}(v) = \widehat{N}_{neg}(v))(\mathbb{1}(v_i > 0) + \mathbb{1}(v_i < 0))\Phi^{-1}(1 - p_{val}) \quad (17)$$

$$\text{SignStatEqual}_i^{(2)} = \frac{\mathbb{1}(\widehat{N}_{pos}(v) = \widehat{N}_{neg}(v))(\mathbb{1}(v_i > 0) + \mathbb{1}(v_i < 0))\Phi^{-1}(1 - p_{val})}{\widehat{N}_{pos} + \widehat{N}_{neg}}. \quad (18)$$

There is one additional extremely rare edge case for this statistic. If many elements of the null distribution have the same sign, then any observed vectors with more than 50% of their entries with one sign but the remaining entries 0 can appear significant. To address this issue and to avoid adding noise to the feature set, we exclude these vectors from the features, and we simply create a new set of features for the cases where the entries are large. Finally, as the extremely rare case does not appear in our data sets, we exclude the related features from our main set of 140 features; we list them in the additional feature section in Sec. D.5.

A.2.4 Configuration Null Model Replicates which have Insufficient Size

Sometimes, a null model replica is too small to have sufficient eigenvectors to run a comparison to the real data. The underlying reason for this is that in the stub-based configuration model described in Appendix A.1, the removal of degree 0 nodes can affect the number of eigenvectors. When the networks are large, the impact of these missing nodes is small. However, an issue arises in Monte Carlo tests when some simulated networks are so small that the size of the network drops below the number of eigenvectors required for the null comparison.

To overcome this issue, as described in the main body, we re-sample any networks that is too small until a network is generated that has sufficient size; we stop at the 10th generated network. As long as the 10th generated network has at least 2 nodes, we use that network; otherwise, we re-sample again, repeating the loop for as long as necessary. As our method requires fewer than 25 eigenvectors, and the communities uncovered in the generating model are often much larger than this, the resampling only required in a small percentage of communities.

To assess statistical significance in our Monte Carlo tests when the null replicas have insufficient size, we compare the i^{th} real eigenvector only with networks that have at least i eigenvectors, thus giving a well defined comparison.

A.3 NetEMD Implementation Details

NetEMD standardises empirical distributions and then computes the minimum EMD over translations. Distributions in which all the mass is concentrated on a single point do not undergo variance scaling, as the variance is 0. However, when using floating point features, such as eigenvectors, variance scaling will inflate the small floating point difference. To address this issue, if the range of a statistic of the network is less than 10^{-10} , we replace the statistic with a point distribution.

A.4 PathFinder

Here we discuss how we construct our original paths of size 3 from which to expand.

Selecting the Starting Paths of Size 3 To avoid considering all paths of size 3, which can be very large, we start with a subset of paths of size 3, and then we expand on the *BeamWidth* paths with the highest fitness, where fitness is smallest edge weight in a path. The procedure to construct our paths of size 3 is as follows.

1. Make a List (*PathsToConsider*) with *BeamWidth* dummy paths of weight 0:
2. For each node (*CurrentNode*) in the network:
 - (a) Consider the node (*CurrentOut*) with highest-weighted outgoing edge to *CurrentNode*.
 - (b) Consider the node (*CurrentIn*) with highest-weighted incoming edge from *CurrentNode*.
 - (c) If the minimum fitness of the paths in *PathsToConsider* is less than the fitness of the path *CurrentIn* -> *CurrentNode* -> *CurrentOut*.
 - i. Add path to *PathsToConsider*, removing the path with the lowest fitness.
 - ii. If there are no more incoming and outgoing edges to consider, go to **2** and consider the next *CurrentNode*.
 - iii. If the largest unconsidered incoming edge to *CurrentNode* has a higher weight than the largest unconsidered outgoing edge from *CurrentNode*, then update *CurrentIn* to the relevant node and go to (c).
 - iv. Update *CurrentOut* to the node corresponding to the largest unconsidered outgoing edge and go to (c).
 - (d) Go to **2** and consider the next *CurrentNode*

The order in which the nodes are considered does play a role in the selection of the paths of size 3, as the if statement in (c) will only add paths if they have higher fitness than those previously observed.

Underlying Data structure, and Breaking Ties Our underlying implementation of the method uses a fixed size heap (using the Python module `heapq`) to which we are adding and removing elements. The computational complexity of adding an element to a heap and removing the least fit element is $O(\log(\text{number of elements in the heap}))$. We set the size of the heap to the *BeamWidth*. Adding the ρ path extensions has complexity $O(\rho \log(\text{number of elements in the heap/ beam width}))$, with $O(\text{number of elements in the heap/ beam width})$ storage.

As we are using the inbuilt `heapq` module, we break ties in fitness using the inherent Python 2 order of lists, which compares the first element, and then the second and so on. The lists in question are the lists of node indices involved in a path arranged in the order they are visited, in which larger indexes are considered fitter.

B Regions of Detectability

In the main body of the paper, we discussed limiting the parameter regimes to those where the expected number of anomalous networks that would occur at random is less than 1. In this section of the appendix, we derive the expression used in the paper (Sec. B.1) and we show that the case of path 5 is the most restrictive case (Sec. B.2).

B.1 Derivation of Regions of Detectability

Anomalies will only be detectable if they are unusual in some way, in the underlying network. In this paper, we require that the expected counts of the anomaly in a network without planted anomalies is less than 1. The parameter region which ensures that the expected counts are less than 1 is what we call the *region of detectability* for that anomaly. To compute the bounds on the expected number of anomalies, let $f^{(A(k,w))}(G, X)$ be the number of anomalies of type A of size k and minimum weight w , observed on the subnetwork of G induced by the nodes in X . Let the total number of such anomalies over the network be $F^{(A(k,w))}(G) = \sum_{\substack{X \subseteq V \\ |X|=k}} f^{(A(k,w))}(G, X)$. We assume that the function f preserves the exchangeability in the sense that the second equality in (19) holds

$$E[F^{(A(k,w))}(G)] = \sum_{\substack{X \subseteq V \\ |X|=k}} E[f^{(A(k,w))}(G, X)] = \binom{n}{k} E[f^{(A(k,w))}(G, X_{arb})], \quad (19)$$

where the final equality comes from the fact that all nodes in an ER network are exchangeable, and X_{arb} is an arbitrary set of nodes of size k . For example, subnetwork counts preserve the exchangeability.

Here, we calculate the expectations under a directed ER model with edge probability p and edge weight $W_{ij} \sim \text{Uniform}(0, 1)$. An edge is *heavy* if its edge weight W_{ij} is at least w , where w is a fixed number. Thus the probability of seeing a heavy edge is $p(1 - w)$. The structures for which we find the expected counts are

1. a clique on k nodes in which there is at least one heavy edge between any two nodes;
2. a star on k nodes in which the centre has in-degree at least k_1 and out-degree at least $k - k_1 - 1$, with the additional requirement that the k_1 in-degree edges come from different nodes than the nodes that the $k - k_1 - 1$ out-degree edges connect to;
3. a path on k nodes $v_1, \dots, v_k$ starting in v_1 , with a directed heavy edge from v_i to v_{i+1} , for $i = 1, \dots, k-1$;
4. a ring of k nodes - a directed completion of a heavy path;
5. a particular tree with 1 root (the inner shell) connecting to 3 nodes (the middle shell) connecting to 5 nodes each (the outmost shell), so that there are 9 nodes in the tree and $5 \times 3 + 3 \times 1 = 18$ edges in total, with the specified connecting edges again being heavy.

Note that, in all cases, we do not condition on the edges which are not specified by the anomaly. The calculations are as follows.

1. The derivation for the clique is as follows. The probability of observing a clique of size k on a fixed set of k nodes in an undirected ER network is given by $p^{\binom{k}{2}}$. Further, the probability of observing an edge in either direction in a directed ER network is given by $1 - (1 - p)^2$ (one minus the probability of there being no edge in either direction). Observing that the probability of a heavy edge in our network model is given by $p(1 - w)$, we obtain the following inequality

$$\begin{aligned} 1 &> E[F^{(A^{(clique)}(k,w))}(G)] \\ &= \binom{n}{k} E[f^{(A^{(clique)}(k,w))}(G, X_{arb})] \\ &= \binom{n}{k} (1 - (1 - (1 - w)p)^2)^{\binom{k}{2}} \end{aligned}$$

which is satisfied when

$$(1 - w)p < 1 - \left(1 - \left(\binom{n}{k}^{-1}\right)^{\frac{1}{\binom{k}{2}}}\right)^{\frac{1}{2}}. \quad (20)$$

2. The derivation for the star is as follows. We calculate the expected counts for stars on k nodes. To make the computation easier, we parameterise the number of nodes that have incoming heavy edges into the star as k_1 , where $k_1 < k$. For a star with k nodes, the probability that $k - 1$ heavy edges occur is $((1 - w)p)^{k-1}$. Starting with a set of k nodes, there are $\binom{k}{k_1}$ ways of selecting k_1 nodes with edges into the centre node. There are then $k - k_1$ ways of choosing the central node, which also fixes the number of nodes with edges from the centre node. Putting this all together in an expression for $f^{(A^{(Star)}(k,w))}(G, X_{arb})$, we arrive at

$$\begin{aligned} E[f^{(A^{(Star)}(k,w,k_1))}(G, X_{arb})] &= \sum_{\substack{X_1, X_2, X_3 \subset X_{arb} \\ X_1 \cup X_2 \cup X_3 = X_{arb} \\ |X_1 \cap X_2| = |X_2 \cap X_3| = |X_1 \cap X_3| = 0 \\ |X_1| = k_1, |X_2| = 1,}} ((1 - w)p)^{k-1} \\ &= \binom{k}{k_1} \binom{k - k_1}{1} ((1 - w)p)^{k-1}. \end{aligned}$$

Substituting this into equation (19) and rearranging we obtain

$$1 > E[F^{(A^{(Star)}(k,w,k_1))}(G)] = \binom{n}{k} \binom{k}{k_1} \binom{k - k_1}{1} ((1 - w)p)^{k-1}$$

giving that

$$(1 - w)p < \left(\frac{1}{\binom{n}{k} \binom{k}{k_1} \binom{k - k_1}{1}} \right)^{\frac{1}{k-1}}. \quad (21)$$

3. The derivation for the heavy path is similar, as it also has $k - 1$ heavy edges. In this case, there are k unique positions and therefore $k!$ different ways to assign the nodes in X_{arb} to the positions. Substituting this into equation (19) and rearranging leads us to

$$\begin{aligned} 1 &> E[F^{(A^{(Path)}(k,w))}(G)] \\ &= \binom{n}{k} (k)! ((1 - w)p)^{k-1} \end{aligned}$$

which is satisfied if

$$(1 - w)p < \left(\frac{1}{\binom{n}{k} (k)!} \right)^{\frac{1}{k-1}},$$

giving the expression (8) in the main text.

4. In the case of the ring, due to its rotational symmetry we fix the position of one node and then note that there are $(k - 1)!$ ways of placing the remaining nodes. Substituting this into equation (19) and rearranging we obtain

$$\begin{aligned} 1 &> E[F^{(A^{(Ring)}(k,w))}(G)] \\ &= \binom{n}{k} (k - 1)! ((1 - w)p)^k \end{aligned}$$

giving that

$$(1 - w)p < \left(\frac{1}{\binom{n}{k} (k - 1)!} \right)^{\frac{1}{k}}. \quad (22)$$

5. The final structure is the tree with 5 nodes in the outmost shell, 3 nodes in the middle shell and 1 node in the inner shell, so that there are $5 \times 3 + 3 \times 1 = 18$ edges. Fixing k as 9, as this structure is only defined on 9 nodes, there are $\binom{9}{5}$ ways of selecting the first shell nodes, and $\binom{4}{1}$ ways of choosing the third shell. Substituting this into equation (19) and rearranging we obtain

$$\begin{aligned} 1 &> E[F^{(A^{(Tree)}(k,w))}(G)] \\ &= \binom{n}{9} \binom{9}{5} \binom{4}{1} ((1-w)p)^{18} \end{aligned}$$

which amounts to

$$(1-w)p < \left(\frac{1}{4 \binom{n}{9} \binom{9}{5}} \right)^{\frac{1}{18}}. \quad (23)$$

B.2 Proof that Path 5 is the most restrictive case for $k \geq 5$

To validate our use of Path 5 as the most restrictive structure among all structures on at least $k = 5$ nodes (not taking the special case of the tree into account) we compare the respective expectations. First we show that a path of size 5 is more restrictive than any other path size, and then we demonstrate that a path of size k is more restrictive than another structure of size k .

Let us first consider the path of size k and a path of size $k+1$ where $k \geq 5$. We first note that

$$\left(\frac{1}{\binom{n}{k}(k)!} \right)^{\frac{1}{k-1}} = \left(\prod_{i=0}^{k-1} \frac{1}{n-i} \right)^{\frac{1}{k-1}}.$$

Now let us consider the ratio between the bound for a path of size k and the bound for a path of size $k+1$. We first note that for $\alpha < 0$ and $\beta < k < n$, then $(n-\beta)^\alpha < (n-k)^\alpha$, then

$$\begin{aligned} \frac{\left(\prod_{i=0}^{k-1} \frac{1}{n-i} \right)^{\frac{1}{k-1}}}{\left(\prod_{i=0}^k \frac{1}{n-i} \right)^{\frac{1}{k}}} &= \frac{\left(\prod_{i=0}^{k-1} \frac{1}{n-i} \right)^{\frac{1}{k-1}}}{\left(\frac{1}{n-k} \right)^{\frac{1}{k}} \left(\prod_{i=0}^{k-1} \frac{1}{n-i} \right)^{\frac{1}{k}}} = (n-k)^{\frac{1}{k}} \prod_{i=0}^{k-1} (n-i)^{\frac{1}{k} - \frac{1}{k-1}} \\ &< (n-k)^{\frac{1}{k}} \prod_{i=0}^{k-1} (n-k)^{\frac{1}{k} - \frac{1}{k-1}} = (n-k)^{1 - \frac{k}{k-1} + \frac{1}{k}} = (n-k)^{\frac{-1}{k(k-1)}} < 1. \end{aligned}$$

We note that the first equality comes from observing that $\frac{1}{k} - \frac{1}{k-1} < 0$, and applying the inequality $(n-\beta)^\alpha < (n-k)^\alpha$. As the power of the penultimate statement is negative, the ratio is less than 1. Hence the sequence is monotonically increasing, and therefore path 5 is the most restrictive case of the path-based options.

Let us now consider the remaining structures. First consider the ring, and consider the ratio between a ring of size k and a path of size k :

$$\frac{\left(\frac{1}{\binom{n}{k}(k-1)!} \right)^{\frac{1}{k}}}{\left(\prod_{i=0}^{k-1} \frac{1}{n-i} \right)^{\frac{1}{k-1}}} = \frac{\left(k \prod_{i=0}^{k-1} \frac{1}{n-i} \right)^{\frac{1}{k}}}{\left(\prod_{i=0}^{k-1} \frac{1}{n-i} \right)^{\frac{1}{k-1}}} = k^{\frac{1}{k}} \prod_{i=0}^{k-1} (n-i)^{\frac{1}{k-1} - \frac{1}{k}} \geq 1.$$

For $k < n$, each term in the product is greater than or equal to 1, and therefore the product is also, implying that the ratio is greater than 1 and thus the bound for the ring is larger.

Next, let us consider an analogous calculation for the star graph

$$\frac{\left(\frac{1}{\binom{n}{k} \binom{k}{k_1} \binom{k-k_1}{1}} \right)^{\frac{1}{k-1}}}{\left(\frac{1}{\binom{n}{k}(k)!} \right)^{\frac{1}{k-1}}} = \left(\frac{k!}{\binom{k}{k_1}(k-k_1)} \right)^{\frac{1}{k-1}} = \left(\frac{k_1!(k-k_1)!}{k-k_1} \right)^{\frac{1}{k-1}} = (k_1!(k-k_1-1)!)^{\frac{1}{k-1}} \geq 1,$$

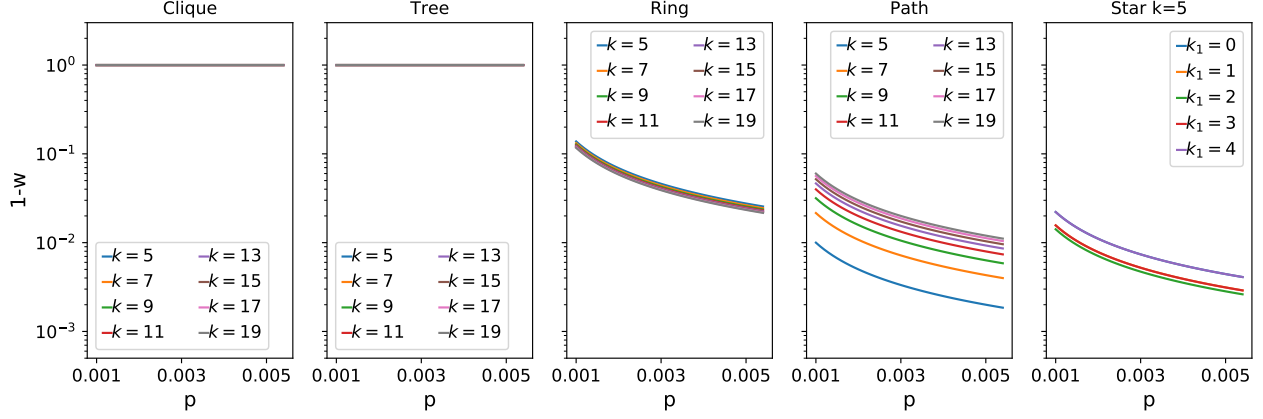


Figure 17: The bounds on the values of p and $1-w$ such that the expected number of anomalous structures are less than one. Each plot shows the bounds of a given structure (described in the title) of different sizes, with the exception of the final panel, which shows a star of size 5 with differing numbers of incoming and outgoing edges.

where the penultimate equality holds as we know that $k_1 < k$ - the case $k_1 = k$ is not possible because one node must be the centre node. Clearly, this ratio is greater than 1, therefore the path case is more restrictive.

For the clique, the bound is given by the following calculation

$$1 - \left(1 - \left(\binom{n}{k}^{-1} \right)^{\frac{1}{\binom{k}{2}}} \right)^{\frac{1}{2}} \geq 1 - \left(1 - \left(\binom{n}{k}^{-1} \right)^{\frac{1}{\binom{k}{2}}} \right) = \left(\binom{n}{k}^{-1} \right)^{\frac{1}{\binom{k}{2}}} \quad (24)$$

As $\binom{n}{k}^{-1} < 1$, the quantity in the main bracket is between 0 and 1 and thus as $x \geq x^2$ for $0 \leq x \leq 1$, the inequality holds. Therefore let us consider the ratio of the lower bound and the path bound on k nodes:

$$\frac{\left(\binom{n}{k}^{-1} \right)^{\frac{1}{\binom{k}{2}}}}{\left(\frac{1}{\binom{n}{k}(k)!} \right)^{\frac{1}{k-1}}} = \frac{\left(\binom{n}{k}^{-1} \right)^{\frac{1}{\frac{1}{2}k(k-1)}}}{\left(\frac{1}{\binom{n}{k}(k)!^{\frac{1}{2}k}} \right)^{\frac{1}{\frac{1}{2}k(k-1)}}} = \left(\frac{\left(\binom{n}{k}(k)! \right)^{\frac{k}{2}}}{\binom{n}{k}} \right)^{\frac{1}{\frac{1}{2}k(k-1)}} = \left(\binom{n}{k}^{\frac{k}{2}-1} \left((k)!^{\frac{k}{2}} \right)^{\frac{1}{\frac{1}{2}k(k-1)}} \right) \geq 1.$$

The final term is greater than 1 as by assumption $k \geq 5$, thus $\frac{k}{2} - 1$ is greater than 0, and thus all terms in the product are greater than 1. Thus the ratio is greater than one, and therefore the path case is more restrictive.

Finally, as the tree is of fixed size, we can simply compare the values directly. The bound for the tree is ≈ 0.0144154 , whereas the bound for a path of size 5 is around 10^{-5} , indicating that the path bound is more restrictive. Hence the smallest overall bound for the subnetwork probabilities under consideration is the bound for paths of size 5.

Fig. 17 shows the bound for different sized versions of each structure. In the case of the clique, tree, ring and path we plot structures of size $k = 5, 7, \dots, 19$, whereas for the star we fix $k = 5$ and vary the number of incoming edge $k_1 = 0, 2, \dots, 4$. We observe that the curve for the Path of size 5 is the most restrictive of all (4th-panel blue line). To confirm that there is not a star with a combination of k and k_1 which would outperform Path 5, we plot each combination in Fig. 18. The restrictions for $k = 5$ are stronger than those of all other sizes, thus illustrating the analytical result.

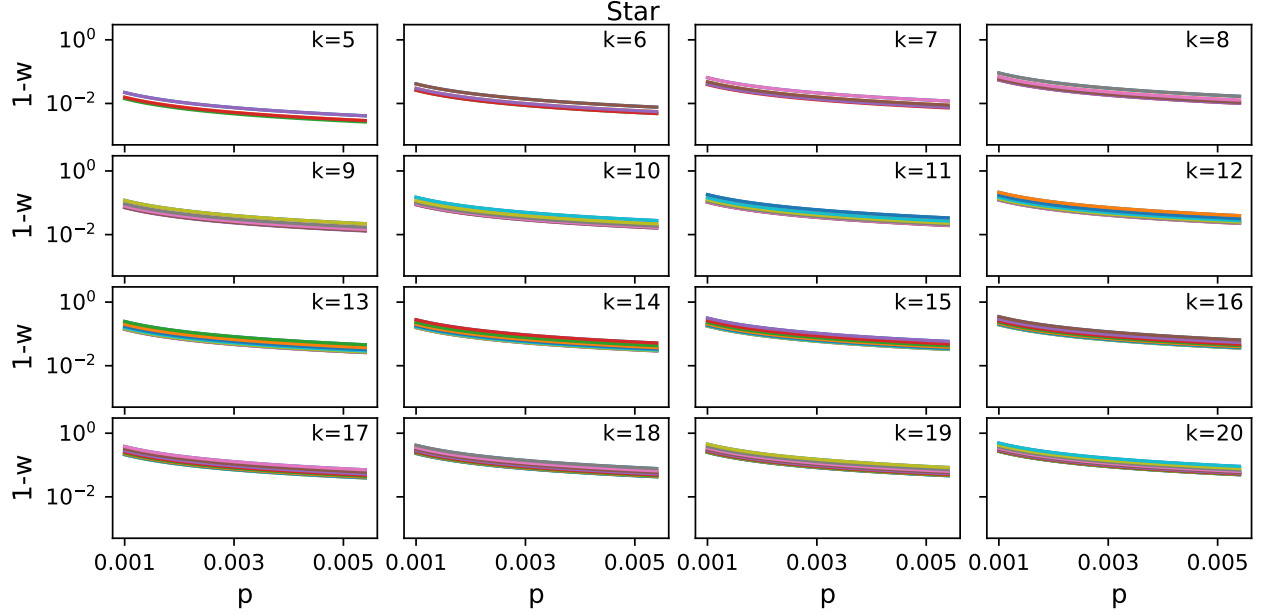


Figure 18: The bounds on the values of p and $1 - w$ such that the expected number of stars of a given size and a given number of incoming connections are less than one. The label in each plot shows the size of the structure in each panel; each line in the panel represents a different number of incoming edges.

B.3 Approximate Detectability of the Anomalies in the Accenture Model

We claimed in the paper that the planted structures (with the exception of the path of size 5) in a simplified version of the model constructed by Accenture are detectable. In this section, we provide justification for this assertion.

The Accenture model fixes the in- and out-degree sequence of the network using draws from different normal distributions (mean 21 and standard deviation 3 for the incoming edges, and mean 19 with standard deviation of 2 for the outgoing edges). To guarantee a feasible in- and out-degree sequence, the model reduces the in-degree or the out-degree of randomly selected nodes until the sum of the degree sequences match.

While analysing this procedure exactly is difficult, the number of connections is likely to be driven by the out-degree as it has a smaller mean. Then, the relevant bounds for a directed weighted ER network with $p = 19/55000$ are a reasonable approximation to the detectability of the planted anomalies under this model.

Applying Eq. (20), Eq. (22) and Eq. (8) to this case with $n = 55,000$, with cliques of sizes 8 and 12; rings of sizes 4 and 10, and heavy paths of sizes 5 and 10 we observe the following

Clique 8	$(1-w)p < 1 - \left(1 - \binom{55000}{8}^{-\frac{1}{8}}\right)^{\frac{1}{2}} \approx 0.0328$ $(1-w) \lesssim 95.0316$
Clique 12	$(1-w)p < 1 - \left(1 - \binom{55000}{12}^{-\frac{1}{12}}\right)^{\frac{1}{2}} \approx 0.0978$ $(1-w) \lesssim 2707.79$
Ring 4	$(1-w)p < \left(\binom{55000}{4}(4-1)!\right)^{-\frac{1}{4}} \approx 2.57^{-5}$ $(1-w) \lesssim 0.0744343$
Ring 10	$(1-w)p < \left(\binom{55000}{10}(10-1)!\right)^{-\frac{1}{10}} \approx 2.29^{-5}$ $(1-w) \lesssim 0.0662647$
Path 5	$(1-w)p < \left(\binom{55000}{5}(5)!\right)^{-\frac{1}{5-1}} \approx 1.19 \times 10^{-6}$ $(1-w) \lesssim 0.00118732$
Path 10	$(1-w)p < \left(\binom{55000}{10}(10)!\right)^{-\frac{1}{10-1}} \approx 5.41 \times 10^{-6}$ $(1-w) \lesssim 0.0156523.$

As $1-w$ is equal to the range of the c.d.f. covered by the weights, we can compare the bound directly on the range of the c.d.f. considered in the Accenture model. This range is from 0.99 to 0.99999, spanning $\approx 1\%$. Therefore, all planted anomalies other than the path of size 5 are detectable in the Accenture model, in the sense that the number expected at random is approximately less than 1.

C Proof that the Selection of the Direction in the Eigenvector Terminates

In NetEMD, the distribution of the entries of the eigenvectors are used as a feature in the comparison. However, as for an eigenvector v , both v and $-v$ are solutions to the eigenvector equation, and we need to select between them. To this end, we use the following procedure.

1. Compute the number of entries greater than 0 (*numGreater*).
2. Compute the number of entries less than 0 (*numLesser*).
3. If the vector is symmetric then use v (as in this case the choice of sign does not matter).
4. If $\text{numGreater} > \text{numLesser}$ then use v .
5. If $\text{numGreater} < \text{numLesser}$ then use $-v$.
6. If $\text{numGreater} = \text{numLesser}$ then:
 - (i) Set a to 0
 - (ii) If $\sum_j (v_j)^{2a+1} > 0$ use v
 - (iii) If $\sum_j (v_j)^{2a+1} < 0$ use $-v$

(iv) If $\sum_j (v_j)^{2a+1} = 0$ use increase a by 1 and go back to (ii)

As, potentially, such an algorithm may result in an infinite loop, we show that the procedure will always result in a selection between v and $-v$ in finite time. More precisely, we shall show that there always exists an exponent $a \in \mathbb{N} \cup \{0\}$ such that $\sum_j (v_j)^{2a+1} \neq 0$. For this proof, we borrow ideas from [61].

Let $w_1 \leq w_2 \leq \dots \leq w_n$ be the elements of the eigenvector v which are not equal to 0. Assume that the procedure has reached Step 6. Then the number of positive entries must be equal to the number of negative entries. Let the negative entries be q_i such that $q_1 \leq q_2 \leq q_3 \leq \dots \leq q_{\frac{n}{2}} < 0$ and r_i be the positive entries $0 < r_1 \leq r_2 \leq r_3 \leq \dots \leq r_{\frac{n}{2}}$.

As $\sum_j (v_j)^{2a+1}$ is a sum of odd powers (power sums), we can remove any pair of elements (q_i, r_j) such that $|q_i| = |r_j|$, as their odd powers cancel each other in the sum. We first consider (q_1, r_1) , and the pair remove if they are equal. We then consider each pair in a fixed ordering assuming that either of the elements have not already been removed. This leaves $\tilde{q}_1 \leq \tilde{q}_2 \leq \tilde{q}_3 \leq \dots \leq \tilde{q}_{\frac{n}{2}} \leq 0$ and $0 \leq \tilde{r}_1 \leq \tilde{r}_2 \leq \tilde{r}_3 \leq \dots \leq \tilde{r}_{\frac{n}{2}}$. By assumption, the sequence is not symmetric and therefore $\tilde{n} \neq 0$.

To show that the procedure terminates in finitely many steps, we show that for every set of values which is not symmetric around 0, there is an odd power such that the sum of the values to this odd power sums is not equal to 0. Clearly, if $\sum_i v_i^{2a+1} = \sum_i \tilde{q}_i^{2a+1} + \sum_i \tilde{r}_i^{2a+1} = 0$ then $\sum_i (bv_i)^{2a+1} = \sum_i (b\tilde{q}_i)^{2a+1} + \sum_i (b\tilde{r}_i)^{2a+1} = 0$.

Further, as all pairs of values that cancel each other out in the sum of odd powers have been removed, $|\tilde{q}_1| \neq |\tilde{r}_{\frac{n}{2}}|$. Without loss of generality, let $|\tilde{q}_1| \leq |\tilde{r}_{\frac{n}{2}}|$. Let $b = \frac{1}{|\tilde{r}_{\frac{n}{2}}|}$, let $\tilde{q}'_i = \frac{\tilde{q}_i}{b}$ and let $\tilde{r}'_i = \frac{\tilde{r}_i}{b}$. Then

$$0 = \sum_{i=1}^{\frac{n}{2}} (b\tilde{q}_i)^{2a+1} + \sum_{i=1}^{\frac{n}{2}} (b\tilde{r}_i)^{2a+1} = \sum_{i=1}^{\frac{n}{2}} (\tilde{q}'_i)^{2a+1} + \sum_{i=1}^{\frac{n}{2}-1} (\tilde{r}'_i)^{2a+1} + 1.$$

Using that the $r_i > 0$ and that $q_i < 0$, it follows that

$$\frac{\tilde{n}}{2} (\tilde{q}'_1)^{2a+1} + 1 \leq 0.$$

Re-arranging and taking logarithms,

$$\log(1) \leq \log\left(\frac{\tilde{n}}{2}\right) + (2a+1)\log(-(\tilde{q}'_1)).$$

Thus,

$$-\frac{\log(\frac{\tilde{n}}{2})}{\log(-(\tilde{q}'_1))} \geq (2a+1),$$

where the direction of the inequality changes as by assumption $-1 < \tilde{q}'_1 < 0$, and therefore $\log(-(\tilde{q}'_1)) < 0$.

As $-(\tilde{q}'_1) \neq 1$, the left-hand side of the last inequality is a finite positive quantity which does not depend on a . Therefore, there exists a positive integer a such that the inequality does not hold. Thus, there is a power sum which is non-zero, and hence the algorithm terminates after finitely many steps, which concludes the proof.

D List of All Anomaly Detection Features

D.1 Basic Node Statistics

All of the statistics below are node statistics in that they give a different value to each node - but some of them give the same value to each node in a community. Here is the list of basic node statistics.

1. Standardised Degree

2. Community Density Full, i.e. the density of the community divided by overall density
3. Community Density Average, i.e. the density of the community divided by overall density and also divided by the size of the community
4. Community GAW Full, i.e. the Geometric Average Weight of the community divided by overall Geometric Average Weight
5. Community GAW Average, i.e. the Geometric Average Weight of the community divided by overall Geometric Average Weight and also divided by the size of the community
6. Geometric Average Weight (GAW)
7. Geometric Average Weight Top 10% (GAW Top 10%)
8. Geometric Average Weight Top 20% (GAW Top 20%)
9. Community Density Configuration Model - $\text{Max}(\Phi(1 - p_{val}), 0)$, where Φ^{-1} is the inverse c.d.f. of a standard normal distribution and p_{val} is the p -value of the Monte Carlo test which compares the density of the community with respect to a null distribution of densities of communities of configuration model graphs
10. Small Community Category - As discussed in Sec. 3.2

D.2 Path Statistics

The statistics labelled **11-40** are obtained by using the PathFinder method. To maintain consistency with the embedded paths, we denote the size of a path is the number of nodes in the path. in general we include path statistics of size 3-32.

11. Path of size 3
12. Path of size 4
- ⋮
39. Path of size 31
40. Path of size 32

For the main experiments we only compute path of size up to 21 (which corresponds to 20 edges), with the remaining features left blank.

D.3 NetEMD Measures

The statistics labelled **41-72** are features related to 16 motifs with NetEMD statistics 1 and 2. The statistics **73-80** are features related to the upper adjacency matrix, the lower adjacency matrix, the combinatorial Laplacian and the random-walk Laplacian with NetEMD scores 1 and 2 from Eq. (6) and Eq. (7).

41. Motif 1 - NetEMD Score 1 (Eq. (6))
42. Motif 1 - NetEMD Score 2 (Eq. (7))
43. Motif 2 - NetEMD Score 1 (Eq. (6))
44. Motif 2 - NetEMD Score 2 (Eq. (7))
- ⋮

- 69. Motif 15 - NetEMD Score 1 (Eq. (6))
- 70. Motif 15 - NetEMD Score 2 (Eq. (7))
- 71. Motif 16 - NetEMD Score 1 (Eq. (6))
- 72. Motif 16 - NetEMD Score 2 (Eq. (7))
- 73. Adj. Upper Eigenvectors - NetEMD Score 1 (Eq. (6))
- 74. Adj. Upper Eigenvectors - NetEMD Score 2 (Eq. (7))
- 75. Adj. Lower Eigenvectors - NetEMD Score 1 (Eq. (6))
- 76. Adj. Lower Eigenvectors - NetEMD Score 2 (Eq. (7))
- 77. Combinatorial Laplacian Eigenvectors - NetEMD Score 1 (Eq. (6))
- 78. Combinatorial Laplacian Eigenvectors - NetEMD Score 2 (Eq. (7))
- 79. Random Walk Laplacian Eigenvectors - NetEMD Score 1 (Eq. (6))
- 80. Random Walk Laplacian Eigenvectors - NetEMD Score 2 (Eq. (7))

D.4 Localisation Statistics

The statistics labelled **81 - 140** are the localisation statistics on the upper adjacency matrix, the lower adjacency matrix, the combinatorial Laplacian and the random-walk Laplacian with the following localisation measures:

- 81. Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 1 (Eq. (11))
- 82. Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 2 (Eq. (12))
- 83. Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 3 (Eq. (13))
- 84. Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 4 (Eq. (14))
- 85. Adj. Upper Eigenvectors - Exponential - Norm Node Statistic 1 (Eq. (11))
- 86. Adj. Upper Eigenvectors - Exponential - Norm Node Statistic 2 (Eq. (12))
- 87. Adj. Upper Eigenvectors - Exponential - Norm Node Statistic 3 (Eq. (13))
- 88. Adj. Upper Eigenvectors - Exponential - Norm Node Statistic 4 (Eq. (14))
- 89. Adj. Upper Eigenvectors - 90 percent Contribution in Inverse Participation Ratio
- 90. Adj. Upper Eigenvectors - 90 percent Contribution in absolute value
- 91. Adj. Upper Eigenvectors - Sign Statistic 1 (Eq. (15))
- 92. Adj. Upper Eigenvectors - Sign Statistic 2 (Eq. (16))
- 93. Adj. Upper Eigenvectors - Sign Statistic Equal 1 (Eq. (17))
- 94. Adj. Upper Eigenvectors - Sign Statistic Equal 2 (Eq. (18))
- 95. Adj. Upper Eigenvectors - Absolute value of the eigenvector
- 96. Adj. Lower Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 1 (Eq. (11))

97. Adj. Lower Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 2 (Eq. (12))
98. Adj. Lower Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 3 (Eq. (13))
99. Adj. Lower Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 4 (Eq. (14))
100. Adj. Lower Eigenvectors - Exponential - Norm Node Statistic 1 (Eq. (11))
101. Adj. Lower Eigenvectors - Exponential - Norm Node Statistic 2 (Eq. (12))
102. Adj. Lower Eigenvectors - Exponential - Norm Node Statistic 3 (Eq. (13))
103. Adj. Lower Eigenvectors - Exponential - Norm Node Statistic 4 (Eq. (14))
104. Adj. Lower Eigenvectors - 90 percent Contribution in Inverse Participation Ratio
105. Adj. Lower Eigenvectors - 90 percent Contribution in absolute value
106. Adj. Lower Eigenvectors - Sign Statistic 1 (Eq. (15))
107. Adj. Lower Eigenvectors - Sign Statistic 2 (Eq. (16))
108. Adj. Lower Eigenvectors - Sign Statistic Equal 1 (Eq. (17))
109. Adj. Lower Eigenvectors - Sign Statistic Equal 2 (Eq. (18))
110. Adj. Lower Eigenvectors - Absolute value of the eigenvector
111. Combinatorial Laplacian - Inverse Participation Ratio - Norm Node Statistic 1 (Eq. (11))
112. Combinatorial Laplacian - Inverse Participation Ratio - Norm Node Statistic 2 (Eq. (12))
113. Combinatorial Laplacian - Inverse Participation Ratio - Norm Node Statistic 3 (Eq. (13))
114. Combinatorial Laplacian - Inverse Participation Ratio - Norm Node Statistic 4 (Eq. (14))
115. Combinatorial Laplacian - Exponential - Norm Node Statistic 1 (Eq. (11))
116. Combinatorial Laplacian - Exponential - Norm Node Statistic 2 (Eq. (12))
117. Combinatorial Laplacian - Exponential - Norm Node Statistic 3 (Eq. (13))
118. Combinatorial Laplacian - Exponential - Norm Node Statistic 4 (Eq. (14))
119. Combinatorial Laplacian - 90 percent Contribution in Inverse Participation Ratio
120. Combinatorial Laplacian - 90 percent Contribution in absolute value
121. Combinatorial Laplacian - Sign Statistic 1 (Eq. (15))
122. Combinatorial Laplacian - Sign Statistic 2 (Eq. (16))
123. Combinatorial Laplacian - Sign Statistic Equal 1 (Eq. (17))
124. Combinatorial Laplacian - Sign Statistic Equal 2 (Eq. (18))
125. Combinatorial Laplacian - Absolute value of the eigenvector
126. Random Walk Laplacian - Inverse Participation Ratio - Norm Node Statistic 1 (Eq. (11))
127. Random Walk Laplacian - Inverse Participation Ratio - Norm Node Statistic 2 (Eq. (12))
128. Random Walk Laplacian - Inverse Participation Ratio - Norm Node Statistic 3 (Eq. (13))

129. Random Walk Laplacian - Inverse Participation Ratio - Norm Node Statistic 4 (Eq. (14))
130. Random Walk Laplacian - Exponential - Norm Node Statistic 1 (Eq. (11))
131. Random Walk Laplacian - Exponential - Norm Node Statistic 2 (Eq. (12))
132. Random Walk Laplacian - Exponential - Norm Node Statistic 3 (Eq. (13))
133. Random Walk Laplacian - Exponential - Norm Node Statistic 4 (Eq. (14))
134. Random Walk Laplacian - 90 percent Contribution in Inverse Participation Ratio
135. Random Walk Laplacian - 90 percent Contribution in absolute value
136. Random Walk Laplacian - Sign Statistic 1 (Eq. (15))
137. Random Walk Laplacian - Sign Statistic 2 (Eq. (16))
138. Random Walk Laplacian - Sign Statistic Equal 1 (Eq. (17))
139. Random Walk Laplacian - Sign Statistic Equal 2 (Eq. (18))
140. Random Walk Laplacian - Absolute value of the eigenvector

D.5 Additional Features

Finally, we include a small number of additional features which are employed primarily to highlight special cases. However, these features were not added to the set of features considered in the main body of the text, as these special cases did not appear in the data analysis.

For brevity, as this does not appear in the list of our 140 statistics, we do not list each combination of our statistic and matrix. We simply list the set of statistics.

Localisation We flag a special cases by creating the following additional features:

1. Sign based statistic - Statistic 1 Large Number (Eq. (15))
2. Sign based statistic - Statistic 2 Large Number (Eq. (16))
3. Sign based statistic - Statistic Equal 1 Large Number (Eq. (17))
4. Sign based statistic - Statistic Equal 2 Large Number (Eq. (18))

This series of features flags cases where the minimum of the number of positive and negative entries becomes greater than or equal to $\frac{n}{2}$. As this statistic captures the smaller of the number of positive entries and the number of negative entries, it seems surprising that we can get entries that are greater than $\frac{n}{2}$ and are still significant. A statistic that is greater than or equal to $\frac{n}{2}$ is possible if the remaining entries are zero, and thus if a sufficient number of the null replicas are in the same position. However, as this is outside of the original motivation of the feature, we split it out into a set of separate features. We perform a similar action for the case where the number of positive and negative terms are equal, which is only possible if there are exactly $\frac{n}{2}$ positive entries, and $\frac{n}{2}$ negative entries.

E Feature Selection

E.1 List of Selected Features

1. Geometric Average Weight Top 10% (GAW Top 10%)
2. Localisation - Adjacency Upper Eigenvectors - Sign based statistic - Average score
3. NetEMD - Adj. Lower Eigenvectors - NetEMD Score 1 (Eq. (6))
4. NetEMD - Adj. Upper Eigenvectors - NetEMD Score 1 (Eq. (6))
5. Localisation - Adj. Upper Eigenvectors - Exponential - Norm Node Statistic 1 (Eq. (11))
6. Localisation - Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 1 (Eq. (11))
7. Localisation - Adj. Lower Eigenvectors - Exponential - Norm Node Statistic 1 (Eq. (11))
8. Localisation - Adj. Upper Eigenvectors - 90 percent Contribution in absolute value
9. Localisation - Adj. Lower Eigenvectors - 90 percent Contribution in absolute value
10. Localisation - Random Walk Laplacian - 90 percent Contribution in absolute value
11. Basic Statistics - Community GAW Full i.e the Geometric Average Weight of the community divided by overall Geometric Average Weight.
12. NetEMD - Adj. Lower Eigenvectors - NetEMD Score 2 (Eq. (7))
13. Basic Statistics - Standardised Degree
14. Localisation - Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 1 (Eq. (11))
15. Localisation - Adj. Upper Eigenvectors - 90 percent Contribution in Inverse Participation Ratio
16. NetEMD - Random Walk Laplacian - NetEMD Score 1 (Eq. (6))
17. Basic Statistics - Community GAW Average i.e the Geometric Average Weight of the community divided by overall Geometric Average Weight and also divided by the size of the community.
18. Community Density Full i.e the density of the community divided by overall density.
19. Localisation - Random Walk Laplacian - Exponential - Norm Node Statistic 1 (Eq. (11))
20. Path Methods - Path of Size 6
21. Path Methods - Path of Size 5
22. Localisation - Adj. Upper Eigenvectors - Sign Statistic 1 (Eq. (15))
23. Localisation - Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 2 (Eq. (12))
24. Basic Statistics - Community Density Average i.e the density of the community divided by overall density and also divided by the size of the community.
25. NetEMD - Motif 5 - NetEMD Score 2 (Eq. (7))
26. Localisation - Random Walk Laplacian - Sign Statistic 2 (Eq. (16))
27. Localisation - Adj. Upper Eigenvectors - Inverse Participation Ratio - Norm Node Statistic 4 (Eq. (14))
28. NetEMD - Adj. Upper Eigenvectors - NetEMD Score 2 (Eq. (7))

29. NetEMD - Motif 5 - NetEMD Score 1 (Eq. (6))
30. Localisation - Random Walk Laplacian - Sign Statistic 1 (Eq. (15))
31. Localisation - Adj. Upper Eigenvectors - Exponential - Norm Node Statistic 2 (Eq. (12))
32. Localisation - Adj. Upper Eigenvectors - Exponential - Norm Node Statistic 4 (Eq. (14))
33. Localisation - Random Walk Laplacian - Inverse Participation Ratio - Norm Node Statistic 1 (Eq. (11))
34. Basic Statistic - Geometric Average Weight Top 20% (GAW Top 20%)
35. Localisation - Combinatorial Laplacian - 90 percent Contribution in absolute value
36. Localisation - Adj. Lower Eigenvectors - Sign Statistic 2 (Eq. (16))
37. Localisation - Adj. Lower Eigenvectors - Sign Statistic 1 (Eq. (15))
38. Basic Statistics - Community Density Configuration Model
39. NetEMD - Random Walk Laplacian Eigenvectors - NetEMD Score 2 (Eq. (7))
40. NetEMD - Motif 4 - NetEMD Score 2 (Eq. (7))
41. Localisation - Adj. Lower Eigenvectors - 90 percent Contribution in Inverse Participation Ratio
42. Localisation - Combinatorial Laplacian - Exponential - Norm Node Statistic 1 (Eq. (11))
43. Basic Statistics - Geometric Average Weight (GAW)
44. NetEMD - Motif 6 - NetEMD Score 2 (Eq. (7))

E.2 Alternative Feature Selection

To select the feature set for our random forest, we used the average rank over the 27 parameter regimes. Another approach would have been to use the average feature importance score.

The resultant feature plot, which is a direct comparison to the ranking version from Fig. 8A, can be seen in Fig. E.2. This approach would yield substantially fewer features. We decided to use the feature rank approach, because the feature importance scores could be confounded by correlated features; a set of features with high correlation may be seen as less important than a set of features that are not correlated, see for example [4] or [46] for details.

E.3 Full Feature Selection Plots

In the main body, we showed the feature importance detailing the top three features for each parameter regime. In a similar manner to Sec. E.2, we also show the relative feature importance for each of these features. These can be seen in Fig. E.3. The weight on the first few features varies considerably across parameter regimes. Notably, when $p = 0.001$ and $1 - w = 0$ or $1 - w = 0.001$, over 50% of the feature importance is given to only 1 feature. At the other extreme, for example in the region $p = 0.003$ and $1 - w = 0.003$, the feature importance is much less concentrated, with the first three features having feature importances in broadly similar ranges.

F Within-Sample Scores for the RANDOM FOREST Method

To test the performance of the random forest model, we only reported the out-of-sample performance in the main body of the text. For completeness, in this section we report the in-sample performance of the random forest, which can be seen in Fig. 21. We observe almost perfect performance across all of the regimes. This behaviour is not unexpected for a random forest, but it confirms that in the sample there is enough variation in the features to correctly identify all of the anomalies.

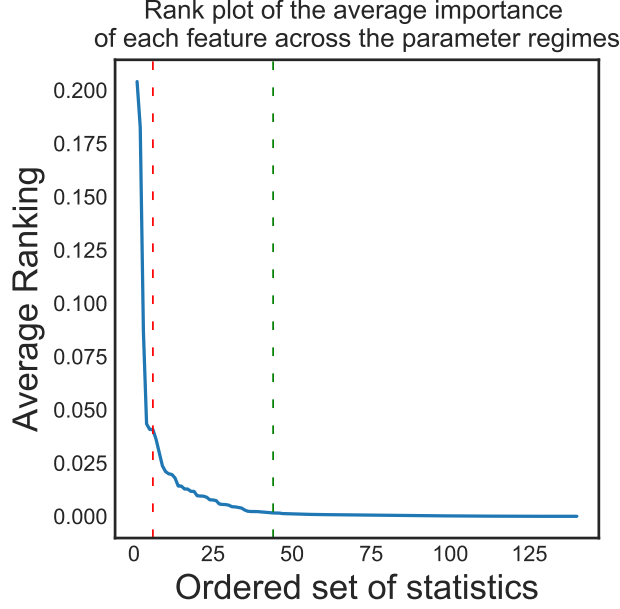


Figure 19: Feature selection using the average score over the 27 parameter regimes. The red dashed line represents a possible feature cut-off point. The green line represents the cut-off point which we selected when using the average rank, as can be seen in Fig. 8A

G Oddball

In this section, we present more details on Oddball, with Sec. G.1 addressing the adaptation of Oddball for directed networks, Sec. G.2 addressing the performance of each of the Oddball statistics on the Weighted ER Model, and Sec. G.3 addressing the performance of each of the Oddball statistics on the Accenture model.

G.1 Oddball for Directed networks

The original formulation of Oddball in [2] was primarily concerned with weighted undirected networks, although stating that it is easily generalisable to directed networks. The software which accompanies the paper provides the generalisation. While in the undirected case, many of the statistics are based on the ego-networks of individual nodes, in the directed generalisation, the ego-networks (egonets) are computed on a symmetrised matrix and thus consider both incoming and outgoing edges. We use this extension in the analysis in the main paper and accompanying appendices.

As the package was developed on an older version of Matlab, we made the following adjustments to run the package on an updated Matlab version. First, we removed plotting commands, some of which were creating errors on our networks, as the plots were not required for our analysis. Second, the directed version of Oddball uses the number of degree 1 neighbours as a statistic, which we removed as several of our networks do not possess degree 1 nodes (the expected number is given by $n(n-1)p(1-p)^{n-2}$ which is 4.52 when $p = 0.001$, but 0.0004 when $p = 0.002$). As this method raised an error when nodes did not exist, we removed the component.

For completeness, we include a list of the comparisons used in the directed version of Oddball.

1. Number of nodes vs number of edges
2. Number of edges vs the total weight in the egonet
3. Egonet out-degree vs egonet out-weight (edges from the ego network from to the remaining network)

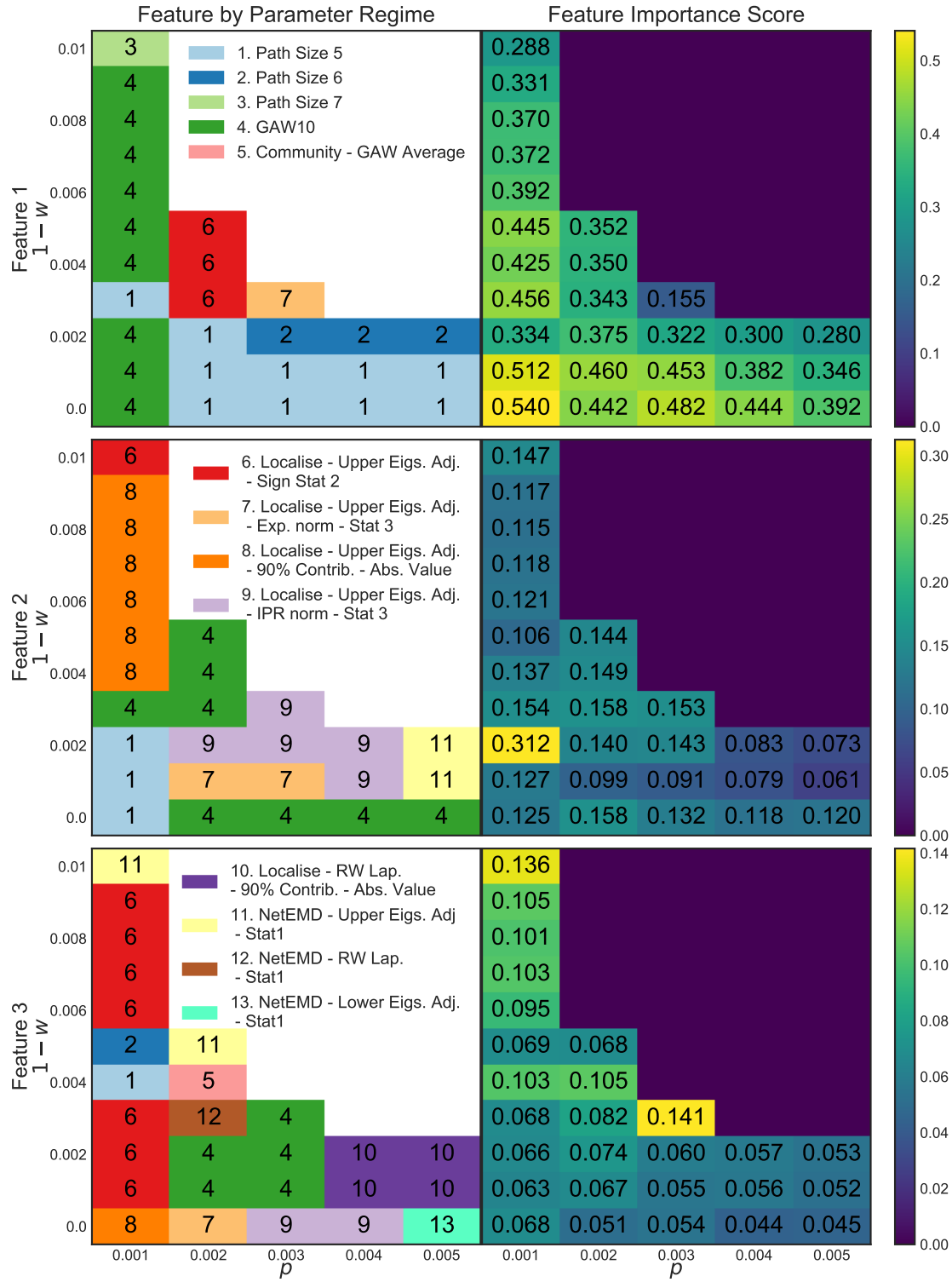


Figure 20: Feature importance plots, as seen in the main body Fig. 9, combined with the corresponding score displayed on the right. To aid understanding, we also include the index of the feature in the left set of plots, and the raw value of the statistic in the right set of plots.

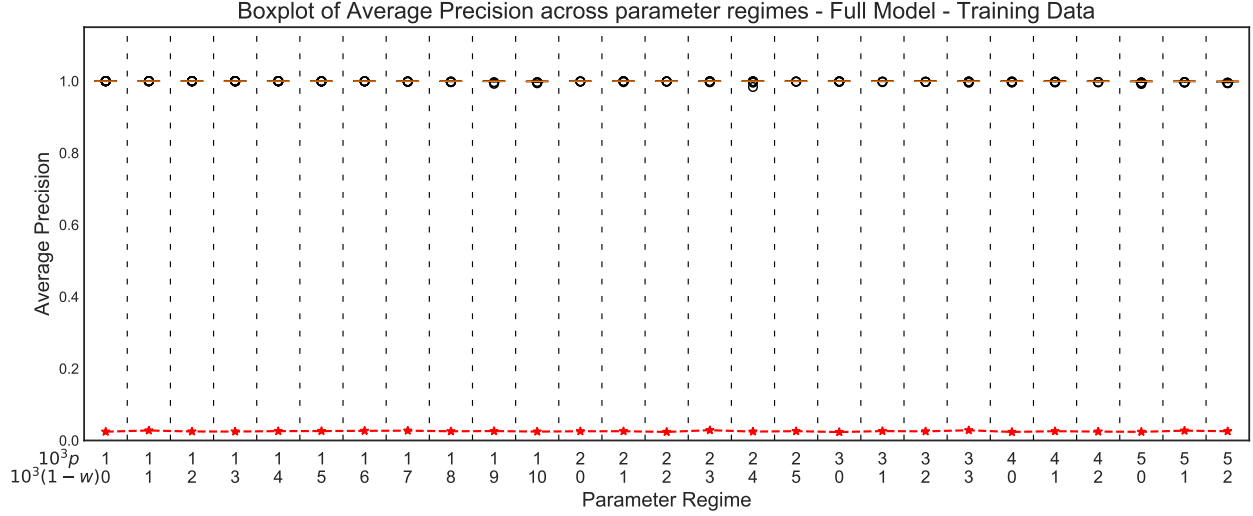


Figure 21: The average precision for the random forest, for each of the parameter regimes using the full model after feature selection and training on the training data (in-sample performance). The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

4. Egonet in-degree vs egonet in-weight (edges to the ego network from the remaining network)
5. Ego out-degree vs ego out-weight
6. Ego in-degree vs ego in-Weight
7. Egonet total weight vs egonet maximum weight
8. Egonet total in-weight vs egonet maximum in-weight
9. Egonet total out-weight vs egonet maximum out-weight.

Here, “ego network” refers to properties of the ego, while “ego net” refers to properties of the 1 hop snowball sample. As above, the software also provides one additional comparison, namely the number of nodes vs number of degree-1 nodes, which we do not use as our networks contain very few such nodes.

G.2 Oddball Individual Statistics - Weighted ER Network

For the Weighted ER network we only reported the performance of a statistic based on the summation of each of the underlying Oddball statistics. In this section, we present the results for each of the statistics individually, in Figs. 22, 23, 24, 25, 26, 27, 28, 29, and 30.

None of the statistics outperform either the feature summation or the random forest approach. A visual inspection indicates that the top four best-performing Oddball statistics are ego out-degree vs ego out-weight, ego in-degree vs ego in-weight, number of nodes vs number of edges, and number of edges vs weight.

G.3 Oddball Individual Statistics - Accenture Model

For completeness, this section shows the recall, precision and average precision plots for each of the underlying Oddball statistics over the 100 Accenture networks. Note that, for some of the networks, some of the underlying Oddball statistics are badly defined, in the sense that they contain Infs. The results can be found in Figs. 31, 32, 33, 34, 35, 36, 37, 38, and 39.

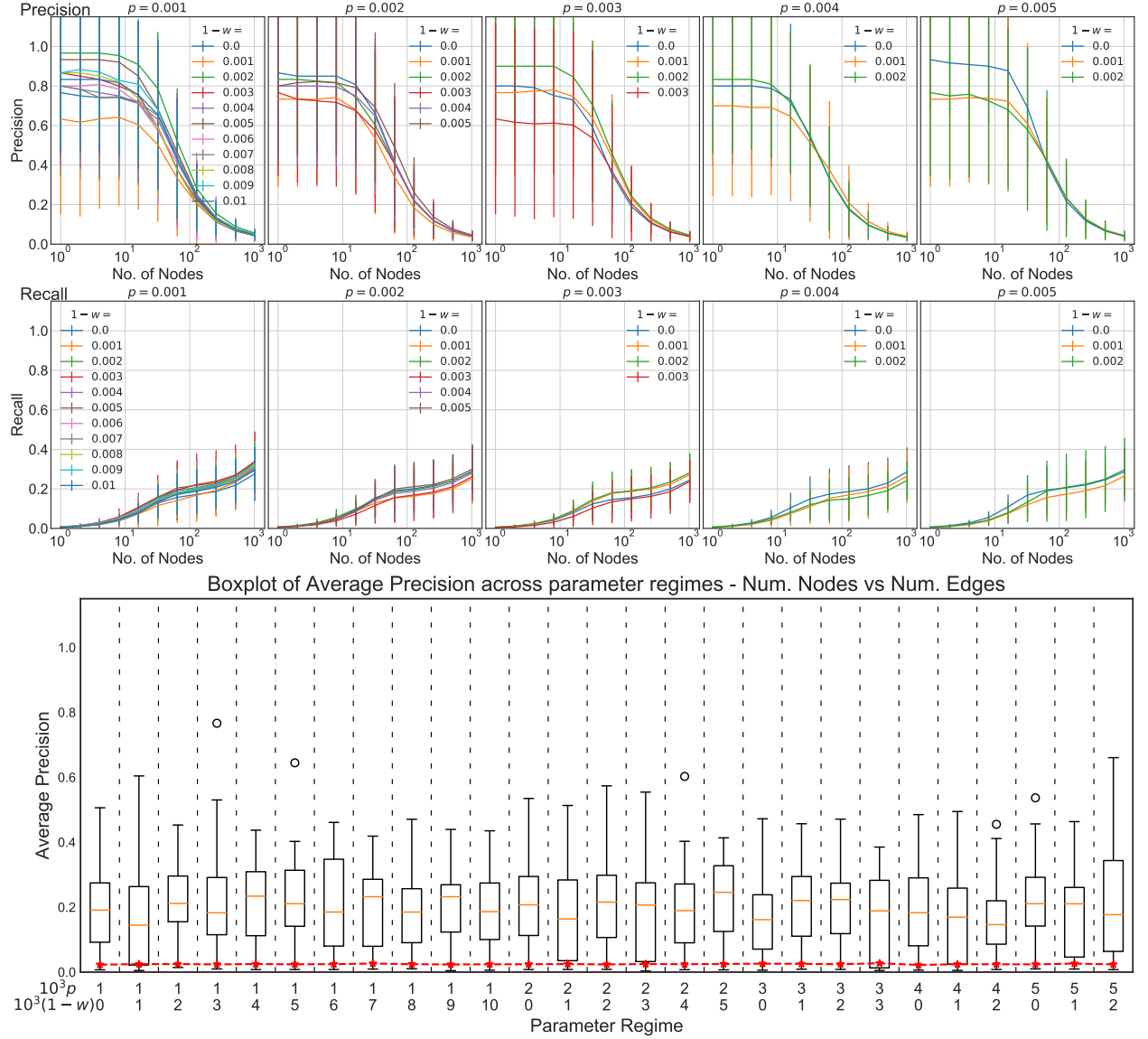


Figure 22: Performance measurements for each of the parameter regimes using the Oddball Number of Nodes vs Number of Edges feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one standard error over the relevant networks. The final panel shows a boxplot of the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

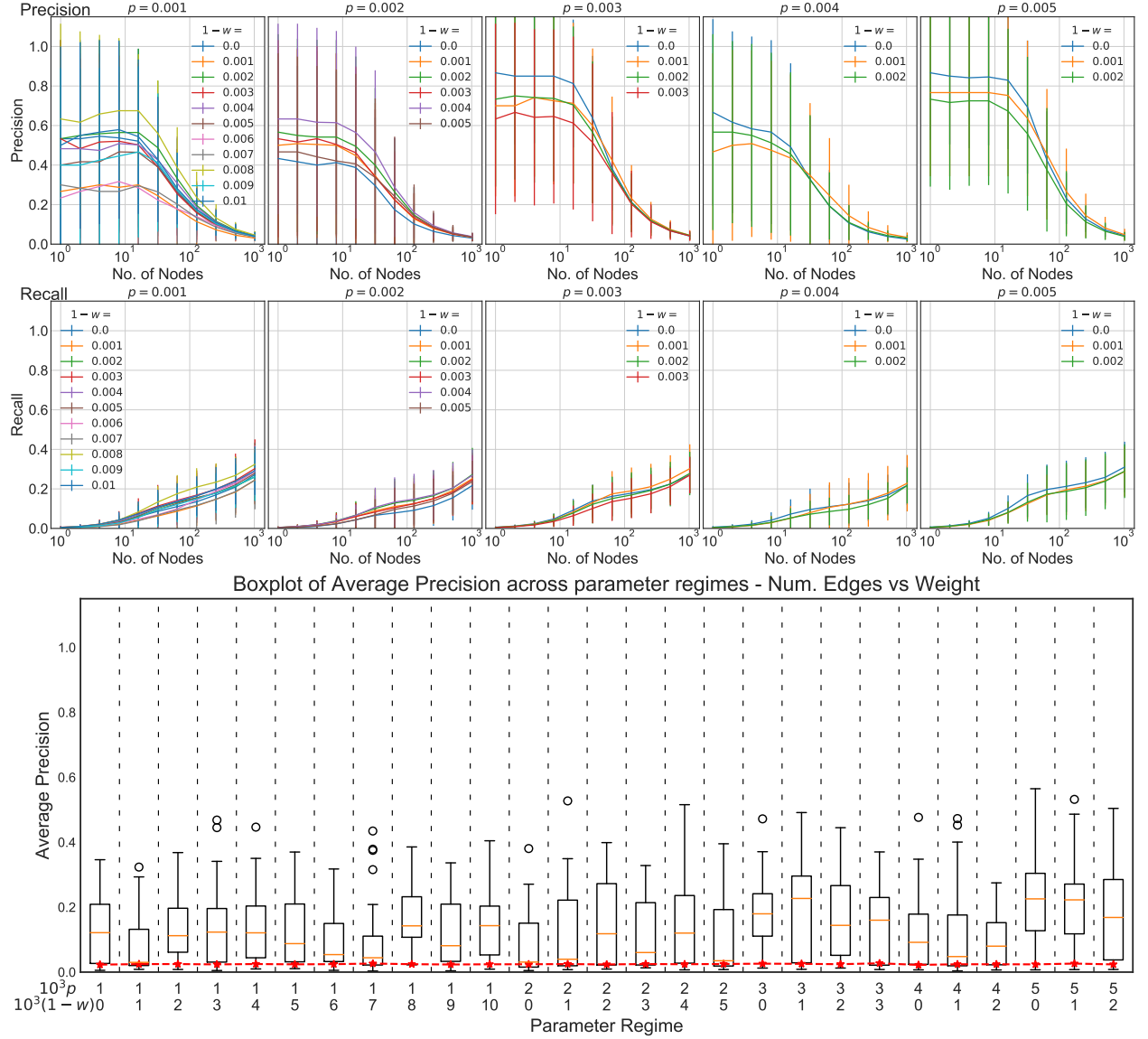


Figure 23: Performance measurements for each of the parameter regimes using the Oddball Number of Edges vs Weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

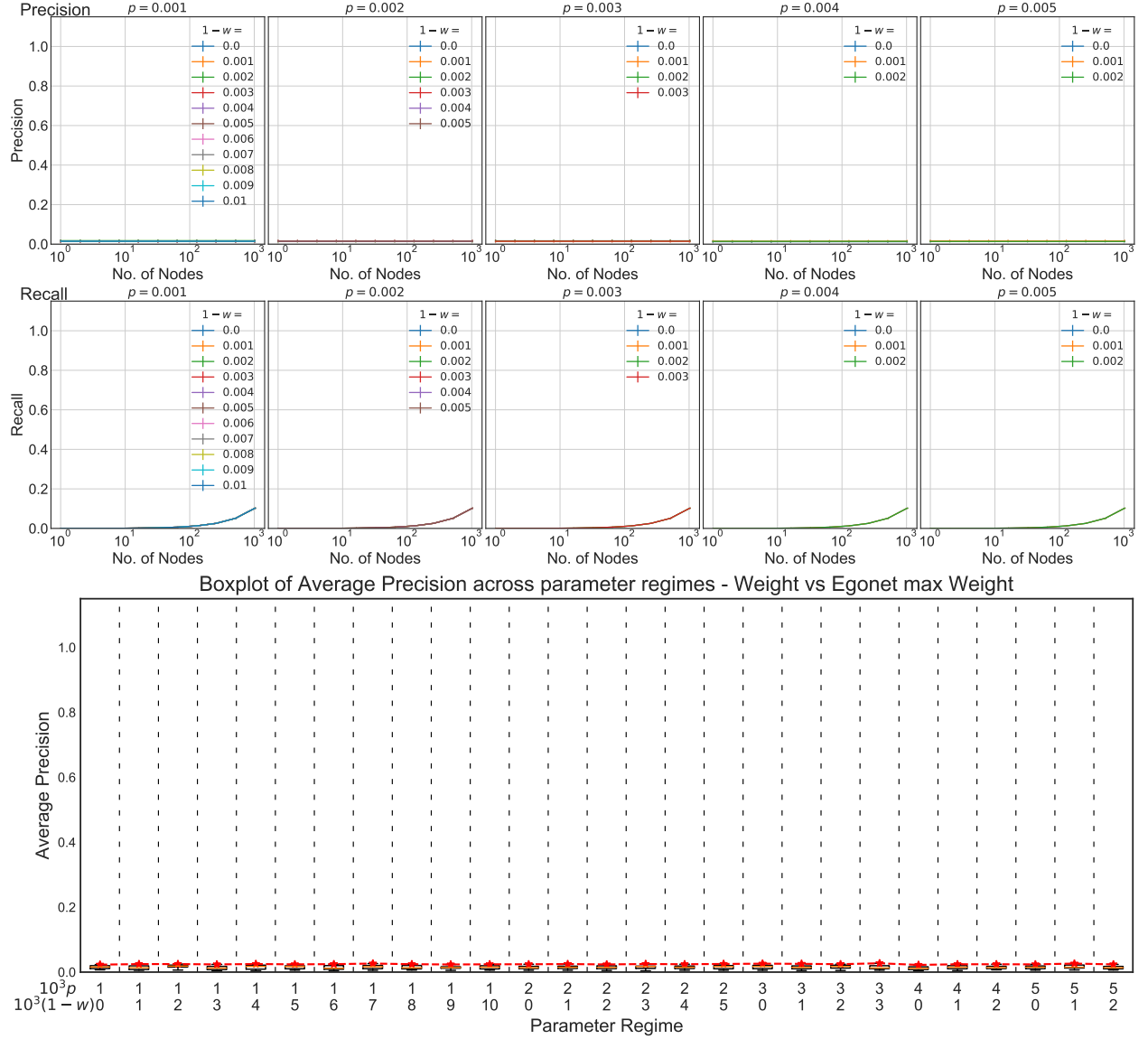


Figure 24: Performance measurements for each of the parameter regimes using the Oddball Weight vs Egonet Maximum Weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

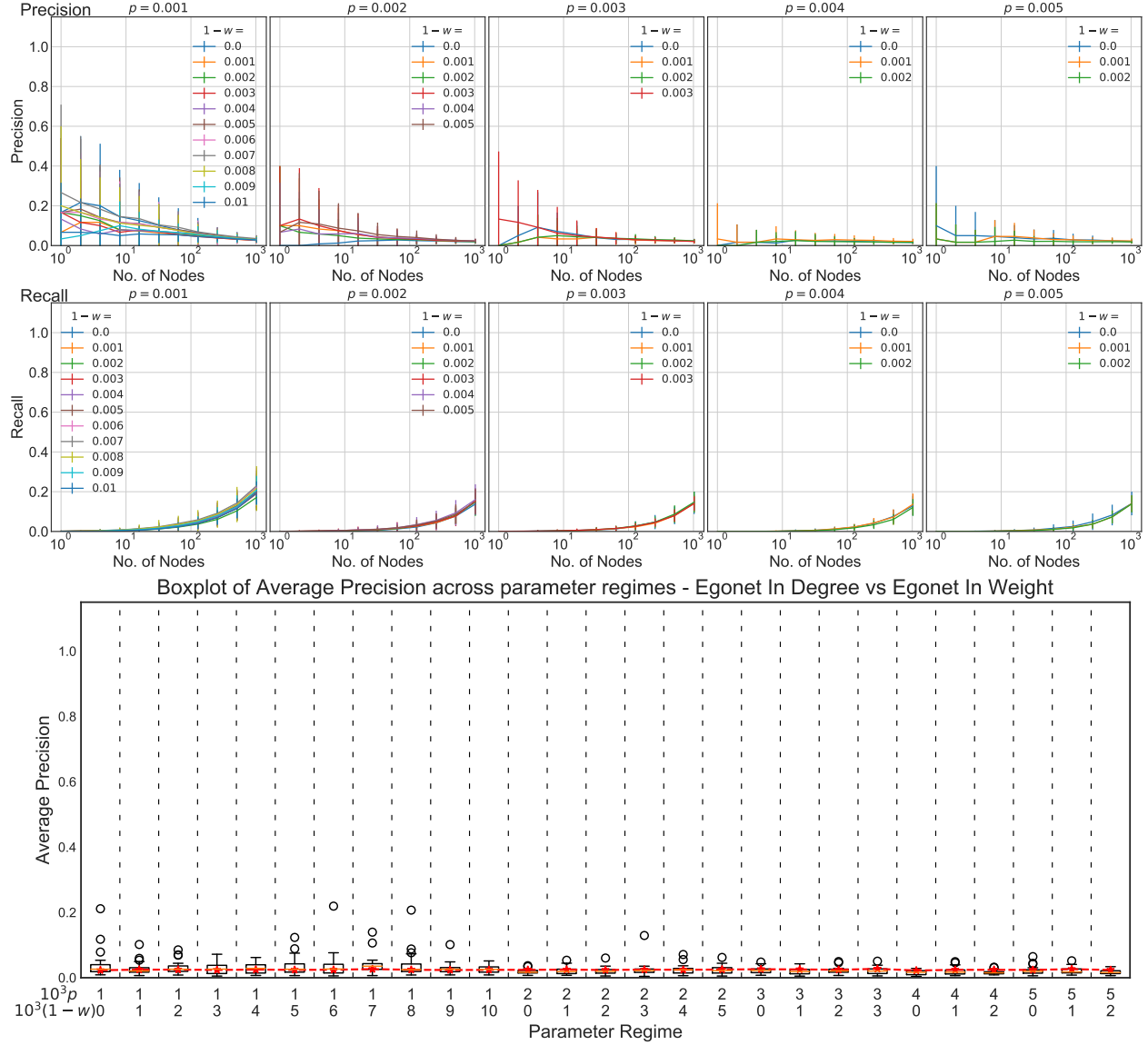


Figure 25: Performance measurements for each of the parameter regimes using the Oddball Egonet In-Degree vs Egonet In-Weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

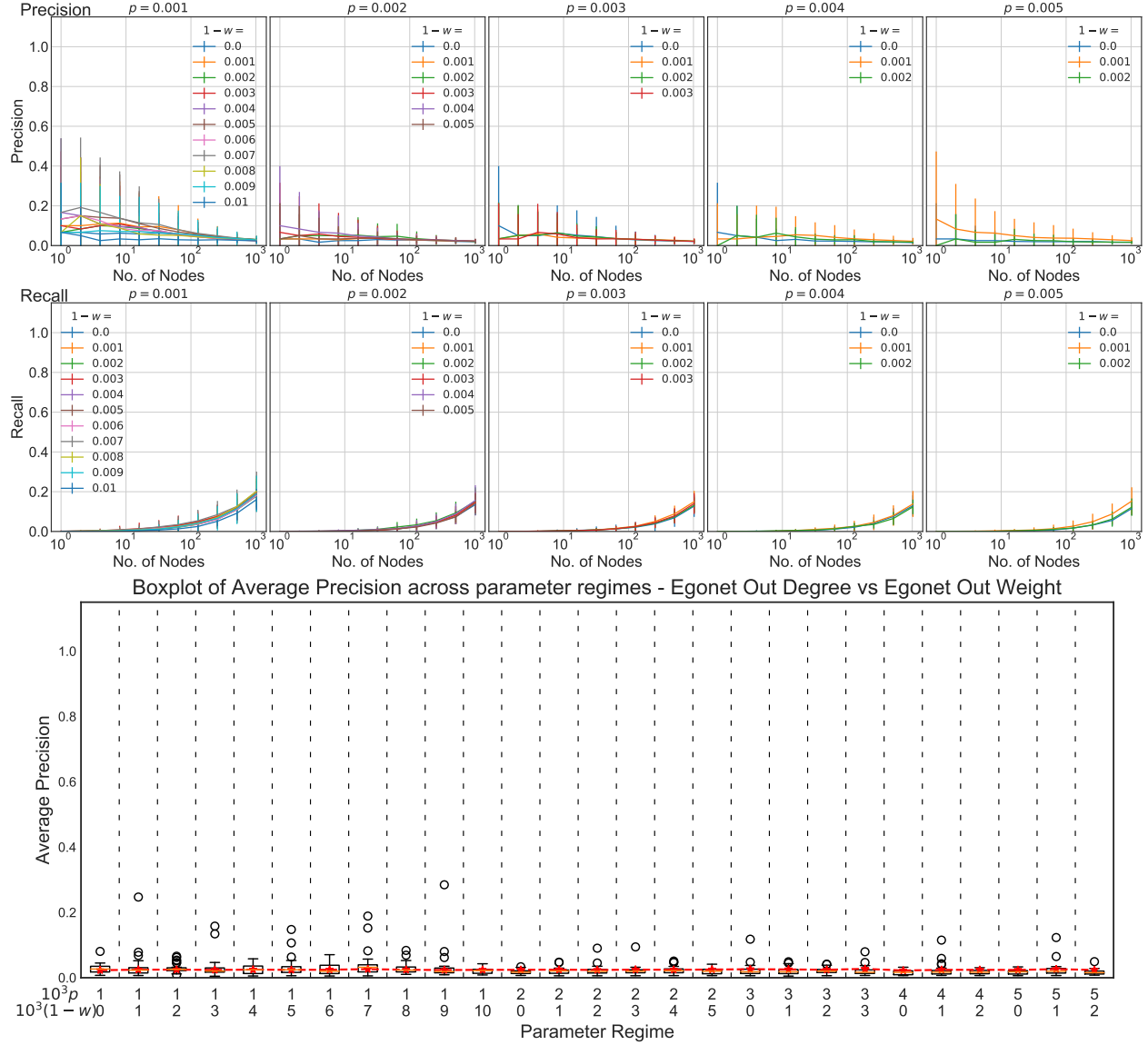


Figure 26: Performance measurements for each of the parameter regimes using the Oddball Egonet Out-Degree vs Egonet out-weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

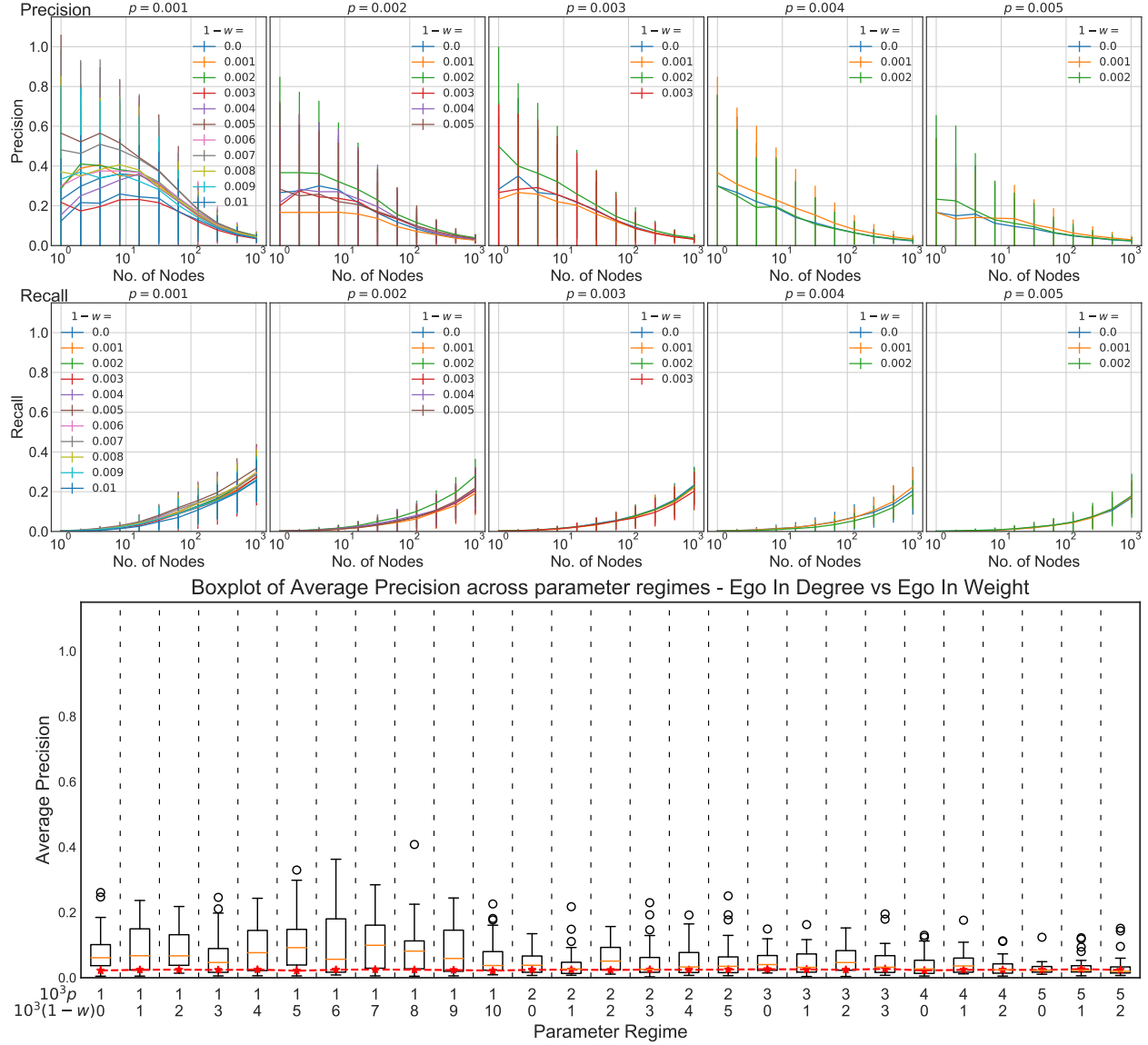


Figure 27: Performance measurements for each of the parameter regimes using the Oddball Ego In-Degree vs Ego In-Weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

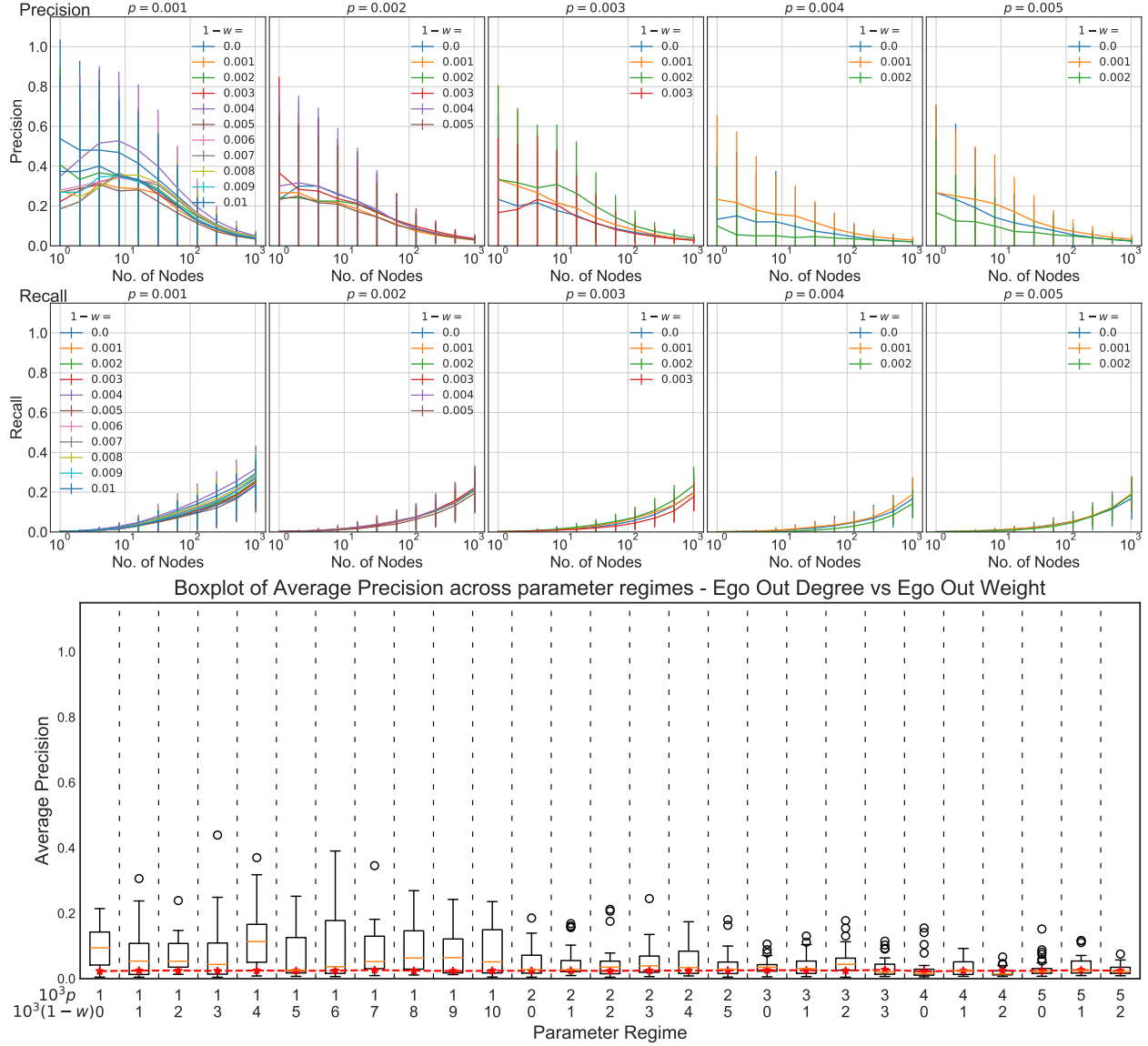


Figure 28: Performance measurements for each of the parameter regimes using the Oddball Ego Out-Degree vs Ego Out-Weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

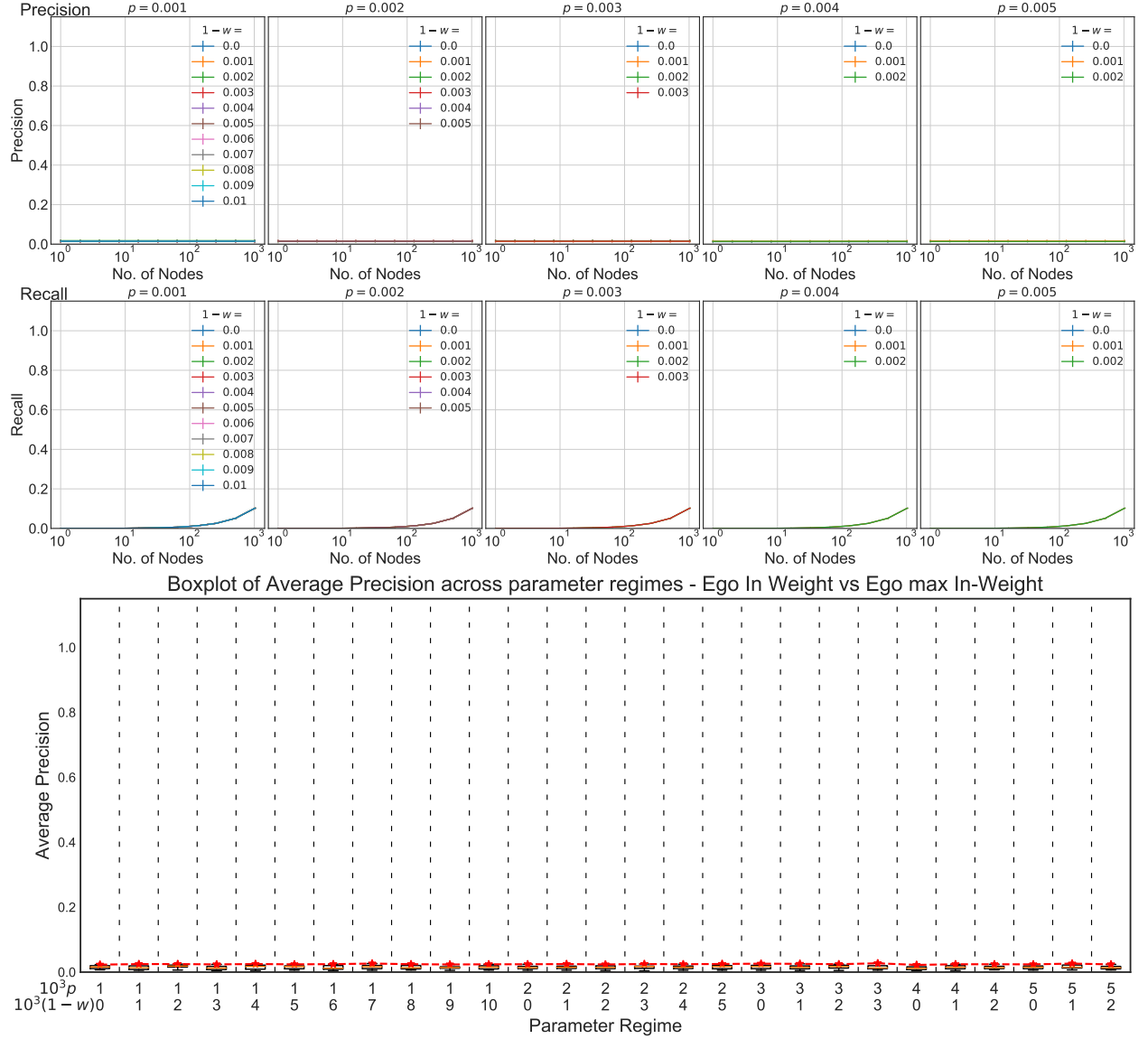


Figure 29: Performance measurements for each of the parameter regimes using the Oddball Ego Out-Weight vs Ego Maximal In-Weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

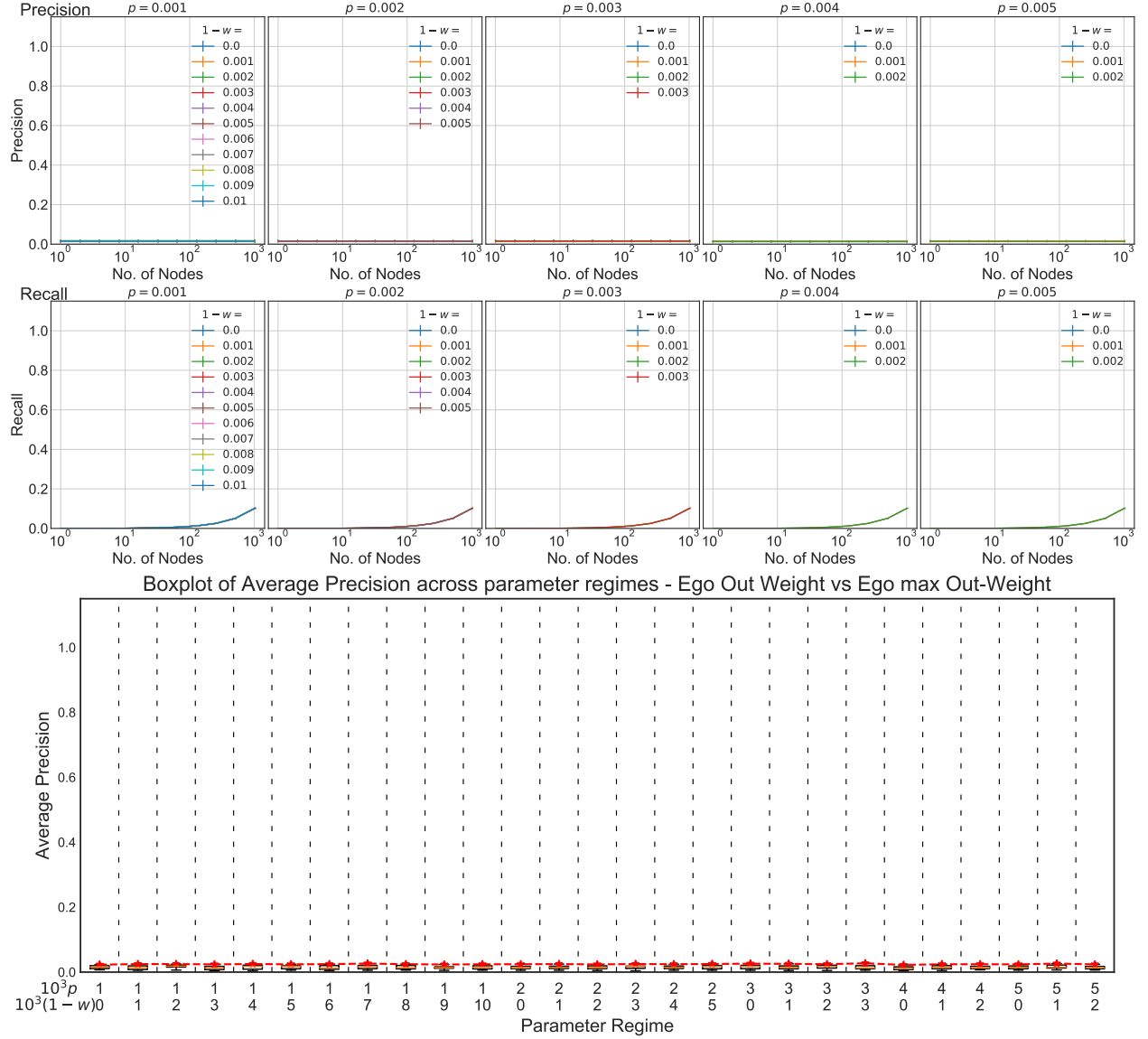


Figure 30: Performance measurements for each of the parameter regimes using the Oddball Ego out-Weight vs Ego maximal Out-Weight feature. The first pair of rows shows the mean precision and mean recall respectively with error bars that are one one standard error over the relevant networks. The final panel shows a boxplot of the the average precision over the test networks. The red starred line indicates the largest percentage of anomalies expected at random using this measure [51].

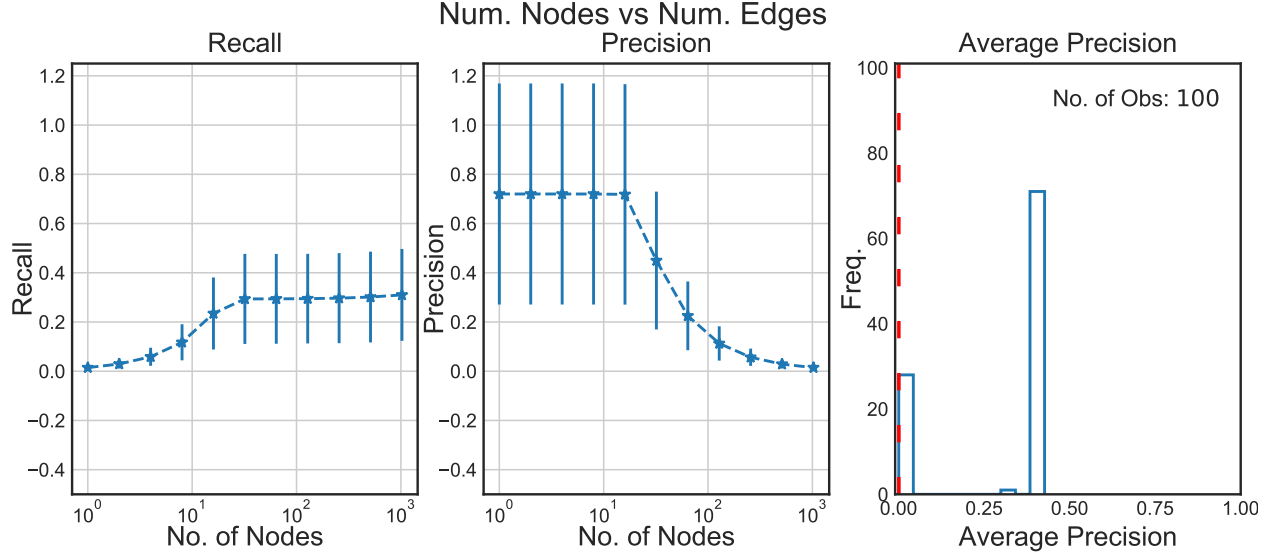


Figure 31: The performance, for each of the parameter regimes using the Number of Nodes vs Number of Edges feature. Performance is measured in three different ways. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons.

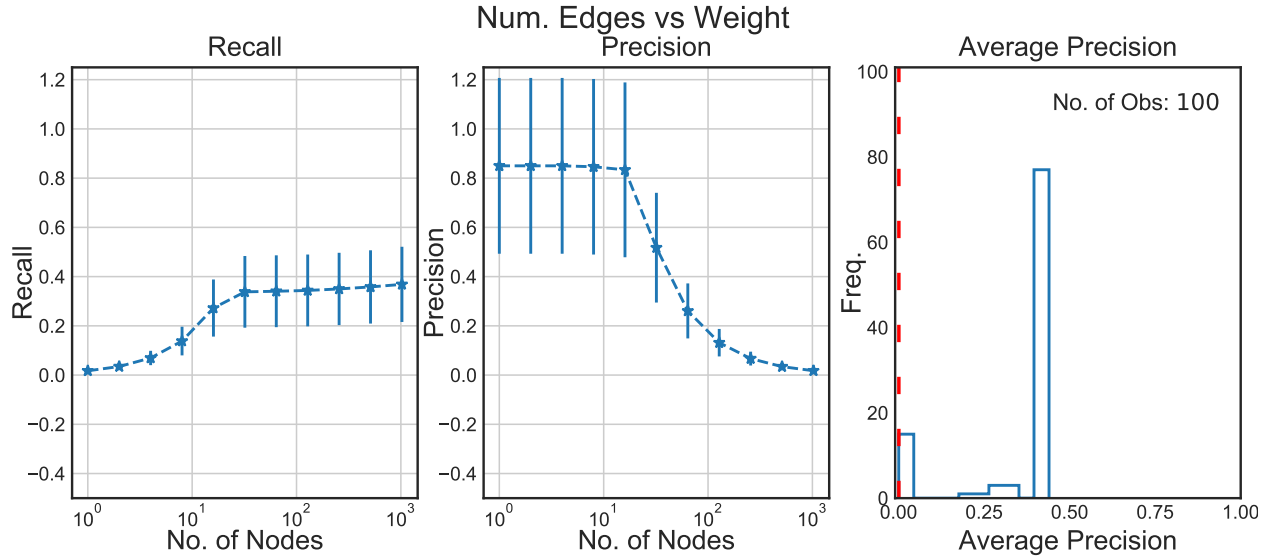


Figure 32: The performance from scikit-learn [51], for each of the parameter regimes using the Number Edges and Weight feature. First (left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons. For ease we thus display the exact number of observations in the top right corner.

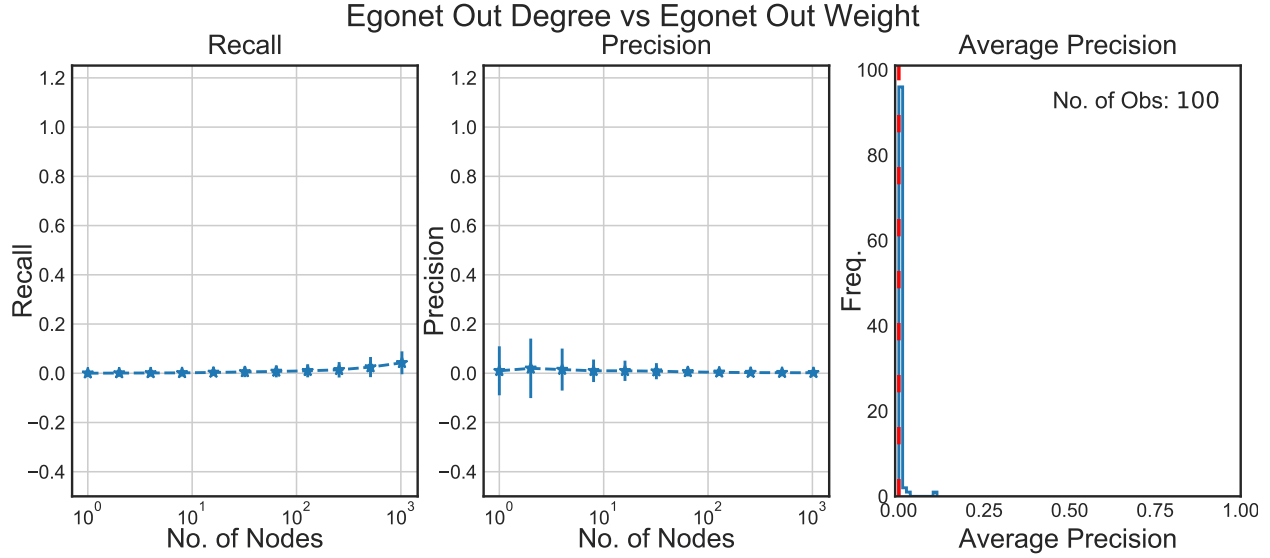


Figure 33: The performance from scikit-learn [51], for each of the parameter regimes using the Egonet Out-Degree and Egonet Out-Weight feature. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons. For ease we thus display the exact number of observations in the top right corner.

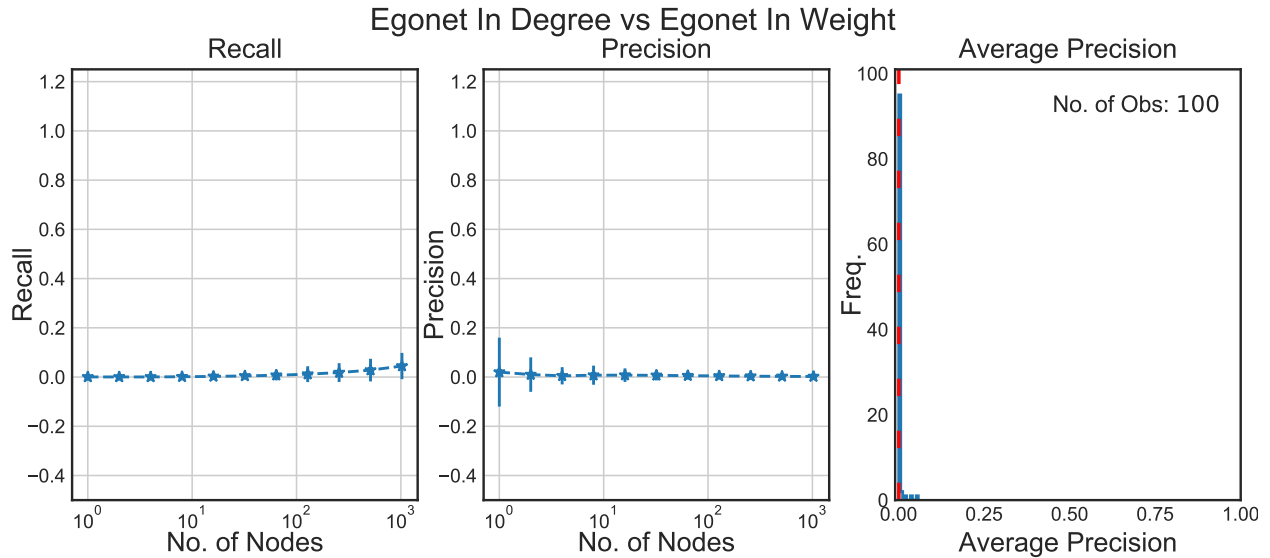


Figure 34: The performance from scikit-learn [51], for each of the parameter regimes using the Egonet In-Degree and Egonet In-Weight feature. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons. For ease we thus display the exact number of observations in the top right corner.

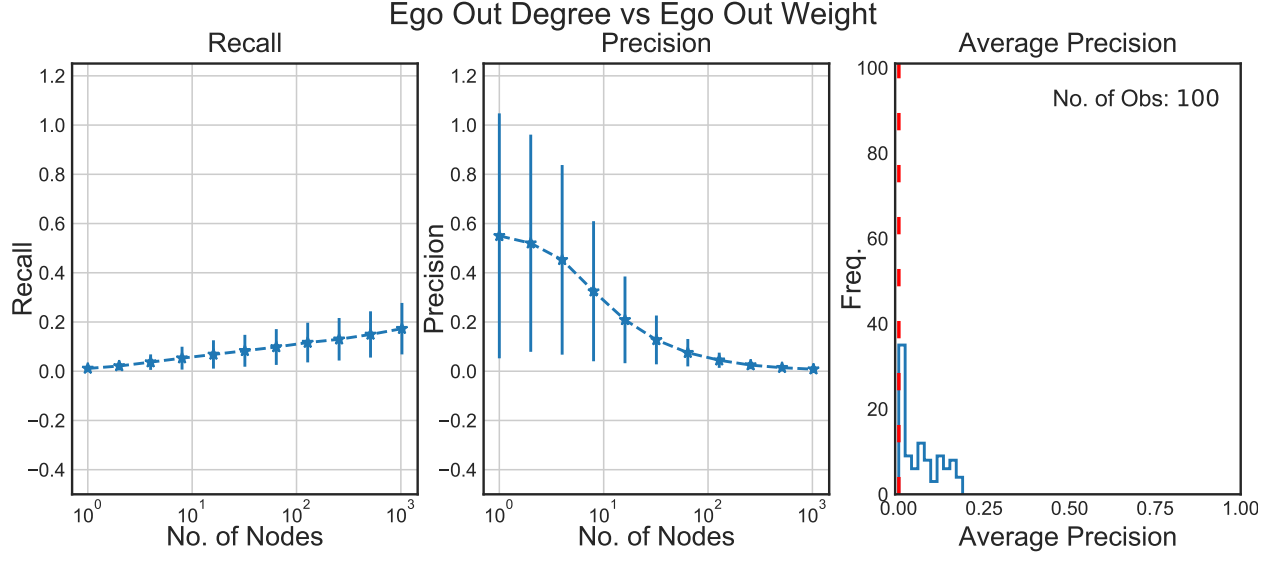


Figure 35: The performance from scikit-learn [51], for each of the parameter regimes using the Ego Out-Degree and Ego Out-Weight feature. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons.

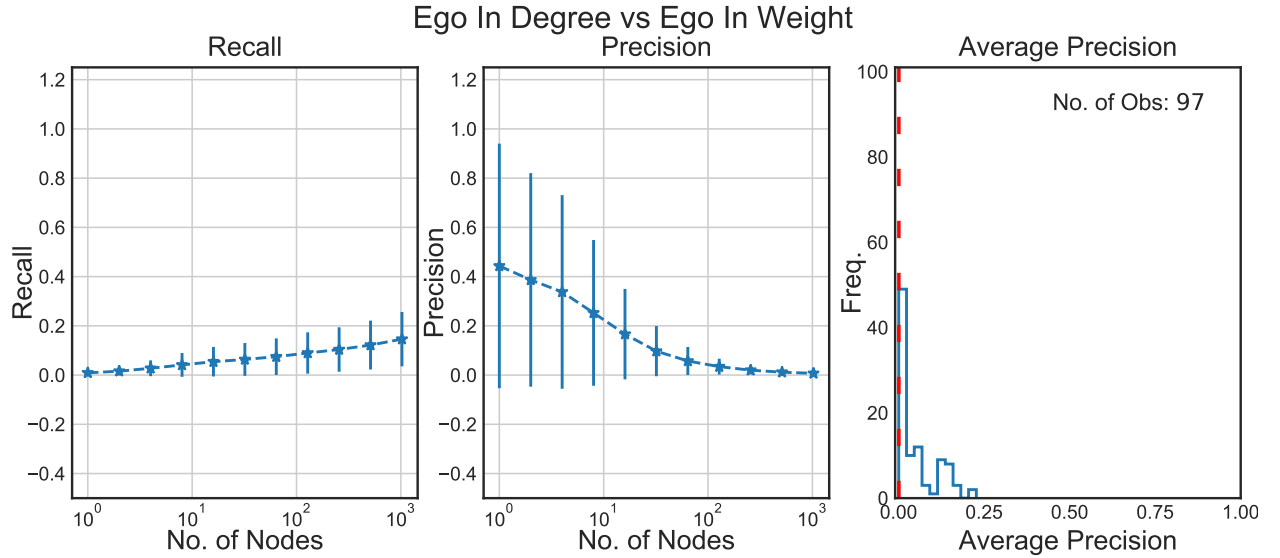


Figure 36: The performance from scikit-learn [51], for each of the parameter regimes using the Ego In-Degree and Ego In-Weight feature. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons. For ease we thus display the exact number of observations in the top right corner.

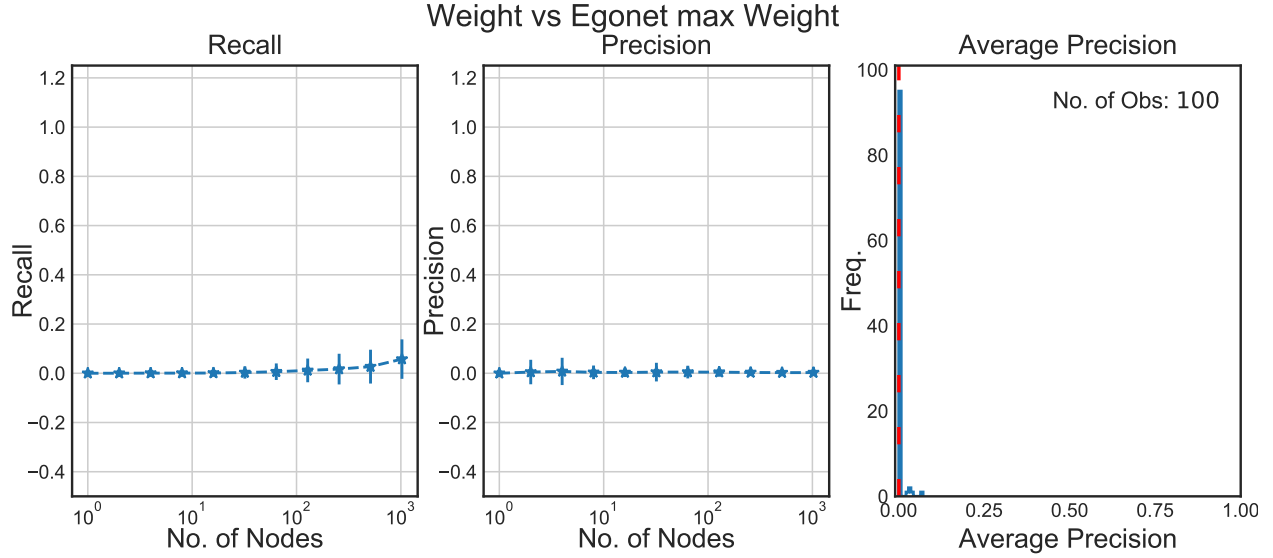


Figure 37: The performance from scikit-learn [51], for each of the parameter regimes using the Egonet weight and Egonet Maximum Weight. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons. For ease we thus display the exact number of observations in the top right corner.

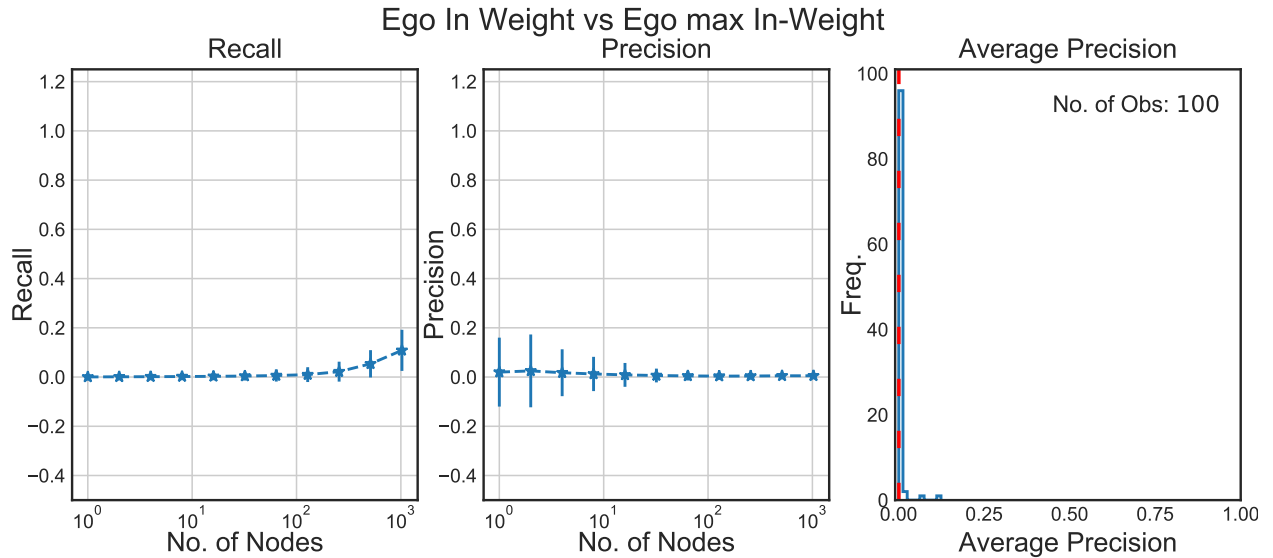


Figure 38: The performance from scikit-learn [51], for each of the parameter regimes using the Ego In-Weight and Ego Maximum In-Weight. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons. For ease we thus display the exact number of observations in the top right corner.

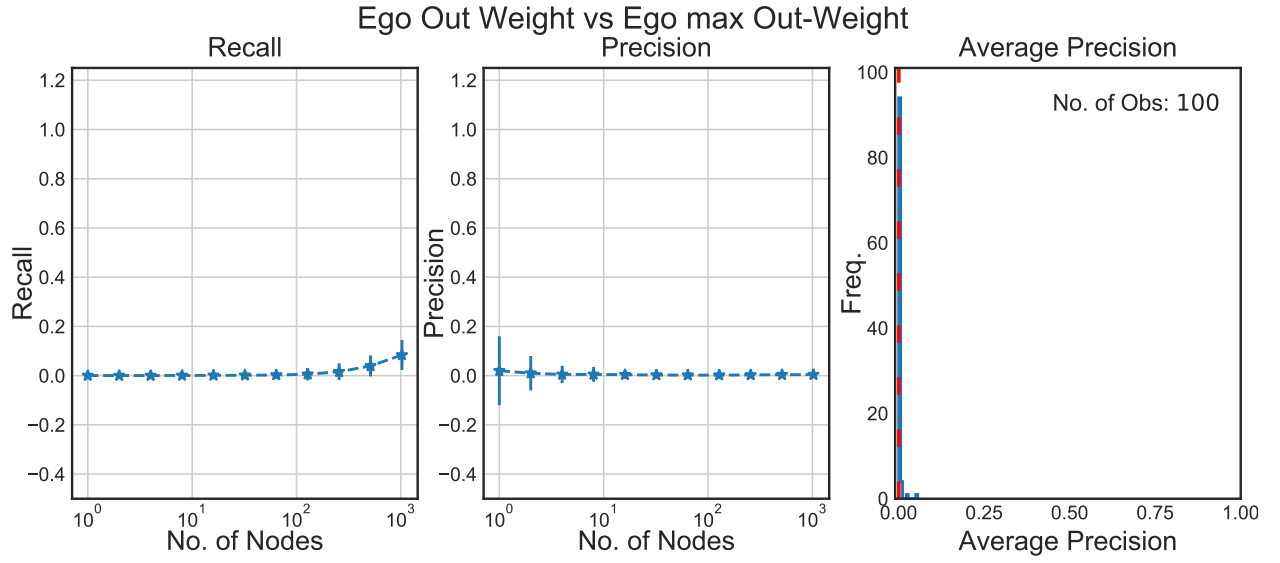


Figure 39: The performance from scikit-learn [51], for each of the parameter regimes using the Ego Out-Weight and Ego Maximal Out-Weight. First (far left): the average performance (error bars denoting 1 standard error) with respect to the recall when we consider the top nodes in our ranking. Second (middle): the average performance (error bars denoting 1 standard error) with respect to precision. Finally (far right): the distribution of the average precision over our 100 test networks. When the performance on a measure resulted in Infs, then we removed the network from all of the comparisons. For ease we thus display the exact number of observations in the top right corner.

Node	Total Degree	Num. of Heavy Edges	Num. of Anom. nodes in 1 hop	Num. of Anom. nodes in 2 hop	Figure name
14708	43	4	0	0	Fig. 40
22745	43	2	0	0	Fig. 41
32251	31	4	0	0	Fig. 42
32545	50	6	0	1	Fig. 43
34446	42	3	0	1	Fig. 44
35493	31	3	0	0	Fig. 45
36434	33	2	0	0	Fig. 46

Table 1: Basic statistics for a subset of nodes that are identified as anomalous, but were not embedded in the Accenture network. Note that, in this context, heavy means a weight greater than 1400 which is 2 standard deviations above the mean.

H Exploring Anomalous Non-Embedded Nodes

In the main body, we explored the ability of our method to detect anomalies that we did not embed. To this end, we visually inspected the nodes which are declared an anomaly by RANDOM FOREST but were not part of a planted anomaly, in an example network generated by the synthetic network procedure constructed by Accenture. In this section, we present additional plots and statistics from this analysis. To that end, in Figures 40, 41, 42, 43, 44, 45 and 46 we present the complete 2-hop snowball sampled plot for each network, rather than a restricted version presented in the main body. Finally, in Table 1 we present the statistics for the graphs.

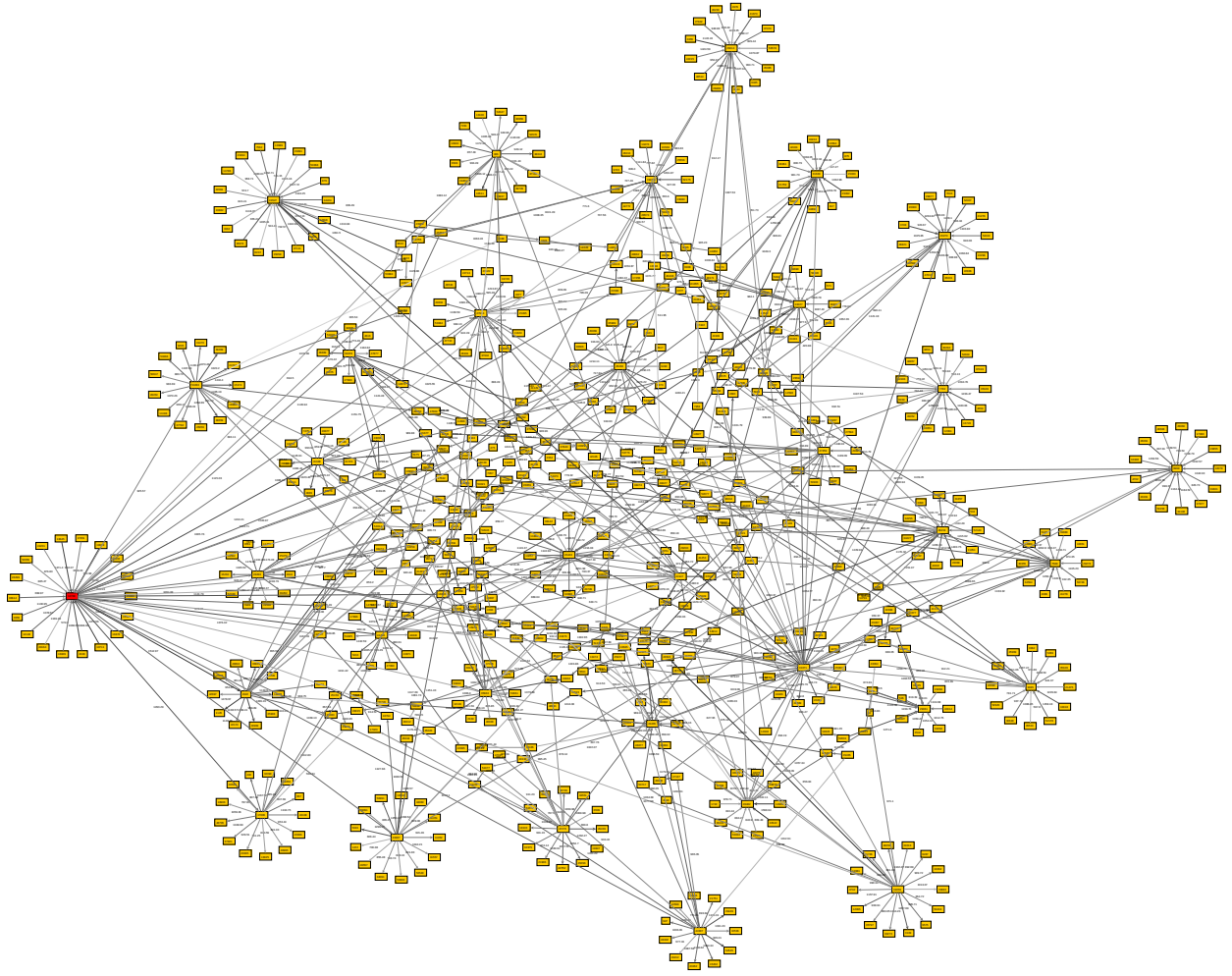


Figure 40: Network plot of the 2-hop snowball sample (ignoring edge direction) of node 14,708 in a realisation of the Accenture synthetic model. This node was selected as it is identified as anomalous in the random forest but was not planted in the network.

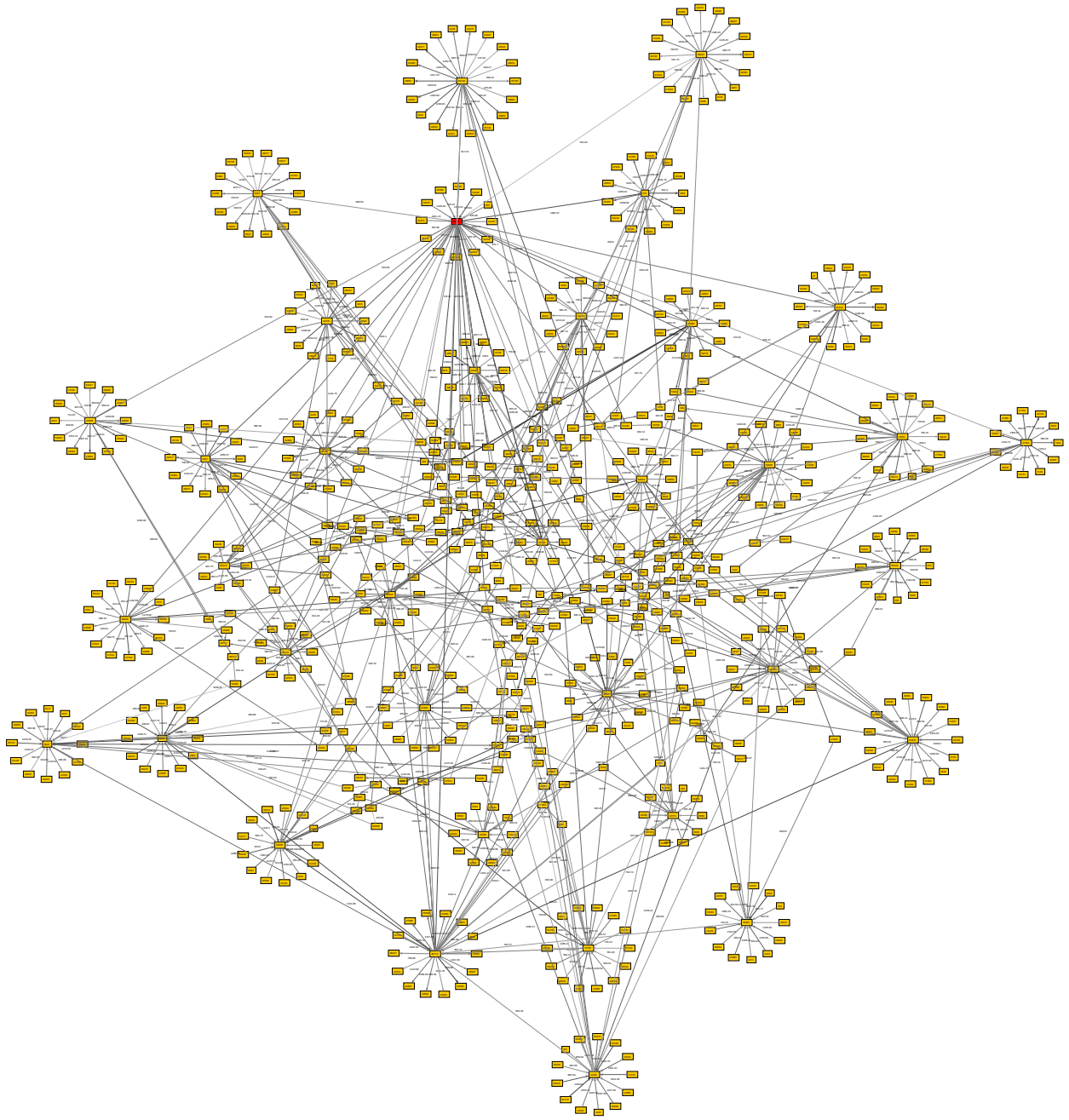


Figure 41: Network plot of the 2 hop snowball sample (ignoring edge direction) of node 22,745 in a realisation of the Accenture synthetic model. This node was selected as it is identified as anomalous in the random forest but was not planted in the network.

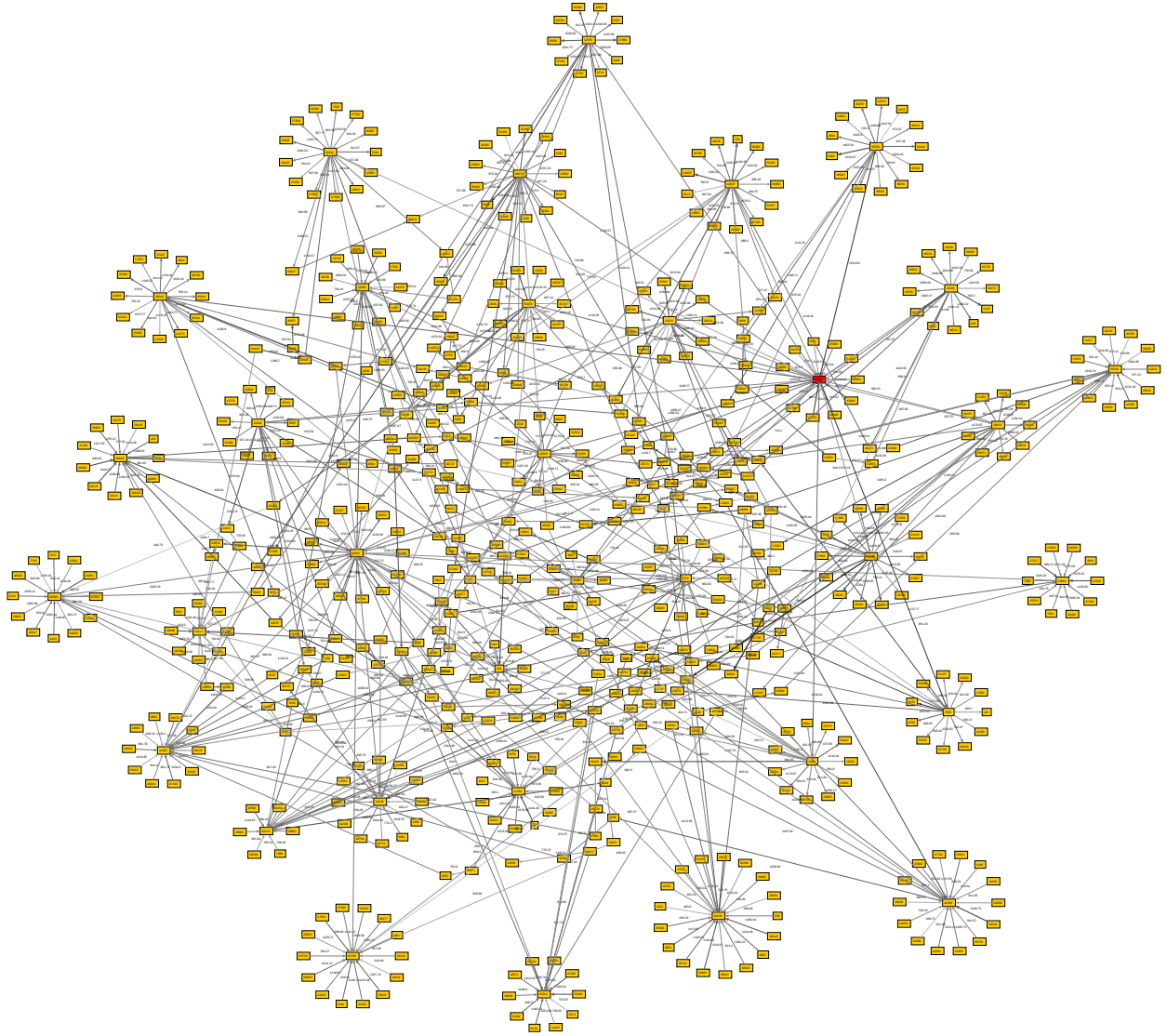


Figure 42: Network plot of the 2 hop snowball sample (ignoring edge direction) of node 32,251 in a realisation of the Accenture synthetic model. This node was selected as it is identified as anomalous in the random forest but was not planted in the network.

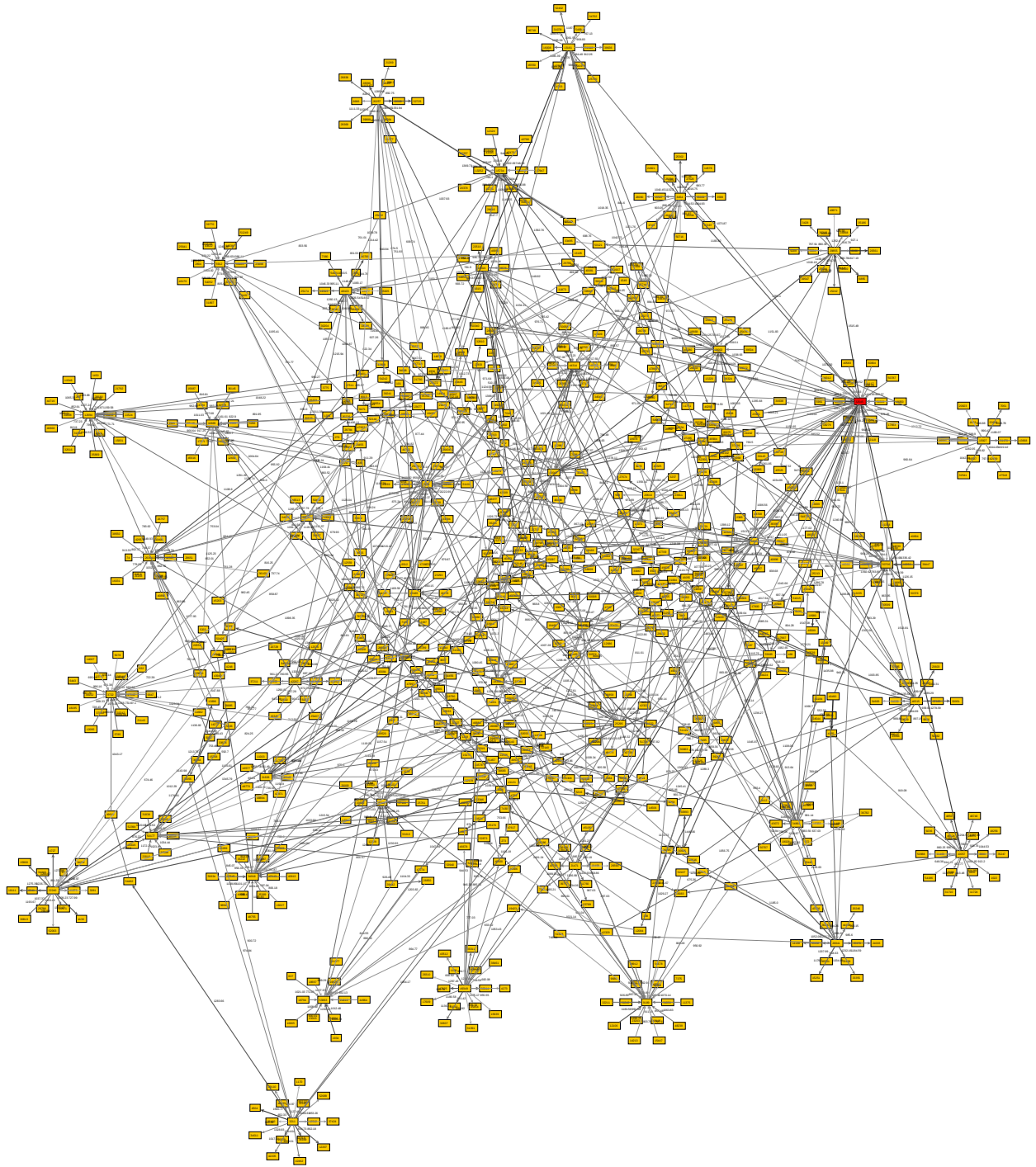


Figure 43: Network plot of the 2 hop snowball sample (ignoring edge direction) of node 32,545 in a realisation of the Accenture synthetic model. This node was selected as it is identified as anomalous in the random forest but was not planted in the network.

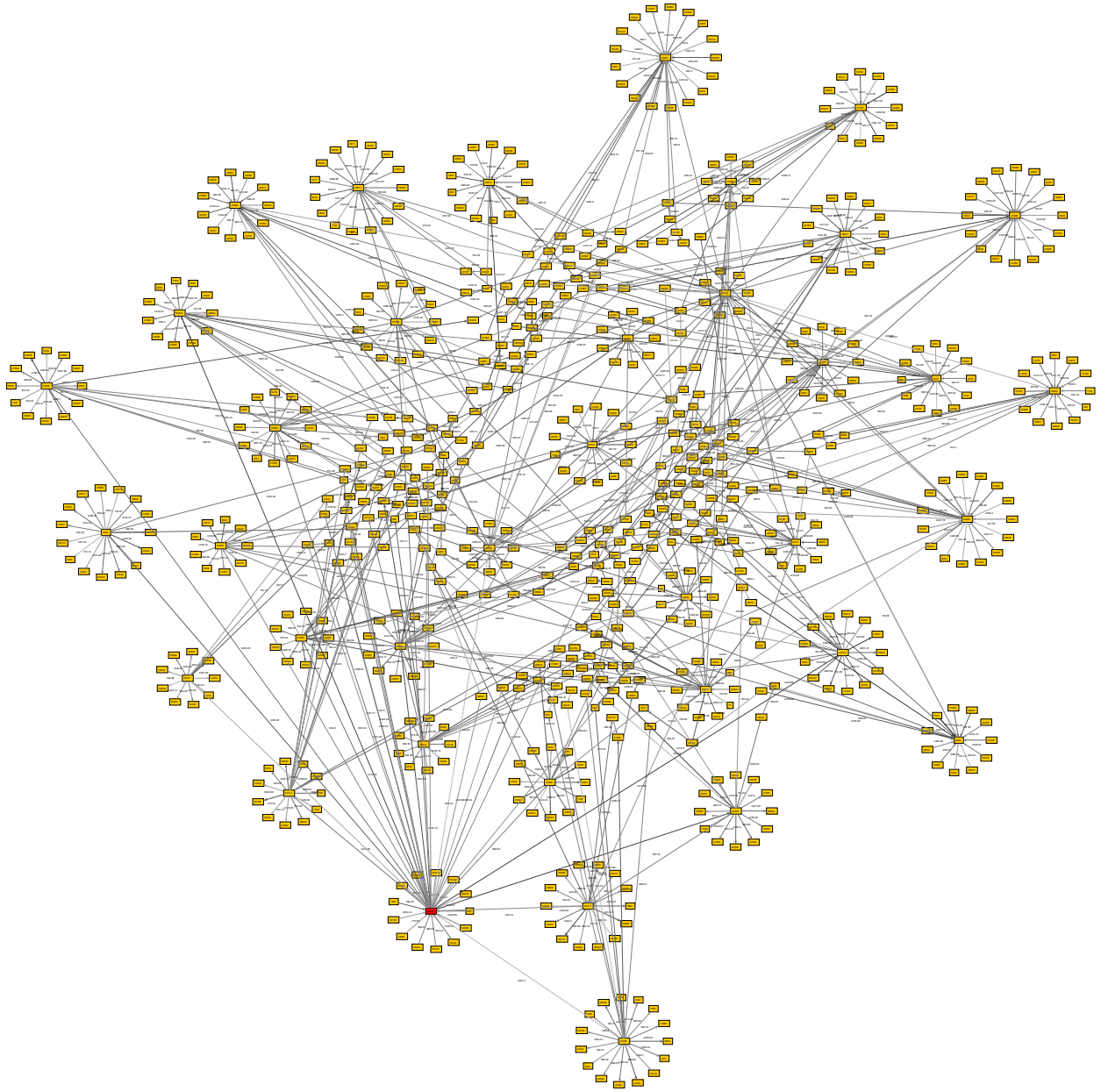


Figure 44: Network plot of the 2 hop snowball sample (ignoring edge direction) of node 34,446 in a realisation of the Accenture synthetic model. This node was selected as it is identified as anomalous in the random forest but was not planted in the network.

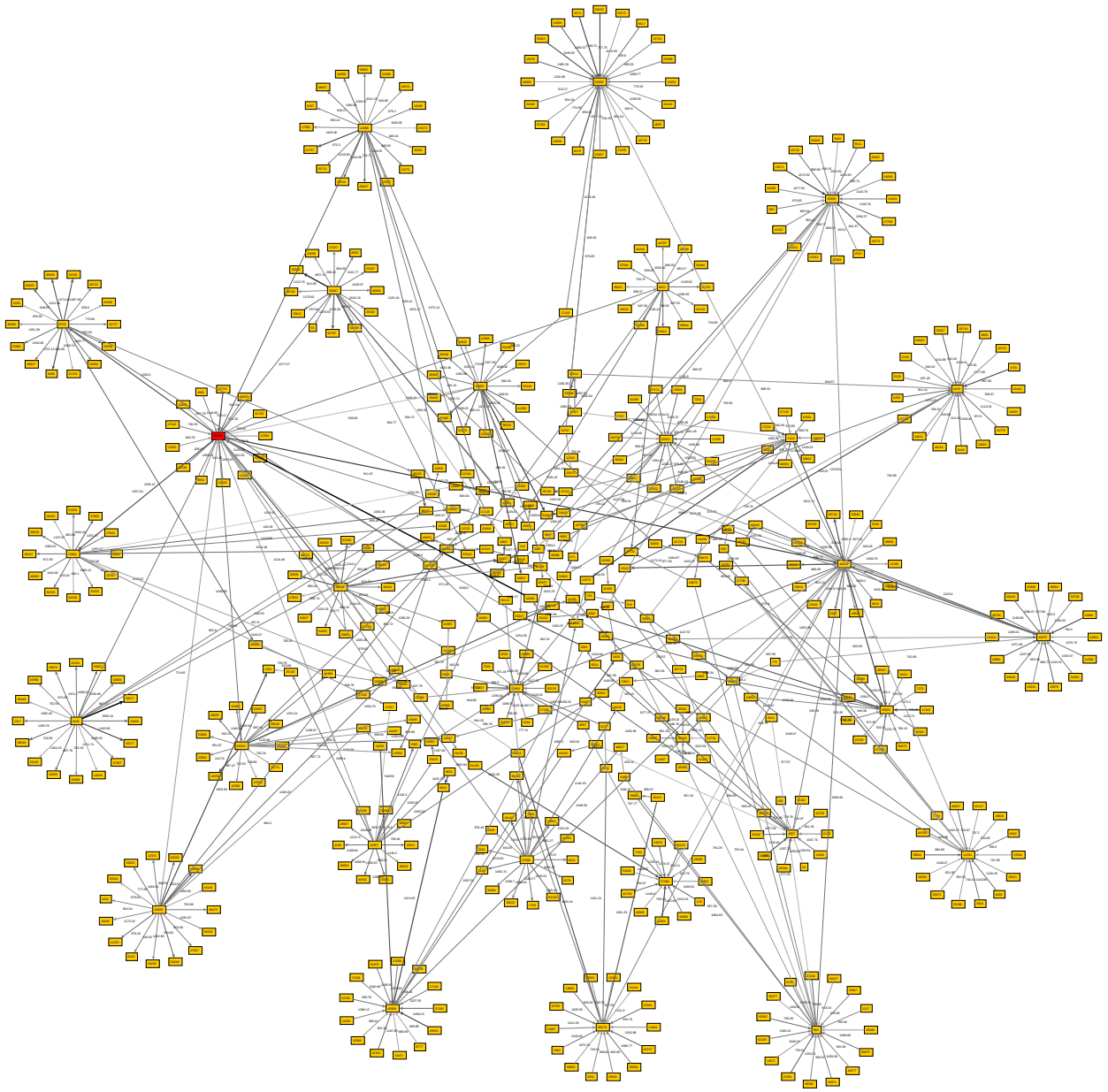


Figure 45: Network plot of the 2 hop snowball sample (ignoring edge direction) of the node 35,493 in a realisation of the Accenture synthetic model. This node was selected as it is identified as anomalous in the random forest but was not planted in the network.

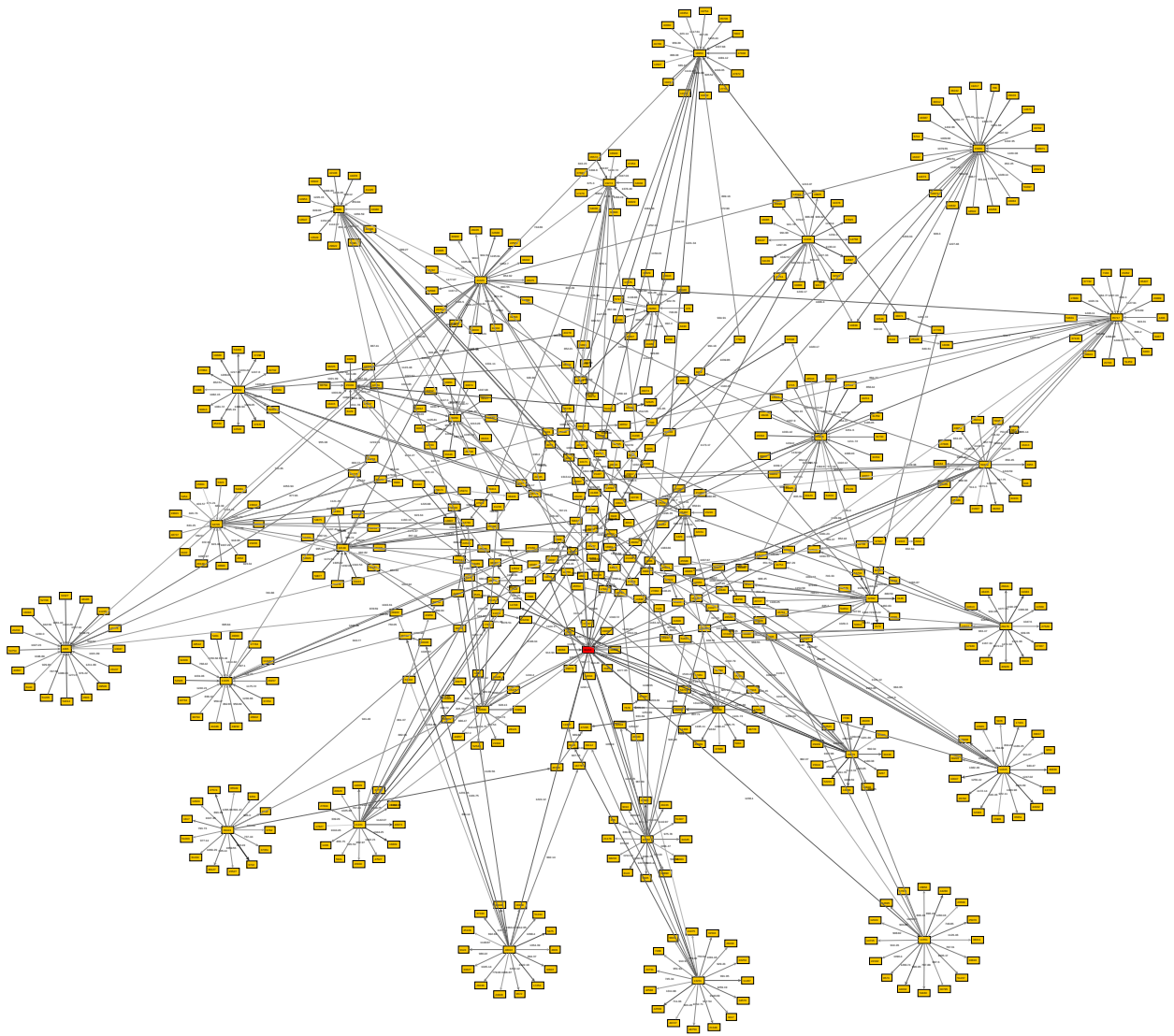


Figure 46: Network plot of the 2 hop snowball sample (ignoring edge direction) of node 36,434 in a realisation of the Accenture synthetic model. This node was selected as it is identified as anomalous in the random forest but was not planted in the network.